

基于傅里叶—小波预测与 MPC 滚动优化的微网储能购电优化策略

摘 要

本文针对微网与外部电网电力调控策略问题，在小区负载、光伏发电和储能设备约束下，建立从数据拟合、日前计划到滚动调整的全流程购电优化模型。

针对问题一，同时构造原始数据直接优化与傅里叶—小波—正弦拟合优化两条路径，前者作为理论基准，后者作为提交策略。采用二阶傅里叶级数和三级 Haar 小波拟合负荷，以截断幂正弦函数拟合光伏，并通过线性规划求解购电与储能调度。负荷拟合的 MAE 为 111.65 kW、RMSE 为 158.40 kW、 R^2 为 0.9737；光伏拟合的 R^2 为 0.9996。

针对问题二，建立滚动预测—购电优化模型。以年度周期、星期效应和近期历史信息预测负荷，通过严格前向贝叶斯优化选择七维参数，并依据报童模型设置 80% 分位数安全裕度；随后采用 48 小时滚动线性规划制定日前策略，真实供电缺口按 5 倍电价紧急购电补足。

针对问题三，建立多时点光伏预报驱动的 MPC-LP 模型，比较 6:00、12:00 和 18:00 新预报的调整价值。验证集选择由 0:00、6:00 和 12:00 组成的 S2 策略；测试集总费用为 270.82 万元，较仅 0:00 策略降低 12.78 万元，紧急购电量下降 44.36%。18:00 更新使费用上升，因此保留 6:00 和 12:00 更新，不再引入 18:00 预报。

针对问题四的实时波动电价，建立“傅里叶—小波—前日偏差修正”电价预测链及双框架购电优化模型。依次利用历史加权轮廓、傅里叶级数、Haar 小波残差重构和前日同刻偏差修正预测电价，并通过带早停机制的严格前向贝叶斯优化确定历史窗口、谐波阶数、小波层数和修正权重。测试集电价预测的 MAE 为 0.0643 元/kWh，MAPE 为 10.35%， R^2 为 0.9352。问题 4-2 测试集总费用为 298.85 万元；问题 4-3 采用 S2 策略后总费用为 286.70 万元，较问题 4-2 降低 12.15 万元，降幅为 4.07%。

本模型的特色在于：（1）傅里叶—小波组合拟合兼顾负荷的全局周期与局部波动；（2）年度谐波—星期效应—岭回归与贝叶斯优化有效提升滚动预测精度；（3）报童模型确定的安全裕度用于权衡计划购电成本与净负荷低估所引起的紧急购电风险；（4）MPC 多时点调整机制在预报精度提升与调整成本之间建立量化决策框架。

关键词：微网储能优化；傅里叶—小波预测；贝叶斯优化；线性规划；模型预测控制。

一、问题重述

1.1 问题背景

微网通过协调分布式光伏、储能与外部电网，实现本地供需平衡与购电成本控制。在非孤岛运行模式下，低价时段购电或利用富余光伏为储能充电，高价时段再由储能放电，因此核心决策是依据负荷、光伏、电价与储能状态合理安排购电和充放电，使供电可靠性与经济性同时得到满足。

1.2 问题重述

问题一：在日内电价与负荷曲线固定、已知当日光伏预测的条件下，建立满足供需平衡及日初日末储能电量一致的计划购电模型，并给出指定时段购电量、充放电量和全天费用。

问题二：当负荷与光伏随日期变化且日前无法获得真实值时，利用历史信息制定 0:00 计划购电策略；实际供电不足部分按当时电价的 5 倍紧急购电，并评价全年计划与紧急购电结果。

问题三：利用 0:00、6:00、12:00 和 18:00 发布的光伏预报，在既有计划上滚动调整购电与储能决策，同时计入增购溢价、减购违约费用和紧急购电成本，并判断哪些新增预报时点值得采用。

问题四：进一步考虑外网电价实时波动，建立动态电价预测与购电优化模型，并分别在问题二的滚动 LP 与问题三的 MPC-LP 框架下重新计算和比较策略效果。

二、问题分析

四个问题可统一写成带状态变量的序贯决策：外生量为负荷、光伏与电价，控制量为购电、充电和放电，储能电量是连接相邻时段的状态。问题一给定外生量求确定性最优解；问题二以日前预测和误差分位数替代未知外生量；问题三按新预报更新条件信息并实施滚动时域控制；问题四进一步把电价也作为随机外生量预测。由此，四问共享同一能量平衡与 SOC 状态方程，只在信息集、目标系数和决策更新频率上递进。

建模主线为“预测—风险修正—优化—回放”：先构造条件预测，再把净负荷误差映射为安全裕度，随后求解线性规划或 MPC，并用真实数据结算。该结构使预测误差不只停留在统计指标上，而能通过紧急购电量和总费用评价其决策影响。

三、模型假设

为在保留微网主要运行特征的基础上简化模型，作出如下假设：

1. 时段离散假设： 将一天划分为 144 个 10 分钟时段，认为负荷功率、光伏功率和外网电价在每个时段内保持不变，各时段功率可按固定时间间隔转换为电量。

2. 功率平衡假设： 忽略微网内部线路传输损耗，认为外网购电、光伏发电和储能放电共同满足小区负荷、储能充电及弃光需求。

3. 储能运行假设： 储能设备的容量上下限、最大充放电功率以及 90% 的充放电效率在研究期内保持不变；忽略储能自放电、设备老化和循环寿命损耗。通过极小的充放电吞吐量惩罚避免无意义的同时充放电。

4. 光伏消纳假设： 光伏发电优先用于满足负荷或为储能充电；当光伏出力超过负荷与储能可接纳能力时，允许弃光，且不额外计入弃光处理费用。

5. 外网供电假设： 外网正常购电和紧急购电均不存在容量上限；当真实供电量低于负荷需求时，紧急购电能够在当前时段立即完成，并按照题目规定的倍数电价结算。

6. 短期规律稳定假设： 在有限历史窗口内，小区负荷、光伏出力和实时电价的周期结构及星期效应变化相对平缓，因此可利用近期历史数据和周期特征预测未来曲线。

7. 预测误差分布稳定假设： 在相同发布时间和相同时段下，近期历史净负荷预测误差的经验分布能够近似反映当前预测风险，因此可利用其经验分位数确定安全裕度。该假设不要求预测误差严格独立同分布。

8. 预报插值假设： 附件 3 给出的整点光伏预报在相邻整点之间连续变化，可采用线性插值转换为 10 分钟分辨率；预报发布时刻的实际光伏功率可作为插值起点。

9. 信息因果性假设： 制定某一时刻的购电策略时，只能使用该时刻以前已经观测或发布的信息，不允许使用未来真实负荷、光伏功率和电价。训练集用于拟合，验证集用于参数和策略选择，测试集仅用于最终评价。

四、符号说明

注：一天被划分为 144 个 10 分钟时段，记 $t = 0, 1, \dots, 143$ ， $\Delta t = \frac{1}{6} \text{ h}$ ，第 t 个时段对应的小时变量为 $h_t = t \Delta t$ 。

符号	说明	单位
t	时段编号， $t=0,1,\dots,143$	—
Δt	时段长度， $\Delta t=1/6$	h
h_t	第 t 个时段对应的小时数	h
L_t	第 t 个时段的负荷功率	kW
L_t^F	第 t 个时段的傅里叶拟合负荷功率	kW
L_t^{FW}	第 t 个时段的傅里叶-小波组合拟合负荷功率	kW
P_t	第 t 个时段的光伏发电功率	kW
P_t^f	第 t 个时段的截断幂正弦拟合光伏功率	kW
p_t	第 t 个时段的电价	元/kWh

符号	说明	单位
G_t	第 t 个时段的计划购电量	kWh
C_t	第 t 个时段的储能充电量	kWh
D_t	第 t 个时段的储能放电量	kWh
W_t	第 t 个时段的弃光电量	kWh
E_t	第 t 个时段开始时储能设备的电量	kWh
H_t	第 t 个时段的紧急购电量	kWh
H_c, η_d	储能充电效率和放电效率，均取 0.9	—
ω	傅里叶基频， $\omega=2\pi/24$	rad/h
d	日期编号	—
$\mathcal{J}_d$	第 d 天可用的历史信息集	—
d	第 d 天滚动窗口内的历史日期集合	—
$\tilde{\omega}_{d,\tau}$	历史日期 τ 相对第 d 天的标准化权重	—
$R_{d,t}$	第 d 天第 t 个时段的安全裕度	kWh
$G_{d,t}^0$	第 d 天第 t 个时段在 0:00 制定的计划购电量	kWh
$G_{d,t}^\tau$	第 d 天第 t 个时段在预报时点 τ 的调整购电量	kWh
$U_{d,t}^\tau$	增购量，即调整量高于计划量的部分	kWh
$V_{d,t}^\tau$	减购量，即计划量高于调整量的部分	kWh
$p_{d,t}$	第 d 天第 t 个时段的实际波动电价	元/kWh
$\hat{p}_{d,t}$	第 d 天第 t 个时段的预测电价	元/kWh
$\bar{p}_{d,t}$	第 d 天第 t 个时段的历史加权电价轮廓	元/kWh
$p_{d,t}^F$	第 d 天经傅里叶级数得到的周期拟合电价	元/kWh
$\tilde{p}_{d,t}$	第 d 天经傅里叶拟合与 Haar 小波修正后的基础电价预测值	元/kWh
γ_d	第 d 天的前一日同刻价格偏差修正权重	—
N_d	第 d 天电价预测使用的历史窗口长度	天
H_d	第 d 天电价预测使用的时间衰减半衰期	天
μ_d	第 d 天电价预测使用的同星期附加权重	—
K_d	第 d 天电价预测采用的傅里叶谐波阶数	—
J_d	第 d 天电价预测采用的 Haar 小波分解层数	—

五、模型建立与求解

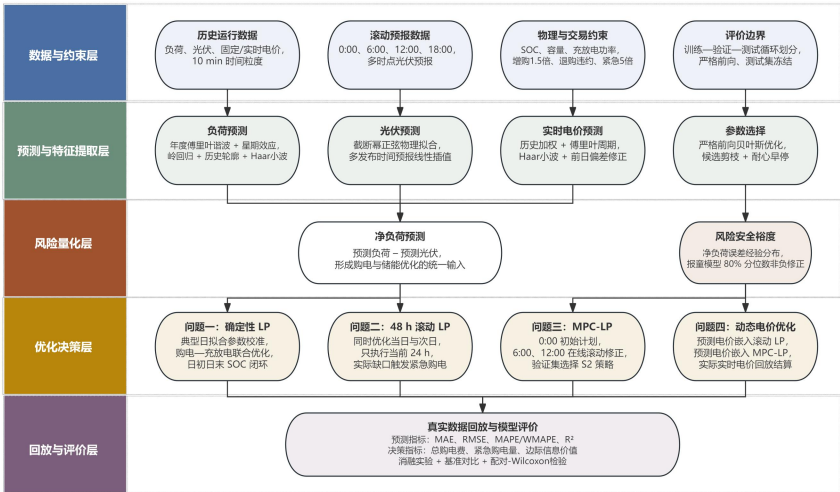


图 0 总体模型结构图

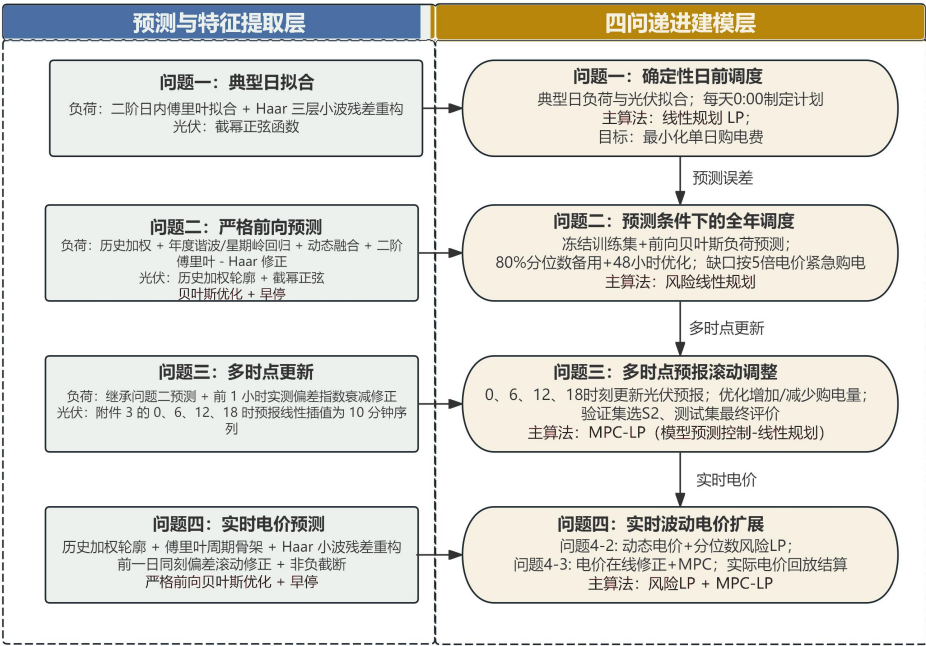


图 1 总体建模思路框架

5.1 问题一模型建立与求解：傅里叶—小波—正弦拟合与储能购电优化

5.1.1 建模思路

问题一采用“原始数据基准解—结构化拟合提交解”双路径。两条路径使用同一组储能约束和线性规划，费用差异只来源于负荷、光伏输入是否经过拟合，因此可把相对费用差解释为拟合误差的经济代价。

负荷用二阶傅里叶项表示低频日周期，再以三级 Haar 小波重构残差中的局部变化；光伏用截断修正弦满足夜间为零、日间单峰的物理边界。二者相减形成净负荷，并与 SOC 状态方程共同进入购电优化。

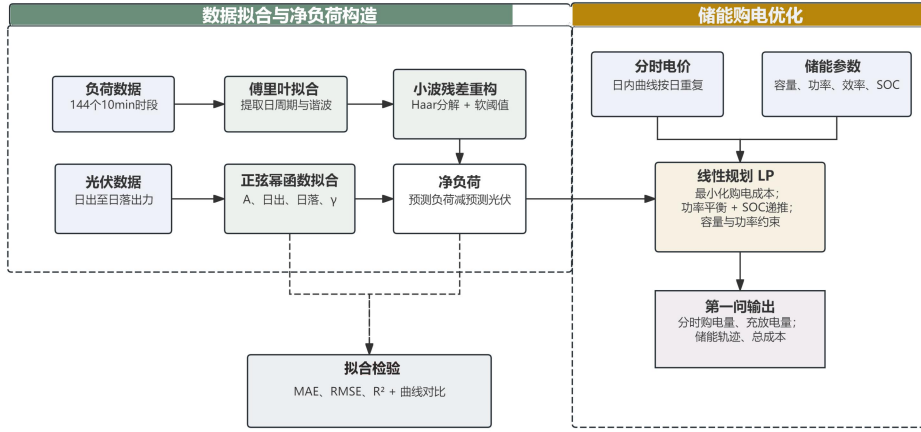


图 2 问题一建模思路框架

5.1.2 负荷的二阶傅里叶拟合

考虑小区负荷具有明显的 24 小时周期和 12 小时次周期特征，采用二阶傅里叶级数描述负荷的主要变化趋势：

$$\hat{L}_t^F = a_0 + a_1 \cos(\omega h_t) + b_1 \sin(\omega h_t) + a_2 \cos(2\omega h_t) + b_2 \sin(2\omega h_t)$$

$$\omega = \frac{2\pi}{24}$$

将模型写成矩阵形式 $L = X\beta + \varepsilon$ ，其中参数向量 $\beta = [a_0, a_1, b_1, a_2, b_2]^T$ ，设计矩阵第 t 行为 $X_t = [1, \cos(\omega h_t), \sin(\omega h_t), \cos(2\omega h_t), \sin(2\omega h_t)]$ 。利用最小二乘法估计傅里叶系数：

$$\hat{\beta} = (X^T X)^{-1} X^T L$$

由于负荷功率不能为负，傅里叶拟合值修正为 $\hat{L}_t^F = \max\{0, X_t \hat{\beta}\}$ 。

傅里叶设计矩阵的五列依次对应常数、24 小时正余弦项和 12 小时正余弦项。最小二乘解是残差平方和的正交投影；若设计矩阵满列秩，参数估计唯一。非负截断只作用于最终功率值，不改变周期系数的估计过程。

5.1.3 傅里叶残差的小波分解与软阈值处理

傅里叶模型能够描述负荷的整体周期趋势，但难以充分反映短时尖峰和局部波动。定义傅里叶拟合残差为 $r_t = L_t - \hat{L}_t^F$ 。将残差表示为局部有效变化与随机噪声之和： $r_t = s_t + \varepsilon_t$ 。

采用 Haar 离散小波变换。令第 0 层近似系数为 $a_{0,k} = r_k$ 。第 j 层 ($j = 1, 2, 3$) 的近似系数和细节系数分别为：

$$a_{j,k} = \frac{a_{j-1,2k} + a_{j-1,2k+1}}{\sqrt{2}}$$

$$d_{j,k} = \frac{a_{j-1,2k} - a_{j-1,2k+1}}{\sqrt{2}}$$

经过三级小波分解后，残差信号可表示为 $r = A_3 + D_3 + D_2 + D_1$ 。为保留有效的局部负荷变化并抑制随机噪声，采用中位数绝对偏差估计第 j 层细节系数的噪声标准差：

$$\hat{\sigma}_j = \frac{\text{median}(|d_{j,k} - \text{median}(d_{j,k})|)}{0.6745}$$

采用通用阈值 $\lambda_j = \hat{\sigma}_j \sqrt{2 \ln n}$ ，其中 $n = 144$ 。对每一层细节系数使用软阈值函数：

$$\tilde{d}_{j,k} = \text{sgn}(d_{j,k}) \max\{|d_{j,k}| - \lambda_j, 0\}$$

5.1.4 小波残差重构与负荷组合模型

利用第三级近似系数和经过阈值处理的细节系数进行逆小波变换： $\hat{r}^W = \text{IDWT}(A_3, \tilde{D}_3, \tilde{D}_2, \tilde{D}_1)$ 。

将重构后的残差与傅里叶趋势叠加，得到傅里叶一小波负荷拟合模型：

小波层在局部尺度上对残差分组。软阈值把幅值低于噪声阈值的细节系数压为零，并对较大系数作连续收缩，因而比硬阈值更少产生重构跳变。组合模型可理解为“低频正交投影 + 稀疏局部校正”，既保留周期解释，又降低噪声驱动的错误充放电。

$$\hat{L}_t^{FW} = \max\{0, \hat{L}_t^F + \hat{r}_t^W\}$$

5.1.5 光伏发电的截断幂正弦模型

根据光伏发电功率在夜间为零、白天先上升后下降的特征，建立截断幂正弦模型：

$$\hat{P}_t = A \left[\sin \left(\frac{\pi (h_t - t_r)}{t_s - t_r} \right) \right]^\gamma, \quad t_r \leq h_t \leq t_s$$

$$\hat{P}_t = 0, \quad \text{其他时段}$$

其中，A 为光伏发电功率幅值， t_r 为等效日出时刻， t_s 为等效日落时刻， γ 为曲线形状参数。日照窗口内的形状基函数 z_t 定义为：

$$z_t = \begin{cases} \sin \left(\pi \frac{h_t - t_r}{t_s - t_r} \right)^\gamma, & t_r \leq h_t \leq t_s \\ 0, & \text{其他时段} \end{cases}$$

对于给定的日照窗口和形状参数，幅值 A 采用非负最小二乘估计，其余参数通过网格搜索校准。

$$\hat{A} = \max \left\{ 0, \frac{\sum_t z_t P_t}{\sum_t z_t^2} \right\}$$

使用网格搜索确定其余参数。当前计算得到 $\hat{A} = 7696.16 \text{ kW}$ ， $\hat{t}_r = 5.17 \text{ h}$ ， $\hat{t}_s = 18.83 \text{ h}$ ， $\hat{\gamma} = 1.725$ 。

截断幂正弦的日照窗口和非负约束将光伏曲线的物理边界直接写入模型，避免一般回归曲线在清晨、傍晚或夜间产生不合理出力；这使负荷与光伏相减后得到的拟合净负荷具有明确的工程含义。

5.1.6 储能购电线性规划模型

决策变量。对每个时段 t ，定义计划购电量 G_t 、储能充电量 C_t 、储能放电量 D_t 、弃光电量 W_t 和储能电量 E_t 。所有变量均为非负： $G_t, C_t, D_t, W_t \geq 0$ 。

目标函数。以全天购电费用最小为主要目标，加入极小的储能吞吐量惩罚：

$$\min Z = \sum_{t=0}^{143} p_t G_t + \varepsilon \sum_{t=0}^{143} (C_t + D_t)$$

其中 $\varepsilon = 10^{-7}$ ，用于在多组等费用最优解中优先选择储能吞吐量较小的方案。

极小吞吐惩罚只在购电费用相同的解之间选择循环量更小者。由于目标函数、功率平衡、SOC 递推和上下界均为线性关系，可将决策向量写成 $x = (G, C, D, W, E)$ ，模型统一表示为 $\min c^T x, s.t. A_e q x = b_e q, A_u b x \leq b_u b, x \geq 0$ ；其可行域为凸多面体，线性规划求得的是全局最优解。

电量平衡约束。每个时段内满足电量平衡：

$$G_t + \hat{P}_t \Delta t + D_t = \hat{L}_t^{FW} \Delta t + C_t + W_t$$

储能状态转移约束：

$$E_{t+1} = E_t + 0.9 C_t - \frac{D_t}{0.9}$$

储能容量和功率约束：1200 ≤ E_t ≤ 10800，0 ≤ C_t, D_t ≤ 833.3333。初始与终止储能约束：E₀ = E₁₄₄ = 6000 kWh。

求解流程为“拟合参数—构造 144 时段系数与约束矩阵—线性规划—可行性校验”。逐时电量平衡保证供需守恒，SOC 方程提供跨时段耦合，E₀ = E₁₄₄ 消除把储能透支到次日的虚假低成本，因此所得方案具有完整的日内物理闭环。

5.1.7 拟合结果与分析

采用平均绝对误差（MAE）、均方根误差（RMSE）和决定系数（R²）评价拟合效果：

$$\begin{aligned} \text{MAE} &= \frac{1}{n} \sum |y_t - \hat{y}_t| \\ \text{RMSE} &= \sqrt{\frac{1}{n} \sum (y_t - \hat{y}_t)^2} \\ R^2 &= 1 - \frac{\sum (y_t - \hat{y}_t)^2}{\sum (y_t - \bar{y})^2} \end{aligned}$$

负荷模型	MAE (kW)	RMSE (kW)	R ²
二阶傅里叶模型	244.21	300.06	0.9058
傅里叶—三级 Haar 小波模型	111.65	158.40	0.9737

表 1 负荷拟合结果对比

光伏截断幂正弦模型的拟合结果为 MAE = 38.25 kW，RMSE = 58.67 kW，R² = 0.9996。同时将附件 1 原始 10 分钟数据直接输入同一线性规划作为理论基准。相较该基准，纯傅里叶拟合策略的成本损失为 2.99%，加入小波残差修正后成本损失降至 1.59%。

小波修正后负荷 MAE 和 RMSE 分别下降 54.28%和 47.21%；相对原始数据直接优化的理论基准，调度成本损失由纯傅里叶的 2.99%降至 1.59%。这说明局部误差的改善通过净负荷进入储能调度，并减少了错误移峰造成的经济损失。

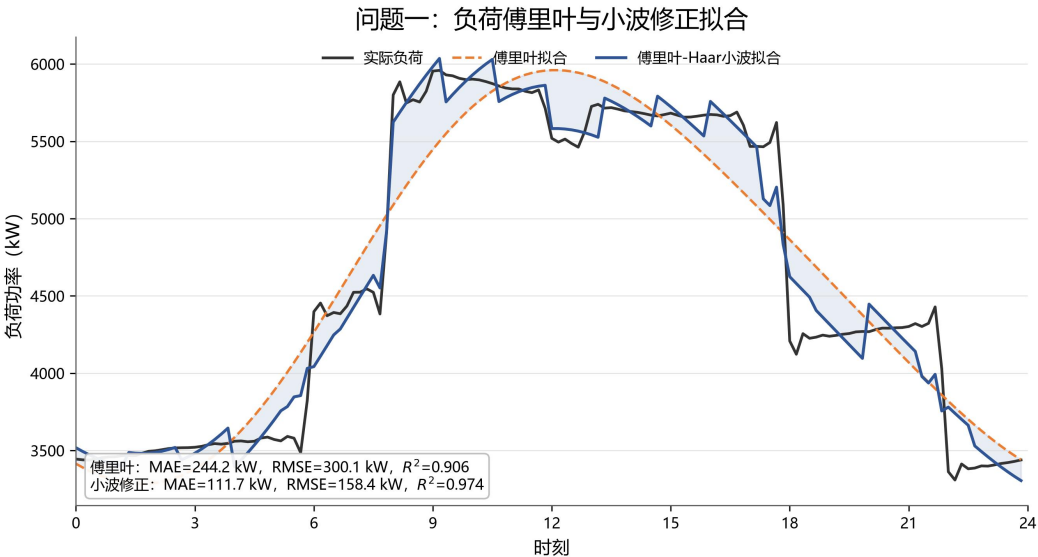


图 3 负荷傅里叶—小波拟合效果

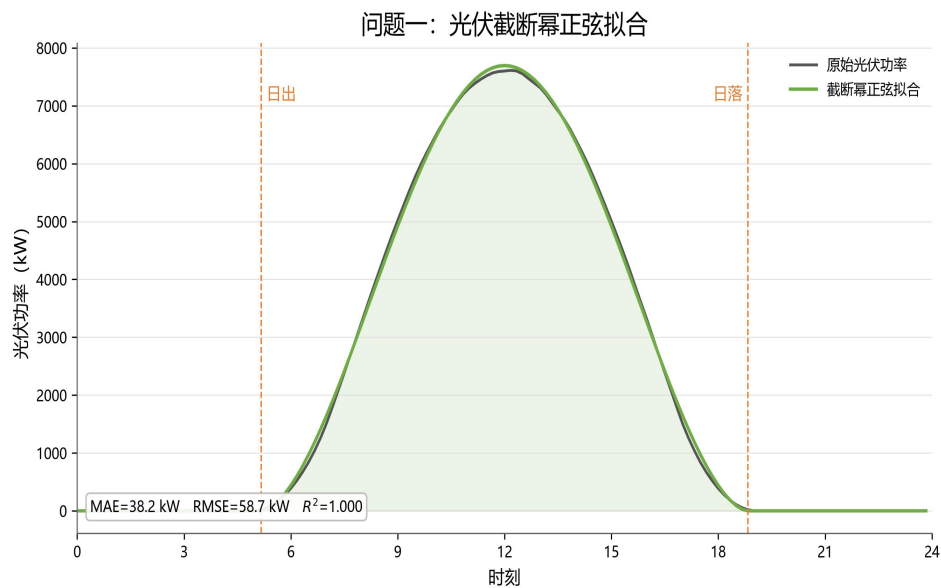


图 4 光伏截断幕正弦拟合效果

时间段	购电量 / kWh	时间段	购电量 / kWh	时间段	购电量 / kWh
10:00 - 10:10	0.00	12:00 - 12:10	481.05	14:00 - 14:10	0.00
16:00 - 16:10	418.70	18:00 - 18:10	697.35	20:00 - 20:10	0.00
全天购电量 / kWh		59,301.66	全天购电费 / 元		34,936.20

表 2 微网在指定时间段的购电量及全天的购电量和购电费

时间段	充电量 / kWh	放电量 / kWh	时间段	充电量 / kWh	放电量 / kWh
0:00 - 4:00	4,500.00	0.00	4:00 - 8:00	833.33	5,609.42
8:00 - 12:00	3,609.46	2,219.78	12:00 - 16:00	6,246.48	154.11
16:00 - 20:00	0.00	5,007.74	20:00 - 24:00	5,333.33	3,632.26
0:00 储电量 / kWh		6,000.00	24:00 储电量 / kWh		6,000.00

表 3 储能设备在指定时间段的充放电量及 0:00 和 24:00 的储电量

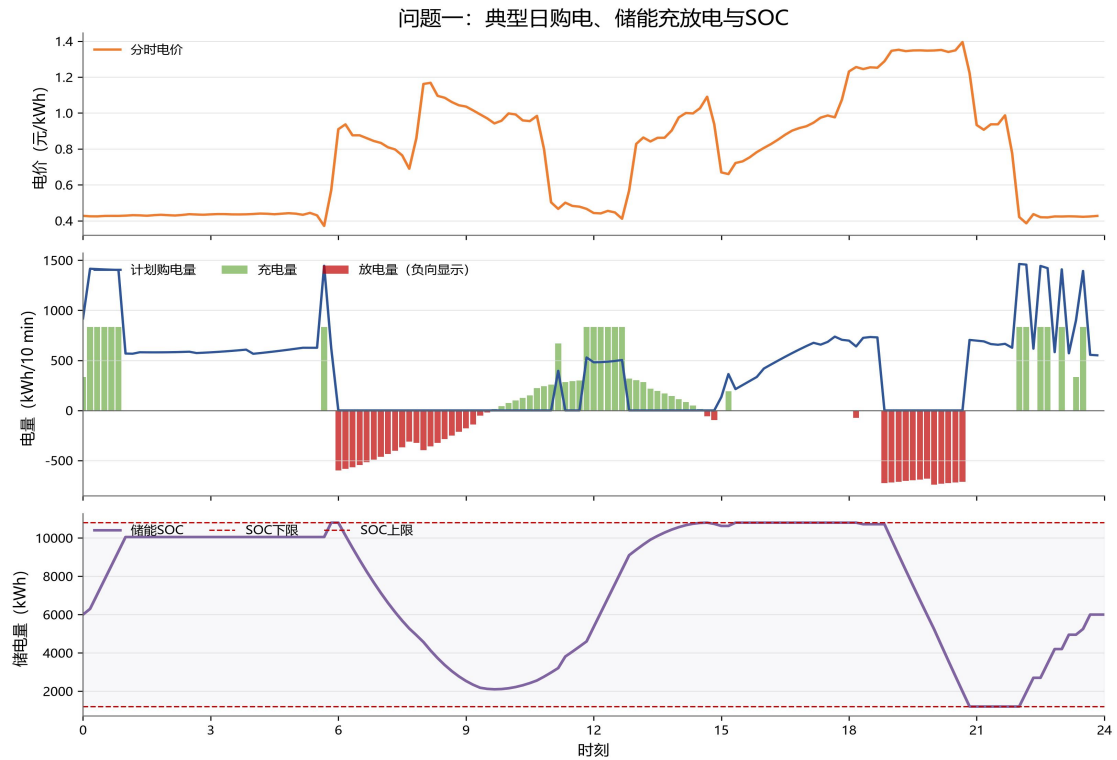


图 5 问题一购电与储能调度策略

5.2 问题二模型建立与求解：滚动预测与紧急购电优化模型

5.2.1 建模思路

问题二在每天 0:00 只利用此前可用信息预测当日负荷和光伏。预测偏高会产生计划过剩，预测偏低则触发 5 倍电价紧急购电，因此目标不是单纯最小化点预测误差，而是把误差分布、安全裕度和储能调度联合起来。

预测由年度谐波一星期效应岭回归、近期历史加权轮廓及日内傅里叶一小波校正组成；参数由严格前向贝叶斯优化确定。随后以报童模型的 80%分位数修正净负荷，并用 48 小时滚动 LP 刻画跨日 SOC 机会成本，只执行当前 24 小时。

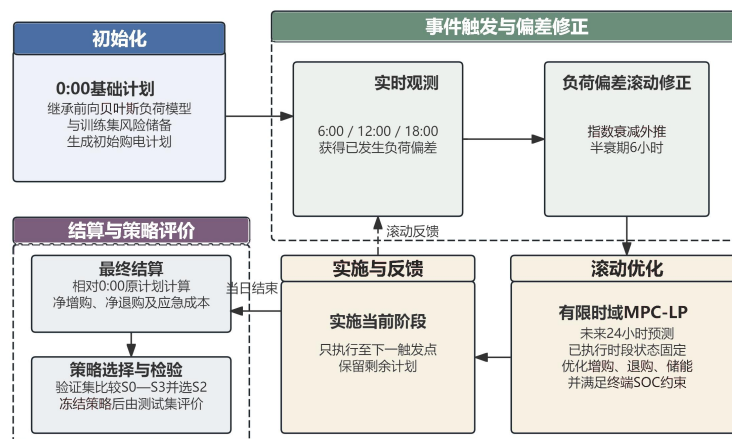


图 6 问题二思路框架

5.2.2 历史信息集与滚动窗口

全年日期按照“训练、训练、训练、验证、训练、测试”的固定循环划分为训练集（244 天）、验证集（61 天）和测试集（60 天）。在第 d 天 0:00，模型只能使用属于训练集且早于第 d 天的数据：

训练、验证和测试按固定六日循环划分。第 d 日模型只使用日期早于 d 且属于训练集的样本；验证集只更新贝叶斯代理模型和选择策略，测试集始终冻结。这一保守信息集牺牲少量已发生数据，却使所有方法在同一评价边界下可比较。

$$I_d = \{L_{\tau,t}, P_{\tau,t}^{PV} : \tau < d, \tau \in D_{\text{train}}\}$$

设滚动窗口长度为 W 。为使较近日期和相同星期类型的数据具有更高权重，定义历史日期 τ 的权重为：

$$\omega_{d,\tau} = \exp[-\rho(d-\tau)](1 + \kappa I_{d,\tau})$$

其中 $I_{d,\tau} = 1$ 当且仅当 d 与 τ 的星期类型相同， $\rho > 0$ 为时间衰减系数， $\kappa \geq 0$ 为相同星期类型的附加权重系数。

指数权重随时间距离单调衰减，同星期指标则对结构相似日加权。归一化后，各历史曲线构成凸组合，因此预测水平不会因权重总量改变而任意缩放；窗口长度控制偏差—方差权衡，半衰期控制对近期变化的响应速度。

5.2.3 年度谐波—星期效应—岭回归负荷预测

令 q_d 为第 d 天在一年中的序号，采用三阶年度傅里叶项和 6 个星期虚拟变量构造日期特征向量 x_d 。对可用训练日期构造特征矩阵 X_d ，将每天 144 个负荷时段组成响应矩阵 Y_d 。通过岭回归估计参数矩阵：

$$P = \text{diag}(0, 1, \dots, 1),$$

$$\hat{B}_d = (X_d^T X_d + \lambda P)^{-1} X_d^T Y_d.$$

其中 P 为岭惩罚矩阵，其首个对角元素为 0，因此截距项不参与正则化，其余回归系数均受到 (L_2) 惩罚。初始模型取 $\lambda_0 = 3$ 。年度—星期动态负荷曲线为 $L_d^R = \max\{0, x_d^T \hat{B}_d\}$ 。与近期历史加权曲线融合：

岭回归在最小化残差平方和的同时惩罚非截距系数的二范数，从而缓解年度谐波与星期虚拟变量的共线性。多输出形式一次估计 144 个时段的系数矩阵，共享相同日期特征但允许各时段有不同响应；融合系数 α 再在稳定年度结构与近期局部水平之间加权。

$$L_d^B = (1 - \alpha) L_d + \alpha \hat{L}_d^R$$

5.2.4 严格前向贝叶斯优化与早停

采用高斯过程贝叶斯优化联合选择参数向量 $\theta = (W, H, \kappa, J, M, \log_{10} \lambda, \alpha)$ 。目标函数为截至当前时刻已经完成的验证日累计 RMSE。每个验证日必须先完成预测，待真实值开放后才可更新高斯过程代理模型；测试集真实值始终不进入寻优。采用期望改进函数选择下一组候选参数：

$$EI(\theta) = (J_{\min} - \mu) \Phi(z) + \sigma \varphi(z), \quad z = \frac{J_{\min} - \mu}{\sigma}$$

若连续 12 个候选未使验证 RMSE 至少下降 0.05 kW，则停止搜索。

贝叶斯优化用高斯过程近似“参数—验证 RMSE”映射，期望改进 EI 同时考虑均值收益和预测不确定性：均值较低的区域用于开发，不确定性较大的区域用于探索。每个验证日完成后才更新代理模型，并以连续 12 次改进不足 0.05 kW 为早停条件，避免未来信息泄漏和无效搜索。

5.2.5 净负荷预测误差与安全裕度

真实净负荷为 $N_{d,t} = L_{d,t} - P_{d,t}^{PV}$ ，预测净负荷为 $\hat{N}_{d,t} = \hat{L}_{d,t} - \hat{P}_{d,t}$ 。定义历史一步预测误差 $e_{\tau,t} = N_{\tau,t} - \hat{N}_{\tau,t}$ 。基于最近 W_e 天的历史误差，定义安全裕度：

$$R_{d,t} = \max\{0, Q_\alpha(e_{d-W_e,t}, \dots, e_{d-1,t})\}$$

设正常计划购电单价为 p_t 。多买一单位按正常价格支付，超额损失为 $C_o = p_t$ ；少买一单位需

以 $5p_t$ 紧急补足，相对于原计划的增量不足损失为 $C_u=5p_t-p_t=4p_t$ 。由报童模型得到临界分位数与非负安全裕度：

$$\alpha = \frac{C_u}{C_u + C_o} = \frac{4p_t}{4p_t + p_t} = 0.8, R_{d,t} = \max(0, F_{d,t}^{-1}(0.8))$$

由于计划购电与不足惩罚呈 1 : 4 的不对称关系，即使近期统计在低负荷时段出现轻微整体高估，模型也不设置负安全裕度，而采用 $R_{d,t} = \max(0, Q_{d,t}(0.8))$ ，以保证供电可靠性。

报童分位数可由边际期望损失得到：安全裕度增加一单位时，过量概率 $F(R)$ 带来 C_o 成本，不足概率 $1 - F(R)$ 减少 C_u 损失；令边际变化 $C_o F(R) - C_u [1 - F(R)] = 0$ ，得到 $F(R) = C_u / (C_u + C_o) = 0.8$ 。非负截断表示模型只允许增加可靠性裕度，不主动削减基准计划。

$$\alpha = \frac{4p_t}{4p_t + p_t} = 0.8$$

风险修正后的计划负荷功率为 $\tilde{L}_{d,t} = \hat{L}_{d,t} + R_{d,t} / \Delta t$ 。

5.2.6 日前计划购电优化模型

第 d 天 0:00 的计划购电目标函数为：

$$\min Z_d^{\text{plan}} = \sum_{t=0}^{143} p_t G_{d,t} + \varepsilon \sum_{t=0}^{143} (C_{d,t} + D_{d,t})$$

满足电量平衡约束、储能状态转移约束 $E_{d,t+1} = E_{d,t} + 0.9C_{d,t} - D_{d,t}/0.9$ 、容量和功率约束。采用 48 小时滚动优化：

$$E_{d,288} = E_{d,0}$$

为避免优化器在滚动窗口末端无成本释放储能，约束 48 小时预测窗口末端 SOC 回到窗口初始水平；实际仅执行前 144 个时段，次日计划在新的预测信息到达后重新计算。在第 d 天 0:00 同时优化第 d 天和第 $d+1$ 天，只执行前 24 小时的决策。

48 小时窗口并不执行次日方案，而是用次日预测给第 d 日末端 SOC 提供机会成本。每次仅落地前 144 个时段，次日 0:00 重新预测并覆盖未执行部分，因此滚动过程满足信息因果性，同时避免单日模型在 24:00 人为清仓或满电。

5.2.7 真实数据回放与紧急购电

当天实际运行时，计划购电量 $G_{d,t}$ 已经确定。定义供电缺口：

$$\Delta Q_{d,t} = L_{d,t} \Delta t + C_{d,t} - P_{d,t}^{PV} \Delta t - D_{d,t} - G_{d,t}$$

紧急购电量为 $H_{d,t} = \max\{0, \Delta Q_{d,t}\}$ 。实际总购电费用为：

回放阶段固定日前决策，再以真实净负荷计算缺口。低估误差乘以 5 倍电价、高估误差只承担正常购电成本，故总费用构成对预测误差的非对称损失函数，也为 80% 安全分位数提供了经济解释。

$$C_d^{\text{total}} = \sum p_t G_{d,t} + 5 \sum p_t H_{d,t}$$

5.2.8 模型改进效果与全年结果

指标	改进前	改进后	变化
负荷 R^2	0.8491	0.9264	提高 0.0773
负荷 MAE (kW)	424.97	261.81	降低 38.39%
负荷 RMSE (kW)	545.54	380.93	降低 30.17%
净负荷 MAE (kW)	485.68	352.51	降低 27.42%
净负荷 RMSE (kW)	647.71	512.76	降低 20.84%

指标	改进前	改进后	变化
测试集总购电费（元）	2,920,459	2,816,554	降低 3.56%

表 4 贝叶斯优化前后测试集预测与费用对比

训练集、验证集和测试集的 R^2 接近，说明贝叶斯优化后的模型具有较稳定的样本外表现。评价同时采用预测层与决策层两类指标：前者以 MAE、RMSE 和 R^2 衡量误差，后者以总购电费和紧急购电量衡量经济效果。贝叶斯优化后测试集总购电费降低 3.56%，紧急购电占比仅 1.93%，表明预测改进能够通过安全裕度与滚动优化转化为实际费用收益。

指标	数值
全年计划购电量（kWh）	25,327,271
全年紧急购电量（kWh）	497,285
全年计划购电费（元）	15,600,656
全年紧急购电费（元）	1,961,529
全年总购电费（元）	17,562,185
紧急购电占比	1.93%

表 5 问题二全年汇总结果

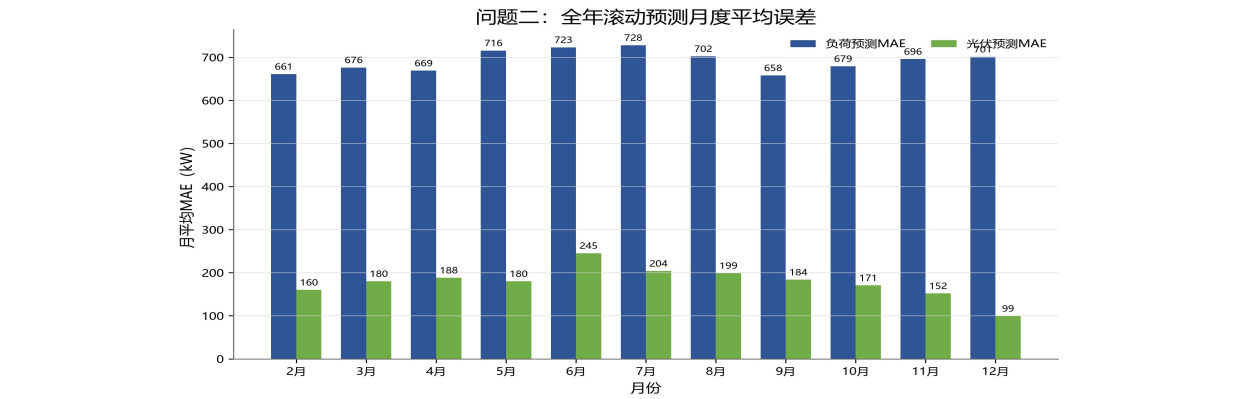


图 7 问题二·月度预测误差分布

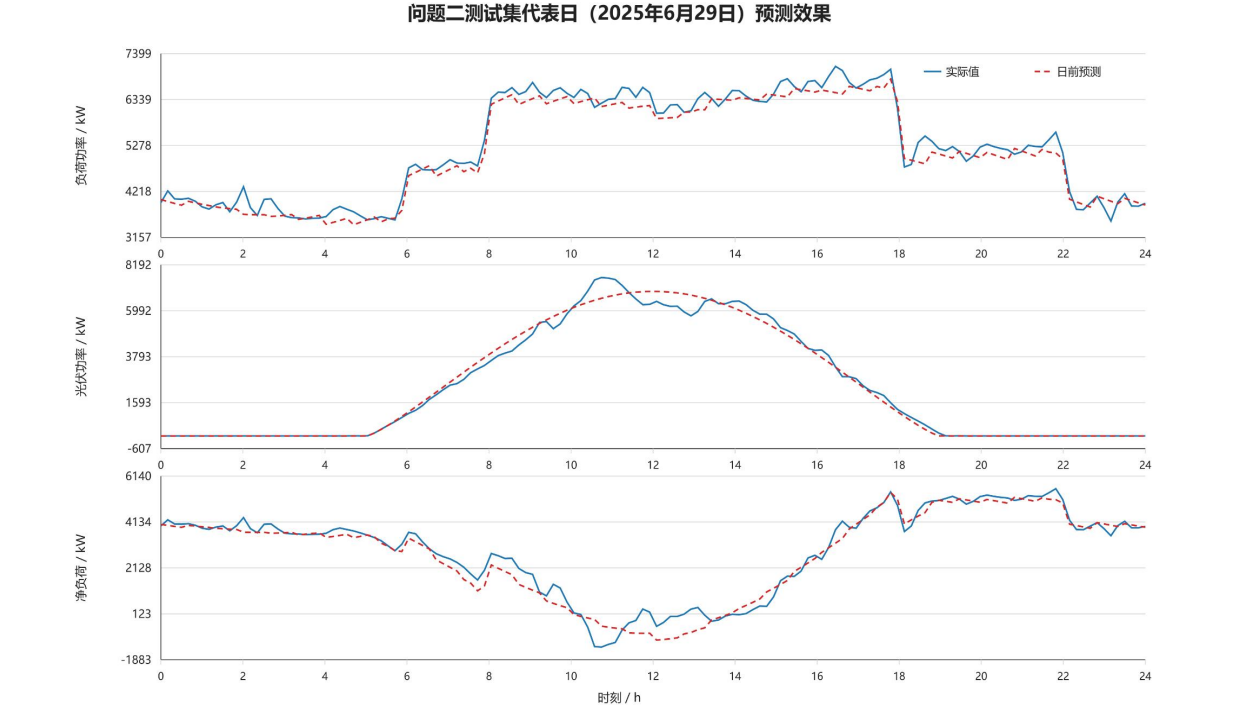


图 8 问题二测试集代表日预测效果

5.2.9 指定日期结果

2025. 3. 20		2025. 6. 21		2025. 9. 23		2025. 12. 21	
时间段	购电量 / kWh	时间段	购电量 / kWh	时间段	购电量 / kWh	时间段	购电量 / kWh
0:20 - 0:40	52.69	5:40 - 5:50	3.87	0:40 - 1:00	32.60	1:20 - 1:30	15.68
1:20 - 1:40	12.89	6:20 - 6:30	8.31	1:10 - 1:20	23.32	1:40 - 1:50	11.93
2:00 - 2:10	14.22			1:50 - 2:40	34.74	3:10 - 3:30	42.92
3:10 - 4:00	113.42			4:00 - 4:10	15.89	3:40 - 4:00	81.57
4:40 - 5:40	74.71			5:00 - 5:20	21.73	5:40 - 6:00	17.46
5:50 - 6:10	7.32			6:20 - 6:30	5.60	10:50 - 11:50	262.16
9:00 - 9:30	18.79			7:50 - 8:00	6.69	15:30 - 15:40	8.43
9:50 - 10:40	71.19			9:40 - 10:30	197.22	17:30 - 17:40	13.17
11:10 - 11:40	44.14			10:40 - 11:20	227.92	18:20 - 18:30	1.29
12:40 - 13:00	33.85			14:30 - 15:30	461.55	18:50 - 19:10	15.25
13:20 - 13:40	36.71			16:00 - 18:00	357.58	19:30 - 19:40	5.31
15:40 - 17:00	153.41			18:20 - 19:00	101.32	20:00 - 20:20	48.95
17:10 - 17:20	12.62			19:10 - 19:40	48.35	22:00 - 22:10	5.48
18:50 - 19:20	85.49			20:00 - 20:40	150.10	23:00 - 23:20	16.33
19:50 - 20:30	81.98			21:00 - 22:10	235.35		
21:30 - 21:40	11.83			22:30 - 22:40	56.56		
23:40 - 23:50	21.26			23:40 - 23:50	13.95		

表 6 问题二指定日期紧急购电结果

注：四日计划购电量分别为 66,357.35、46,121.75、66,939.61 和 98,610.59 kWh；
总购电费分别为 43,769.21、26,453.69、50,101.09 和 65,121.89 元。

5.3 问题三：多时点预测信息价值驱动的风险感知 MPC 购电优化模型

5.3.1 建模思路

问题三把日前开环计划扩展为状态反馈控制。0:00 形成基准计划，6:00、12:00 和 18:00 到达新预报时，仅重优化尚未执行区间；已执行购电不可逆，当前 SOC 成为下一窗口初值。预报频率由验证集费用决定，从而同时考虑信息收益与调整代价。

与单纯更新预测输入不同，模型按预报发布时间和预测提前量分别估计安全裕度，并把当前真实 SOC 作为滚动优化的初始状态；同时以验证集总费用衡量新增预报的信息价值，在预测精度改善与增购溢价、减购违约成本之间进行权衡，因此预报频率由经济价值而非“越多越好”的经验规则决定。

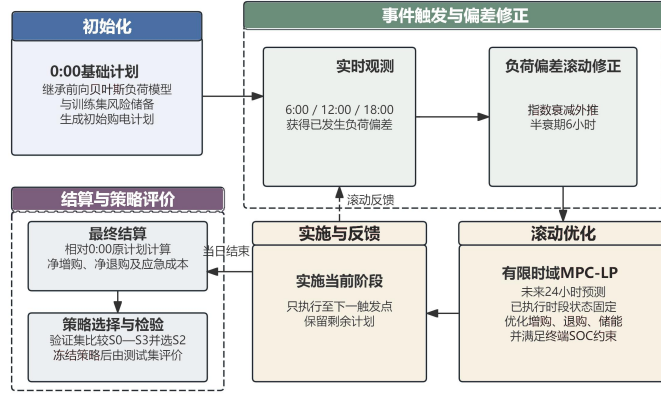


图 9 问题三思路框架

5.3.2 负荷预测与实时修正

负荷继续使用问题二改进后的预测模型。在 6:00、12:00 和 18:00，根据发布时刻之前最近 1 小时的真实负荷修正预测。平均偏差为：

$$b_{d,\tau} = \frac{1}{m} \sum_{j=1}^m (L_{d,\tau-j} - \hat{L}_{d,\tau-j}^{(0)})$$

修正后的未来负荷为：

$$\hat{L}_{d,t}^{(\tau)} = \max\{0, \hat{L}_{d,t}^{(0)} + b_{d,\tau} \cdot 2^{-(t-\tau)/6}\}$$

负荷偏差修正相当于对当日水平误差进行指数衰减的状态估计：发布时刻附近权重重大，远期权重按半衰期下降。这既利用最近观测，又避免把一个小时的偶然扰动永久外推。

5.3.3 光伏预报的 10 分钟插值

附件 3 给出未来 1 至 24 小时的整点光伏功率预报。以预报发布时刻的实际光伏功率作为插值起点，将整点预报线性转换为 10 分钟功率：

$$\hat{P}_{d,t}^{PV,\tau} = (1 - \theta)P_{d,h}^F|_{\tau} + \theta P_{d,h+1}^F|_{\tau}, \quad \theta = (t - h)/1h$$

5.3.4 分发布时间的安全裕度

分别建立 0:00、6:00、12:00 和 18:00 的历史净负荷预测误差库，仅使用训练日期形成的最近 28 个可用误差样本，以其 80% 分位数作为安全裕度：

分发布时间建立误差库，相当于估计条件分布 $F(e|\tau, h)$ ：同一目标时段在不同发布时间具有不同预测提前量，误差尺度并不相同。对各条件分布分别取 80% 分位数，使 MPC 在预报逐步变准时自动收缩裕度，而不是沿用 0:00 的固定风险水平。

$$R_{d,t}^{\tau} = \max\{0, Q_{0.8}(e_{d-28,t}^{\tau}, \dots, e_{d-1,t}^{\tau})\}$$

5.3.5 计划量与调整量

定义 0:00 计划购电量为 $G_{d,t}^0$ ，最新预报下的调整购电量为 $G_{d,t}^{\tau}$ 。使用非负变量 $U_{d,t}^{\tau}$ 和 $V_{d,t}^{\tau}$ 分别表示增购量和减购量：

$$G_{d,t}^{\tau} = G_{d,t}^0 + U_{d,t}^{\tau} - V_{d,t}^{\tau}, \quad U, V \geq 0$$

5.3.6 调整购电结算

当最终购电量高于原计划时，增加部分按交易电价的 1.5 倍结算；当最终购电量低于原计划时，取消部分退回原交易价格，但支付 50% 的违约费用。每个时段的结算费用为：

$$C_{d,t}^{buy} = p_t G_{d,t}^0 + 1.5p_t U_{d,t}^r - 0.5p_t V_{d,t}^r$$

5.3.7 MPC 线性规划

每次预报更新后求解未来 24 小时优化模型：

$$\min Z_{d,\tau} = \sum_t (1.5p_t U_{d,t}^r - 0.5p_t V_{d,t}^r) + \varepsilon \sum_t (C_{d,t} + D_{d,t})$$

预测电量平衡、储能状态方程及容量和功率约束同问题二。每次滚动优化以当前真实 SOC 为初始状态，并约束 24 小时预测窗口末端 SOC 回到窗口初值。对当日尚未执行时段，提交代码采用 $G_{d,t} = G_{d,t}^0 + U_{d,t}^r - V_{d,t}^r$ ，且 G 、 U 、 V 均非负。

$$G_{d,t} = G_{d,t}^0 + U_{d,t}^r - V_{d,t}^r, \quad G \geq 0, \quad U \geq 0, \quad V \geq 0$$

$$C^{\text{act}} = \sum_t^T (p_t G_{d,t}^0 + 1.5p_t U_{d,t}^r - 0.5p_t V_{d,t}^r)$$

基准购电成本 pG^0 在调整时已锁定，是与决策无关的常数，因此可从目标函数中删除。若 U 、 V 同时增加 δ ，最终购电 G 不变而增量成本增加 $p\delta > 0$ ，最优解自然满足 $UV=0$ ；再由 $G=G^0+U-V \geq 0$ 可得最优解中 $V \leq G^0$ ，故代码虽未显式设置该上界，也不会产生无界退款套利。

MPC 的滚动性体现在“更新信息集—读取当前 SOC—求解剩余时域—只执行至下一更新点”。1.5 倍增购、0.5 倍退购扣减和 5 倍紧急购电共同构成分层风险价格，使模型在立即调整与等待真实缺口之间选择成本较低者。

5.3.8 最终执行策略和紧急购电

每个时段最终生效的购电量为对应预报时段的最新调整结果。真实数据下的紧急购电量为：

$$H_{d,t} = \max\{0, L_{d,t} \Delta t + C_{d,t} - P_{d,t}^{PV} \Delta t - D_{d,t} - \bar{G}_{d,t}\}$$

总费用为：

$$Z = \sum C_{d,t}^{buy} + 5 \sum p_t H_{d,t}$$

5.3.9 预报时刻组合比较与结果

程序计算了四种策略，并仅根据验证集总费用选择最终策略。测试集结果如下：

策略	使用预报时刻	紧急购电量 (kWh)	总购电费 (元)	相对 S0 变化
S0 (仅 0:00)	0:00	76,027	2,836,058	—
S1 (+6:00)	0:00/6:00	50,067	2,768,219	节省 67,838 元
S2 (+12:00)	0:00/6:00/12:00	42,304	2,708,241	节省 127,816 元
S3 (完整 MPC)	0:00/6:00/12:00/18:00	49,645	2,782,696	节省 53,361 元

表 7 问题三四四种策略测试集结果对比

5.3.10 预报信息边际价值与最优预测频率

为避免利用测试集反向选择预报时刻，本文仅根据验证集总购电费用计算新增预报时刻的边际信息价值。设当前预报时刻集合为 S ，待考察的新增预报时刻为 r ，则其验证集边际信息价值定义为当前策略与加入新时刻后策略的验证集总费用之差。总费用已经包含计划购电费用、增购溢价、减购违约金和紧急购电费用，因此无需再单独扣除调整成本。当 $MIV_{val} > 0$ 时，说明新增预报能够降低验证集费用；当 $MIV_{val} < 0$ 时，说明新增信息带来的收益不足以抵消相应调整代价。

$$V(\tau|T) = C(T) - C(T \cup \{\tau\})$$

新增时刻	原策略	新策略	边际费用节省 (元)	节省率	紧急购电变化 (kWh)	判断
6:00	S0	S1	34929.64	1.24%	-11585.62	有必要
12:00	S1	S2	43861.59	1.58%	-861.81	有必要
18:00	S2	S3	-67457.44	-2.47%	+5567.46	无必要

表 8 预报时刻的边际信息价值

表 8 表明新增预报的边际价值并非随更新次数单调增加。验证集上, S0 增加 6:00 预报形成 S1 后, 总费用下降 34, 929. 64 元、紧急购电量减少 11, 585. 62 kWh; 在 S1 基础上增加 12:00 预报形成 S2 后, 总费用再下降 43, 861. 59 元。前两次更新分别修正上午与午后预测偏差, 且仍保留足够的储能调节窗口, 因此具有正向经济价值。

继续加入 18:00 预报形成 S3 后, 验证集总费用反而增加 67, 457. 44 元, 紧急购电量增加 5, 567. 46 kWh。此时光伏出力已接近零、剩余调节窗口仅约 6 小时, SOC 与功率约束进一步压缩调整空间, 而增购溢价和减购违约成本仍然存在, 因此新增信息不足以覆盖调整成本。

据此选择由 0:00、6:00 和 12:00 组成的 S2 策略, 并在策略冻结后进行测试集评价。测试集上, S2 总购电费用为 2, 708, 241. 41 元, 较 S0 下降 4. 51%; 紧急购电量由 76, 027. 36 kWh 降至 42, 303. 58 kWh, 下降 44. 36%, 与验证集结论一致。

因此, 预报更新频率的最优选择取决于新增信息价值、剩余调节时间、储能可调能力和购电调整成本的共同作用。本文最终采用 S2, 并在 2025 年 2 月 1 日至 12 月 31 日回放中得到总购电费用 14, 682, 586. 94 元、紧急购电量 225, 453. 01 kWh。附件 3 之外的其他发布时间因缺少独立预报数据不作数值评价; 若未来获得更多预报, 可按同一验证集费用准则决定是否新增更新时点。

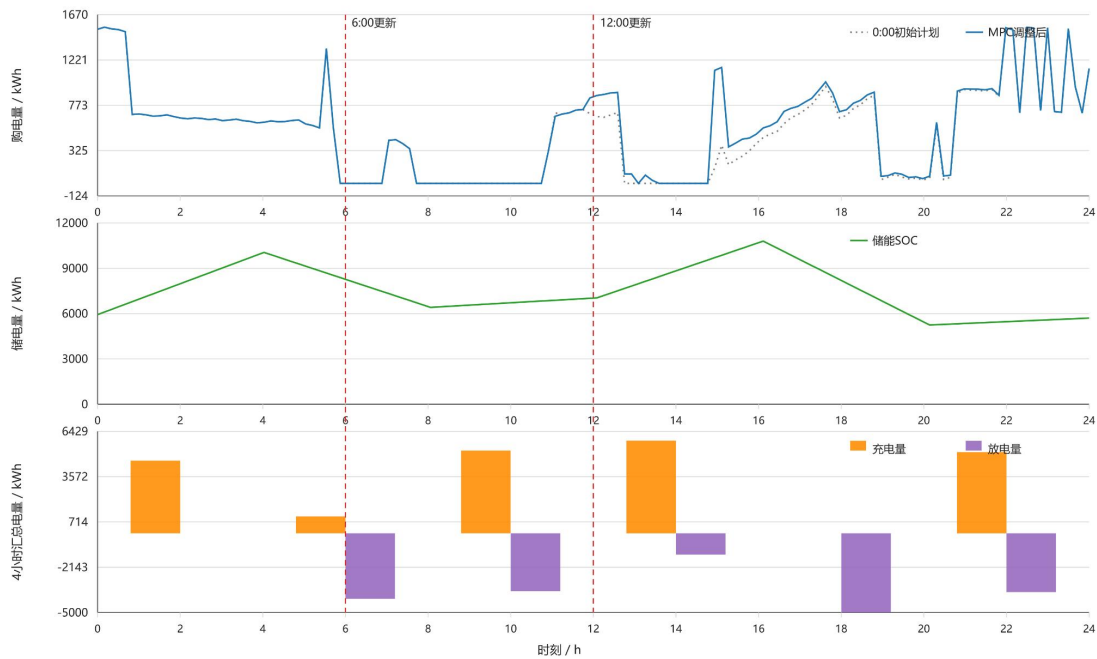


图 10 问题三 MPC 滚动调度过程

5.3.11 指定日期结果

2025. 3. 20		2025. 6. 21		2025. 9. 23		2025. 12. 21	
时间段	购电量 / kWh	时间段	购电量 / kWh	时间段	购电量 / kWh	时间段	购电量 / kWh
0:20 - 0:50	60.22	3:10 - 3:20	0.85	0:40 - 1:30	84.56	1:20 - 1:30	9.23
1:10 - 1:40	36.06	6:20 - 6:30	21.07	1:50 - 2:50	75.48	3:10 - 3:20	23.89
2:00 - 2:10	26.85	7:00 - 7:20	46.67	4:00 - 4:10	29.56	3:40 - 4:00	50.79
2:40 - 3:00	19.50	19:10 - 19:20	8.63	4:30 - 5:20	44.30	9:00 - 9:10	3.21

2025. 3. 20		2025. 6. 21		2025. 9. 23		2025. 12. 21	
时间段	购电量 / kWh	时间段	购电量 / kWh	时间段	购电量 / kWh	时间段	购电量 / kWh
3:10 - 4:10	134. 16			7:50 - 8:10	23. 81	9:30 - 10:00	83. 70
4:40 - 5:40	97. 85			8:30 - 8:40	1. 31	10:50 - 12:00	724. 49
5:50 - 6:10	30. 45			10:10 - 10:20	7. 41	13:00 - 13:20	18. 33
6:50 - 7:00	9. 01			10:50 - 11:10	41. 76	17:30 - 17:40	4. 83
8:10 - 12:00	1, 275. 50			11:50 - 12:00	25. 03	18:50 - 19:00	4. 16
12:40 - 14:00	345. 31			13:00 - 13:20	39. 28	20:00 - 20:20	34. 83
14:10 - 14:30	15. 68			13:50 - 14:00	7. 08	23:00 - 23:20	7. 12
18:50 - 19:20	68. 64			16:10 - 16:30	21. 15		
19:50 - 20:30	91. 94			17:40 - 17:50	34. 28		
21:20 - 21:50	43. 81			18:20 - 19:00	114. 33		
23:40 - 23:50	37. 84			19:10 - 19:40	53. 00		
				20:00 - 20:40	109. 25		
				21:00 - 22:10	235. 39		
				22:30 - 22:40	51. 70		
				23:40 - 23:50	15. 62		

表 9 问题三指定日期结果

注：四日最终调整购电量分别为 66, 248. 08、39, 726. 30、68, 331. 88 和 98, 031. 84 kWh；
总购电费分别为 50, 471. 44、24, 037. 39、47, 725. 43 和 67, 227. 74 元。

5. 4 问题四：实时波动电价下的预测驱动购电优化模型

5. 4. 1 建模思路

问题四将电价由已知系数扩展为预测外生量。历史加权轮廓给出局部水平，傅里叶项提取日周期，小波残差恢复尖峰，前日同刻偏差修正短期漂移；各模块相加后作非负截断，参数由严格前向贝叶斯优化确定。

预测电价分别进入问题二的 48 小时滚动 LP 和问题三的 MPC-LP 用于决策，真实回放统一使用附件 4 实际电价结算。因此评价同时包含“价格预测误差改变调度次序”和“负荷、光伏误差触发紧急购电”两条传导路径。

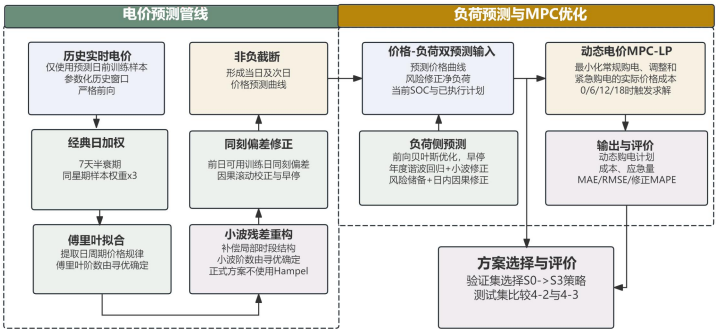


图 11 问题四思路框架

5.4.2 历史加权电价轮廓与傅里叶周期提取

从目标日前已开放的训练日期中取最近 N_d 个可用日，构造目标日的参考轮廓：

$$\bar{p}_{d,t} = \frac{\sum w_{d-j} p_{d-j,t}}{\sum w_{d-j}}$$

采用 K_d 阶傅里叶级数拟合历史加权轮廓：

$$p_{d,t}^F = a_0 + \sum_{k=1}^{K_d} [a_k \cos(k \omega h_t) + b_k \sin(k \omega h_t)]$$

5.4.3 电价残差的小波修正与前日偏差滚动修正

定义傅里叶电价残差 $r_{d,t} = \bar{p}_{d,t} - p_{d,t}^F$ 。采用 Haar 小波分解和软阈值处理，得到电价残差的小波重构值。定义第 t 天的观测偏差为 $e_{d,t} = p_{d,t} - \bar{p}_{d,t}$ 。最终预测为：

从优化角度看，电价预测误差改变的是各时段购电与 SOC 的影子价格：低估未来高价会减少提前充电，高估则可能造成过度储能。傅里叶提供平滑基准，小波和前日偏差负责恢复局部相对价格，最终收益必须通过真实价格回放而非仅靠 MAPE 判断。

$$\hat{p}_{d,t} = \max\{0, \bar{p}_{d,t} + \gamma_d e_{d,t}\}$$

5.4.4 严格前向贝叶斯优化与电价预测评价

将电价参数写成 $\phi_d = (N_d, H_d, \mu_d, K_d, J_d, B_d, \gamma_{0,d})$ 。高斯过程贝叶斯优化以目标日前已完成的验证日累计修正 MAPE 为目标。若连续 10 个候选未使累计 MAPE 至少降低 0.0005，则提前停止搜索。

指标	测试集结果
电价 MAE (元/kWh)	0.0643
电价 RMSE (元/kWh)	0.0860
电价 MAPE	10.35%
电价 R^2	0.9352

表 10 问题四电价预测测试集结果

5.4.5 问题 4-2 与问题 4-3 的结果

问题 4-2 在问题二框架下，将固定电价替换为预测电价进行计划优化，真实回放时用实际波动电价结算。2025 年 2 月 1 日至 12 月 31 日回放总购电费为 15,786,041.60 元，测试集总费用为 2,988,541.12 元。

问题 4-3 在问题三的 MPC-LP 框架下以预测电价完成计划与调整，并按实际波动电价回放结算。需要注意的是，问题 4-2 与问题 4-3 的 S0 虽都只在 0:00 形成当日计划，但前者采用 48 小时滚动 LP，后者采用 24 小时 MPC-LP（仅保留 0:00 更新），因此两者是不同基准框架，费用不应直接等同。

策略	使用预报时刻	紧急购电量 (kWh)	总购电费 (元)	相对 S0 变化
S0 (仅 0:00)	0:00	76,027	2,996,914	—
S1 (+6:00)	0:00/6:00	50,126	2,928,696	节省 68,218 元
S2 (+12:00)	0:00/6:00/12:00	42,389	2,867,048	节省 129,865 元
S3 (完整 MPC)	0:00/6:00/12:00/18:00	49,708	2,938,452	节省 58,462 元

表 11 问题 4-3 四种策略测试集结果对比

验证集最终选定 S2。测试集上，问题 4-3 的 S2 总费用为 2,867,048.23 元，较问题 4-2 的

2,988,541.12 元减少 121,492.89 元，降幅 4.07%；全年回放总费用为 15,488,688.71 元。结合 MAE、RMSE、MAPE、 R^2 及紧急购电量可见，动态电价预测、安全裕度、储能调度与 MPC 更新能够共同降低预测误差带来的购电成本。

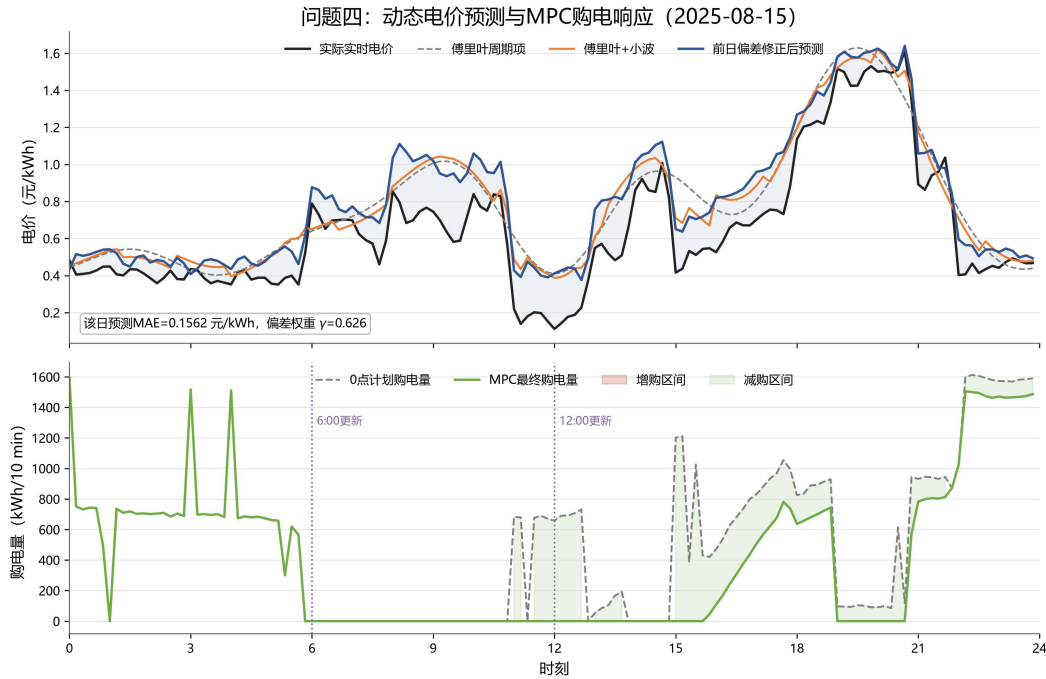


图 12 问题四动态电价预测与调度策略

六、模型分析与检验

6.1 实验设置

全年按“训、训、训、验、训、测”循环划分为 244 个训练日、61 个验证日和 60 个测试日。参数和 MPC 时点仅由训练、验证数据确定，测试集只做一次最终评价。指标包括 MAE、RMSE、MAPE/WMAPE 和 R^2 ；光伏百分比误差只统计实际功率大于 1 kW 的时段，净负荷可能穿越零点，故不以普通 MAPE 作为主指标。

6.2 负荷与光伏预测消融实验

测试集上，完整负荷预测模型的 MAE、RMSE 和 R^2 分别为 261.81 kW、380.93 kW 和 0.9264，较典型日静态基准的 RMSE 降低 62.67%。移除贝叶斯参数更新后 RMSE 升至 545.54 kW，完整模型改进 30.17%；移除小波修正后 RMSE 为 501.86 kW，改进 24.10%。结果表明，贝叶斯动态参数更新增强了模型对季节变化的适应能力，小波修正则有效提升了对日内局部波动的刻画能力。

模型	MAE (kW)	RMSE (kW)	WMAPE (%)	R^2
典型日静态基准	793.42	1,020.46	17.27	0.4721
前一日持续性基准	673.72	1,155.05	14.67	0.3236
仅历史加权轮廓	633.87	793.86	13.80	0.6805
去除周期回归	637.34	798.02	13.88	0.6771
去除小波修正	383.57	501.86	8.35	0.8723
去除贝叶斯更新	424.97	545.54	9.25	0.8491
完整模型	261.81	380.93	5.70	0.9264

表 12 负荷预测消融实验结果（测试集）

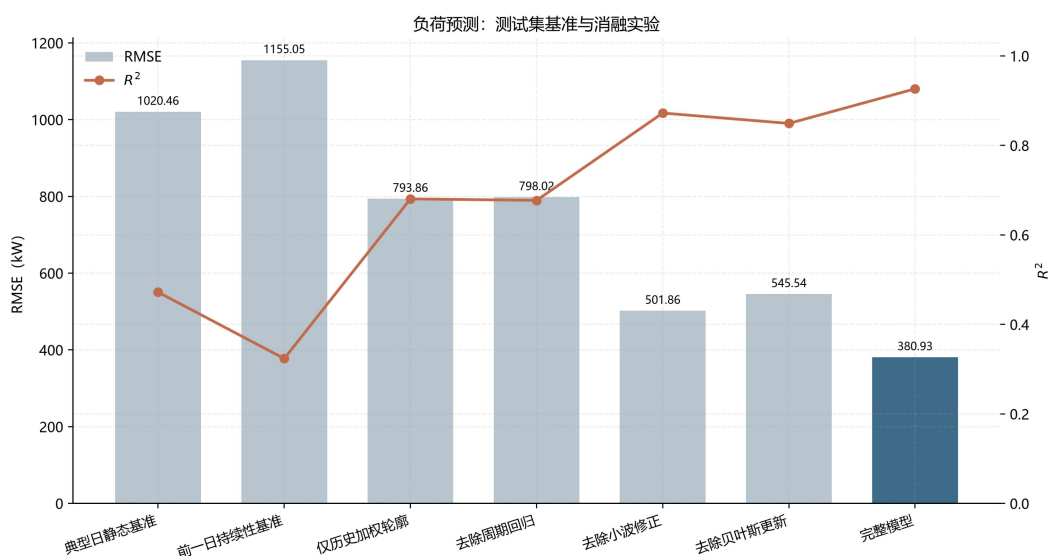


图 13 负荷预测基准与消融实验

光伏预测采用历史加权轮廓与截断幂正弦模型。有效发电时段内，完整模型的 RMSE 为 451.44 kW、WMAPE 为 7.64%、 R^2 为 0.9713。相较历史加权轮廓，截断幂正弦的 RMSE 仅下降 0.65%，其主要价值并非追求统计精度，而是注入严格日照窗口、非负和单峰形态约束，避免夜间负出力或边界阶跃扰动后续储能调度。前一日持续性基准在 MAE 和 WMAPE 上略优，本文对此予以客观保留。

模型	MAE (kW)	RMSE (kW)	WMAPE (%)	R^2
典型日静态基准	867.52	1,031.68	19.27	0.8502
前一日持续性基准	332.33	486.36	7.38	0.9667
历史加权轮廓	344.72	454.41	7.66	0.9709
截断幂正弦完整模型	343.94	451.44	7.64	0.9713

表 13 光伏预测消融实验结果（测试集有效发电时段）

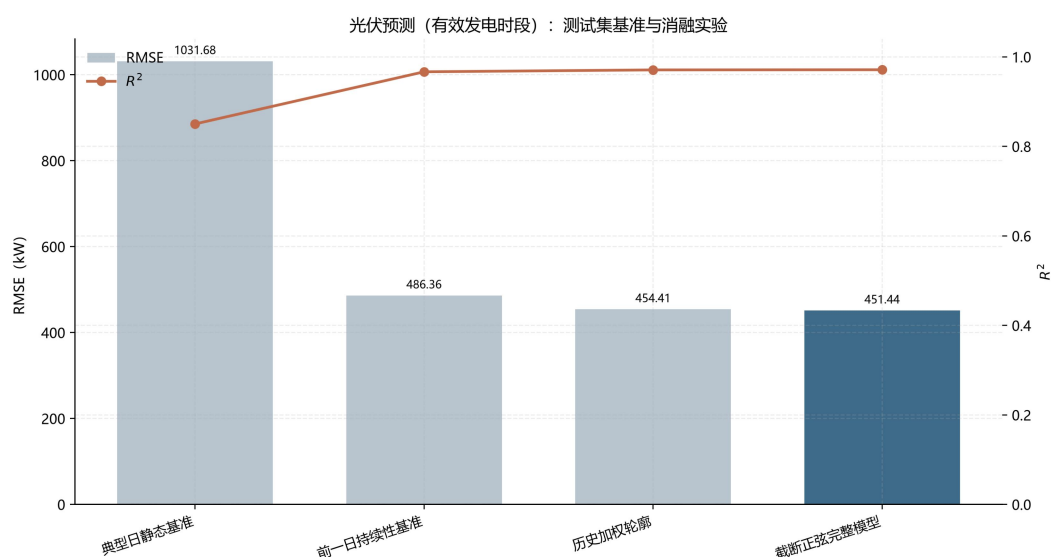


图 14 光伏预测基准对比实验

负荷与光伏组合形成的净负荷是后续购电优化的直接输入。完整模型在测试集上的净负荷 MAE 为 352.51 kW、RMSE 为 512.76 kW、WMAPE 为 11.73%、 R^2 为 0.9558。由于净负荷在光伏大发时段可能接近或穿越零点，普通 MAPE 会被小分母显著放大，故不作为净负荷预测的核心指标。

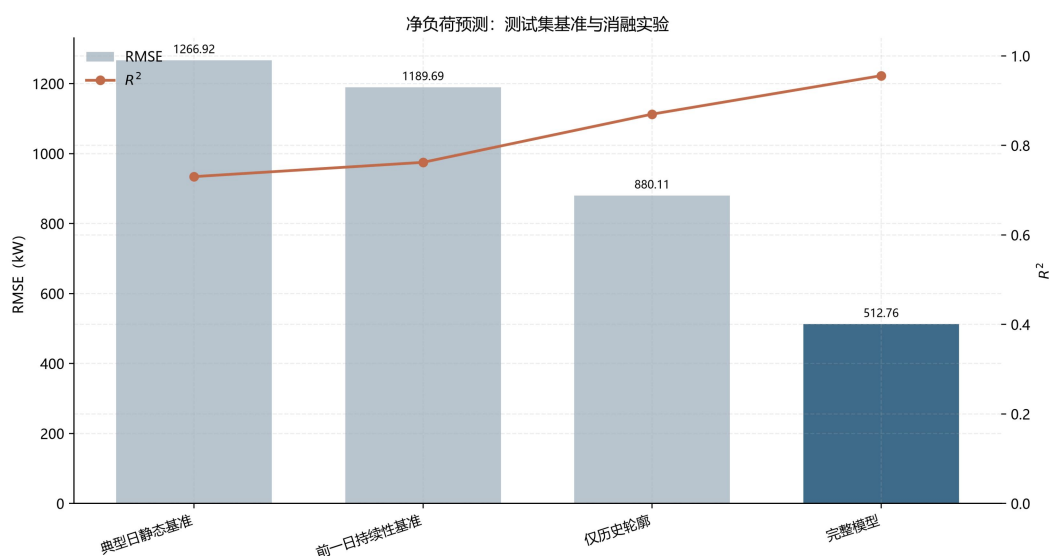


图 15 净负荷预测基准对比实验

6.3 实时电价预测消融实验

测试集上，完整电价预测模型取得 $MAE=0.0643$ 元/kWh、 $RMSE=0.0860$ 元/kWh、 $MAPE=10.35\%$ 和 $R^2=0.9352$ 。相较前一日持续性基准，RMSE 降低 30.90%。消融结果并非单调：仅保留傅里叶周期项时，RMSE 相对历史加权轮廓上升 42.32%，表明单独的低阶周期拟合会过度平滑局部尖峰；依次加入小波重构和前日偏差修正后，RMSE 分别改善 9.12% 和 19.68%。严格前向贝叶斯优化又使固定参数预测链的 RMSE 由 0.1017 元/kWh 降至 0.0860 元/kWh，改善 15.39%。

因此，傅里叶项的价值主要在于提供可解释的周期骨架，必须与小波、偏差修正及自适应参数联合使用，不能将其单独解释为精度增益。

模型	MAE (元/kWh)	RMSE (元/kWh)	WMAPE (%)	R ²
典型日静态基准	0.0977	0.1321	12.81	0.8471
前一日持续性基准	0.0853	0.1245	11.18	0.8643
历史加权轮廓	0.0755	0.0979	9.90	0.9161
仅傅里叶周期项	0.1074	0.1393	14.09	0.8301
傅里叶 + 小波	0.0972	0.1266	12.75	0.8597
增加前日偏差修正	0.0757	0.1017	9.94	0.9095
完整模型（含贝叶斯更新）	0.0643	0.0860	8.43	0.9352

表 14 实时电价预测消融实验结果（测试集）

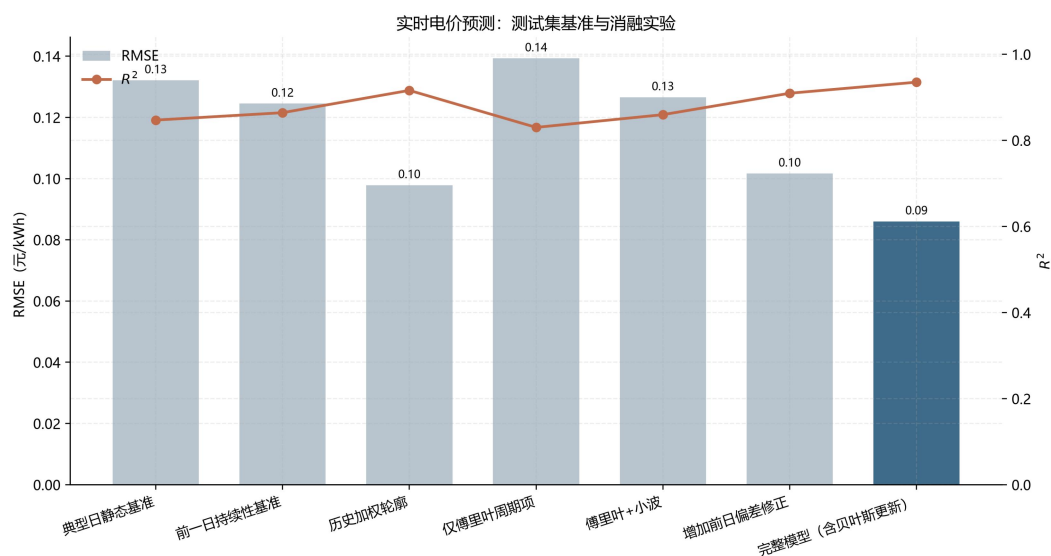


图 16 实时电价预测基准与消融实验

6.4 MPC 滚动优化策略评价

在固定电价条件下，问题三测试集的仅 0 点策略总费用为 2,836,057.56 元；增加 6:00 和 12:00 滚动修正后的 S2 策略费用为 2,708,241.41 元，相对节省 4.51%，紧急购电量削减 44.36%。在动态电价条件下，问题四 S2 策略相对仅 0 点策略节省 4.33%，并使紧急购电量减少 44.25%。结果表明，多时点滚动更新能够利用新到达的负荷观测和光伏预报修正日前计划，并在动态预测电价的引导下重新安排购电与储能充放电，从而降低预测偏差引起的紧急购电成本。

需要指出的是，增加更新次数并不必然单调降低总费用。18:00 时剩余调节窗口较短，且频繁调整可能产生额外的高价增购，因此完整四时刻 MPC 策略未必优于 0:00、6:00 和 12:00 三时刻组合。本文据此使用验证集费用选择 S2 策略，并在测试集上冻结检验，避免根据测试结果反向选取策略。

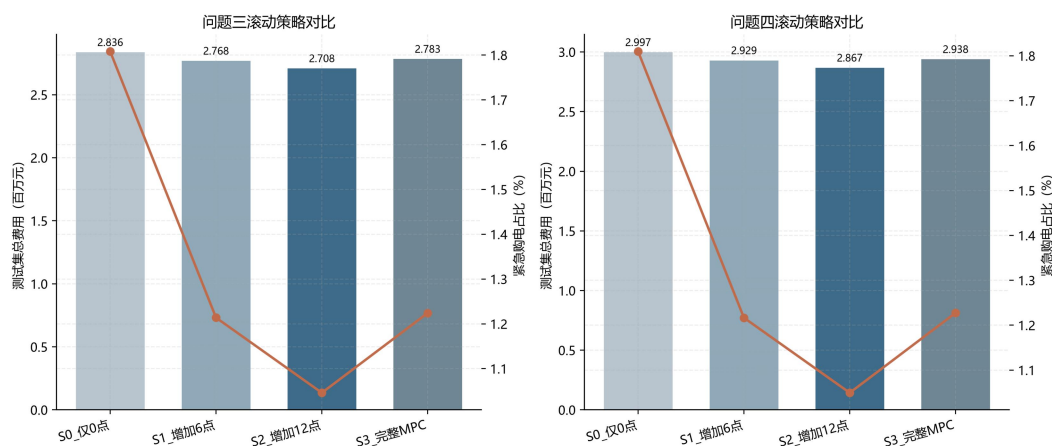


图 17 问题三、问题四 MPC 滚动策略经济性对比

6.5 各模块消融后的性能恶化率分析

定义的性能恶化率以消融模型 RMSE 为分母，衡量“移除模块后丢失了多少相对性能”。各模块在不同基线上逐一移除，且存在非线性交互，因此这些比例不是方差分解，也不能相加为 100%。负值表示该模块单独加入时使误差变大，不等价于其在完整组合中没有结构价值。

$$\text{Loss}_m = \frac{\text{RMSE}_{-m} - \text{RMSE}_{\text{full}}}{\text{RMSE}_{-m}} \times 100\%$$

对象	模块	消融或对照模型	完整或含该模块模型	性能恶化率 (%)
负荷预测	历史信息	典型日静态基准	仅历史加权轮廓	22.21
负荷预测	年度周期与星期回归	去除周期回归	完整模型	52.27
负荷预测	小波残差修正	去除小波修正	完整模型	24.10
负荷预测	贝叶斯参数更新	去除贝叶斯更新	完整模型	30.17
光伏预测	截断幂正弦	历史加权轮廓	截断幂正弦完整模型	0.65
电价预测	历史信息	典型日静态基准	历史加权轮廓	25.93
电价预测	傅里叶周期提取	历史加权轮廓	仅傅里叶周期项	-42.32
电价预测	小波残差重构	仅傅里叶周期项	傅里叶 + 小波	9.12
电价预测	前日偏差修正	傅里叶 + 小波	增加前日偏差修正	19.68
电价预测	贝叶斯参数更新	增加前日偏差修正	完整模型	15.39
问题三购电优化	6:00/12:00 MPC 滚动	S0 仅 0 点	S2 增加 12 点	4.51
问题四购电优化	6:00/12:00 MPC 滚动	S0 仅 0 点	S2 增加 12 点	4.33

表 15 各模块消融后的性能恶化率 (Performance Loss Ratio)

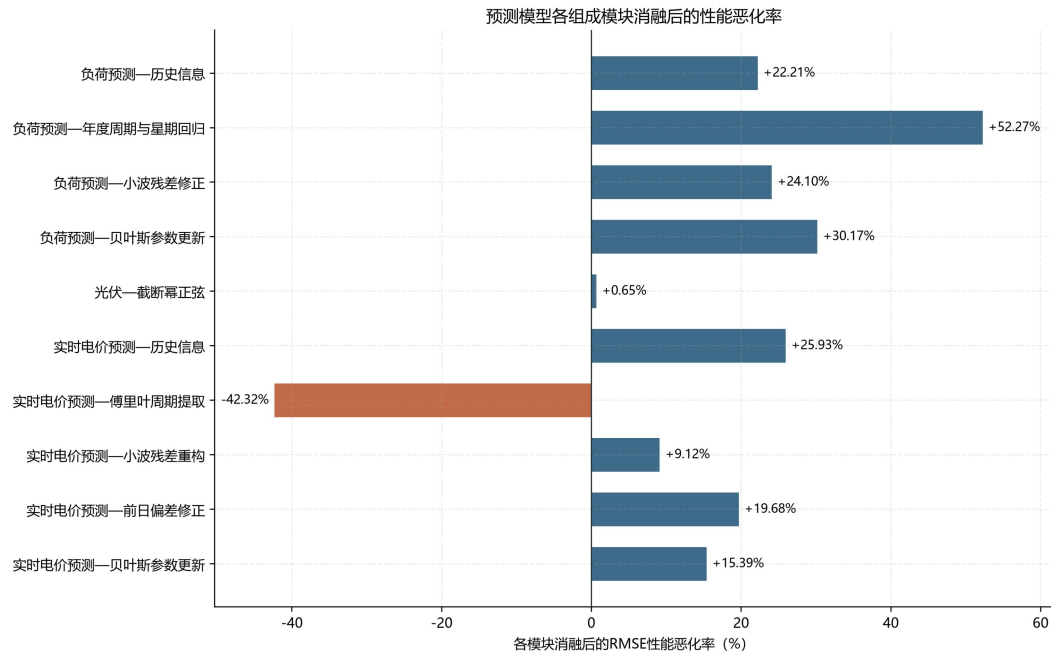


图 18 各模块消融后的性能恶化率

负荷预测中，移除年度周期与星期回归后的性能恶化率为 52.27%，表明周期结构是重要组成部分；移除小波修正和贝叶斯更新后的性能恶化率分别为 24.10%和 30.17%。电价预测中，傅里叶项对应-42.32%，说明低阶傅里叶若脱离小波、偏差修正和贝叶斯更新会过度平滑尖峰。各模块存在非线性交互与协同增益，因此这些消融比例不能直接代数累加，也不应解释为总贡献的组成份额。

6.6 统计显著性检验

令 d_i 为第 i 个测试日的对照模型 MAE 减完整模型 MAE。单侧 Wilcoxon 符号秩检验考察差值分布的位置是否大于 0，不要求日均误差服从正态分布；以 $\alpha = 0.05$ 判断改善是否具有跨测试日稳定性，并保留持续性基准对比不显著的结果。

$$d_i = \text{MAE}_i^{\text{base}} - \text{MAE}_i^{\text{full}}, H_0: d_i \leq 0; H_1: d_i > 0, \alpha = 0.05$$

对象	完整模型 vs 对照	日均 MAE 差值	p 值	显著
负荷预测	vs 典型日静态基准	531.61	8.15×10^{-12}	是
负荷预测	vs 前一日持续性基准	411.91	0.127	否
负荷预测	vs 仅历史加权轮廓	372.06	3.14×10^{-11}	是
负荷预测	vs 去除周期回归	375.53	2.33×10^{-11}	是
负荷预测	vs 去除小波修正	121.75	8.57×10^{-12}	是
负荷预测	vs 去除贝叶斯更新	163.16	1.26×10^{-10}	是
电价预测	vs 典型日静态基准	0.0334	1.06×10^{-6}	是
电价预测	vs 前一日持续性基准	0.0210	0.060	否
电价预测	vs 历史加权轮廓	0.0112	6.00×10^{-4}	是
电价预测	vs 仅傅里叶周期项	0.0431	4.87×10^{-11}	是
电价预测	vs 傅里叶 + 小波	0.0329	1.18×10^{-9}	是
电价预测	vs 增加前日偏差修正	0.0115	1.15×10^{-6}	是

表 16 配对 Wilcoxon 检验结果 (测试集)

负荷预测的 6 组对照中有 5 组显著；相对于前一日持续性基准的日均 MAE 改进不显著 ($p=0.127$)。电价预测同样有 5 组显著，相对于前一日持续性基准的改进未达到 5%显著水平

($p=0.060$)。完整模型在相应 RMSE 和 R^2 指标上仍有优势，但本文不将这两组结果表述为统计显著优越。

6.7 综合评价

综合结果表明，性能提升来自周期特征、局部残差、短期偏差、贝叶斯参数更新和滚动决策的协同。代表日曲线用于说明机制，最终泛化结论仍以 60 个测试日的误差、费用与显著性检验为依据。

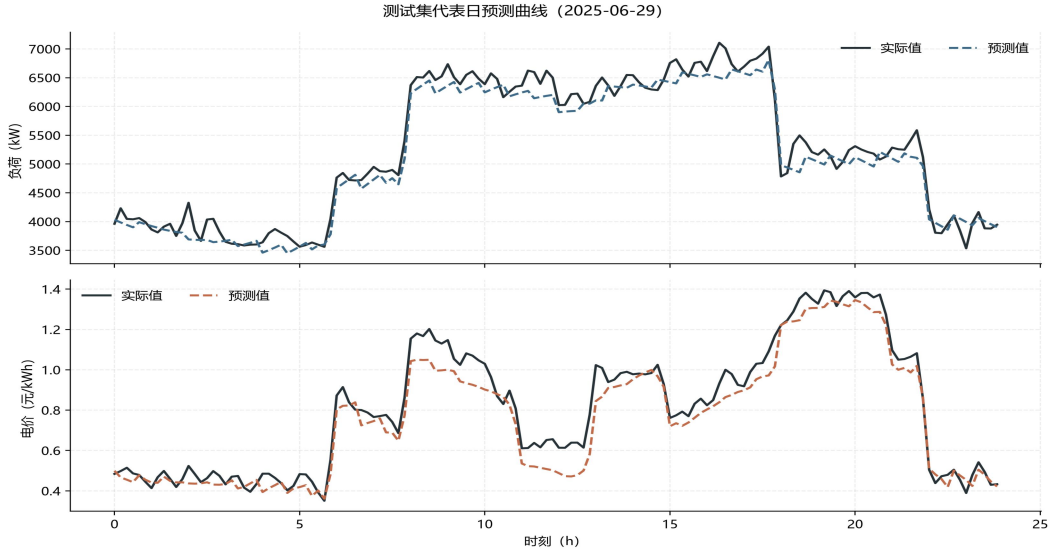


图 19 测试集代表日负荷和实时电价预测曲线

总体上，模型能够把样本外预测改进转化为购电费用和紧急购电量的下降；其主要不足集中在突发尖峰、低功率边界以及晚间短调节窗口等场景。

七、模型的评价、改进与推广

7.1 模型优点

(1) 预测结构清晰且可解释。傅里叶级数负责周期骨架，Haar 小波修正局部波动，岭回归和历史加权用于跨日变化，光伏模型则加入日照窗口与非负约束；各模块职责明确，便于复现和消融分析。

(2) 形成从预测到决策的完整闭环。问题二把预测误差通过报童模型转化为 80% 分位数安全裕度，再由滚动 LP 和真实回放把统计误差映射为购电费用与紧急购电风险，避免只追求预测指标而忽略决策效果。

(3) 能够量化新信息与动态电价的经济价值。问题三利用 6:00 和 12:00 新预报实施 MPC 调整，测试集费用较 S_0 下降 4.51%，紧急购电量下降 44.36%；问题四进一步将实时电价预测嵌入滚动 LP 与 MPC-LP，使模型能够适应价格波动。

(4) 评价流程较规范。参数与策略仅由训练集和验证集确定，测试集用于最终泛化评价，并结合基准模型、消融实验和配对 Wilcoxon 检验验证改进的稳定性。

7.2 模型局限性

(1) 极端场景预测能力有限。傅里叶模型对稳定周期表现较好，但对极端天气、突发负荷和异常电价尖峰的刻画仍不足；光伏模型对云层遮挡等快速波动的适应能力也有限。

(2) 部分指标并非全面优于简单基准。光伏预测中，前一日持续性模型在 MAE 和 WMAPE 上略优于截断幂正弦模型，说明模型优劣应结合 RMSE、 R^2 、形态合理性及最终购电成本综合判断。

(3) 风险修正依赖近期误差分布稳定性。当前安全裕度依据近期历史预测误差估计，当负荷或天气发生结构性变化时，其代表性可能下降；同时尚未显式建立负荷、光伏和电价误差的联合概率分布。

7.3 模型改进与推广

(1) 引入概率预测和不确定性优化。可进一步建立负荷、光伏和电价的概率预测模型，并结合场景生成、随机规划、机会约束、鲁棒优化或 CVaR 方法，提高模型对极端风险的应对能力。

(2) 完善储能退化模型。将电池循环寿命、SOC 状态、自放电、温度和效率变化纳入模型；对于充放电互斥及设备启停等离散约束，可进一步扩展为混合整数规划。

(3) 建立动态更新机制。若用于实际部署，可在保持参数选择规则固定的前提下，于每个新决策日滚动纳入所有已经发生的历史数据重新估计预测模型，以提高历史信息利用率；若未来能够获得更多发布时间的独立预报，还可根据新增信息带来的边际购电收益和风险变化，动态确定 MPC 更新时点。

(4) 拓展源—网—荷—储协同优化。在现有购电和储能调度基础上，进一步加入可转移负荷、电动汽车、需求响应、分布式能源交易等决策变量，并结合购电容量、碳交易和绿证交易等约束，提高模型的实际应用价值。

(5) 推动工程化应用。模型可推广至园区微网、商业建筑、居民小区及光储充系统，但实际部署时仍需结合当地负荷特征、储能参数和市场规则进行重新标定，并完善数据接口、异常检测、计算时限和控制安全机制。

——AI 工具使用声明：本参赛队在竞赛过程中使用了 AI 工具，主要用于语言润色、代码调试，详细使用情况见支撑材料。

附录

附录 A 支撑材料文件列表

支撑材料包括 AI 工具使用详情、四个问题的完整源程序、运行依赖说明、结果工作簿及复现说明。下列文件名与支撑材料压缩包中的目录结构保持一致。赛题提供的原始附件不重复提交。

序号	文件名	内容说明
1	AI 工具使用详情.pdf	AI 工具名称、使用环节、典型交互方式及人工修改与核验说明
2	提交说明.md	支撑材料目录、运行环境、输入文件结构、运行顺序和复现注意事项
3	程序代码/one.py	问题一的典型日负荷与光伏拟合、储能调度和购电线性规划
4	程序代码/two.py	问题二的日前滚动预测、风险裕度、贝叶斯参数选择及 48 小时滚动线性规划
5	程序代码/three.py	问题三的多时点预报更新、偏差修正及 MPC 滚动调整购电
6	程序代码/four.py	问题四的动态电价预测及问题 4-2、问题 4-3 优化求解
7	程序代码/requirements.txt	Python 程序运行所需第三方库及版本
8	结果文件/result1.xlsx	问题一计划购电量与储能充放电结果
9	结果文件/result2.xlsx	问题二计划购电、充放电及紧急购电结果
10	结果文件/result3.xlsx	问题三计划购电、调整购电、充放电及紧急购电结果
11	结果文件/result4-2.xlsx	动态电价下问题 4-2 的计划购电、充放电及紧急购电结果
12	结果文件/result4-3.xlsx	动态电价下问题 4-3 的计划购电、调整购电、充放电及紧急购电结果

附录 B 运行环境与复现方法

B.1 软件与依赖

程序采用 Python 3.12 或更高版本运行，依赖库及冻结版本如下：

```
numpy==2.5.3
scipy==1.18.1
openpyxl==3.1.5
```

在项目根目录执行以下命令安装依赖：

```
python -m pip install -r requirements.txt
```

B.2 输入与输出文件

四个程序置于同一项目根目录。输入文件为赛题提供的附件 1、附件 2 小区负载、附件 2 光伏发电实际功率、附件 3、附件 4 及附件 5 中的五个结果模板；输出文件分别为 result1.xlsx、result2.xlsx、result3.xlsx、result4-2.xlsx 和 result4-3.xlsx。

推荐使用以下相对目录结构：

```
项目根目录/
├── one.py
├── two.py
├── three.py
├── four.py
```

```
|— requirements.txt
|— C 题/附件/
    |— csv/附件 1.csv
    |— csv/附件 2_小区负载.csv
    |— csv/附件 2_光伏发电实际功率.csv
    |— csv/附件 3.csv
    |— 附件 4.xlsx
    |— 附件 5/result1.xlsx 至 result4-3.xlsx
```

B.3 运行顺序

在项目根目录依次执行：

```
python one.py
python two.py
python three.py
python four.py
```

正式复现默认调用已经通过严格前向贝叶斯优化确定并冻结的参数轨迹，无需重新执行耗时的参数寻优。运行前应关闭 Excel 中已打开的结果工作簿，避免 Windows 文件占用导致输出失败。

附录 C 完整源程序

以下代码为模型计算和结果工作簿生成所实际采用的完整程序。源代码同时以独立文件形式置于支撑材料的“程序代码”目录中。

C.1 问题一程序 one.py

程序说明：读取典型日负荷、光伏及电价数据，采用傅里叶级数与 Haar 小波修正负荷轮廓，以截断幂正弦拟合光伏出力，并通过线性规划求解购电、充电、放电、弃电与储能状态。程序完成约束校验后写入 result1.xlsx。

```
# 主模型直接使用附件 1 的原始 10 分钟数据求最优购电策略。
# 改进模型先以二阶傅里叶级数提取负载的日周期趋势，再用三级 Haar
# 小波软阈值重构傅里叶残差中的局部变化；光伏仍采用截断幂正弦拟合。
# 随后用同一线性规划求解，并同时报告纯傅里叶模型和小波改进模型。

# 默认将傅里叶—小波负荷和截断正弦光伏对应的策略写入 result1.xlsx；
# 仍可通过 --result-scenario actual 输出原始数据理论最优解。

# 默认输入：C 题/附件/csv/附件 1.csv
# 默认模板：C 题/附件/附件 5/result1.xlsx
# 默认输出：C 题/result1.xlsx
# 默认对照：C 题/问题一拟合对比.csv

# 依赖：numpy、scipy、openpyxl（Haar 小波由本文件实现，无需 PyWavelets）
# 运行：python one.py
```

```
from __future__ import annotations

import argparse
import csv
import math
import shutil
from dataclasses import dataclass
from pathlib import Path
from typing import Iterable

import numpy as np
from openpyxl import load_workbook

ROOT = Path(__file__).resolve().parent
DEFAULT_INPUT = ROOT / "C 题" / "附件" / "csv" / "附件 1.csv"
DEFAULT_TEMPLATE = ROOT / "C 题" / "附件" / "附件 5" / "result1.xlsx"
DEFAULT_OUTPUT = ROOT / "C 题" / "result1.xlsx"
DEFAULT_COMPARISON = ROOT / "C 题" / "问题一拟合对比.csv"

@dataclass(frozen=True)
class Config:
    dt_hours: float = 1.0 / 6.0
    capacity_kwh: float = 12000.0
    soc_min_kwh: float = 1200.0
    soc_max_kwh: float = 10800.0
    initial_soc_kwh: float = 6000.0
    max_charge_power_kw: float = 5000.0
    max_discharge_power_kw: float = 5000.0
    charge_efficiency: float = 0.90
    discharge_efficiency: float = 0.90
    throughput_tie_breaker: float = 1.0e-7

    @property
    def max_charge_kwh(self) -> float:
        return self.max_charge_power_kw * self.dt_hours

    @property
    def max_discharge_kwh(self) -> float:
        return self.max_discharge_power_kw * self.dt_hours

@dataclass
class InputData:
    times: list[str]
    price: np.ndarray
```

```

load_kw: np.ndarray
pv_kw: np.ndarray

@dataclass
class FitResult:
    load_fourier_kw: np.ndarray
    load_kw: np.ndarray
    pv_kw: np.ndarray
    load_fourier_mae: float
    load_fourier_rmse: float
    load_fourier_r2: float
    load_mae: float
    load_rmse: float
    load_r2: float
    pv_mae: float
    pv_rmse: float
    pv_r2: float
    pv_amplitude_kw: float
    sunrise_hour: float
    sunset_hour: float
    pv_shape: float
    wavelet_name: str
    wavelet_level: int

@dataclass
class Solution:
    grid_kwh: np.ndarray
    charge_kwh: np.ndarray
    discharge_kwh: np.ndarray
    curtailment_kwh: np.ndarray
    soc_kwh: np.ndarray
    total_cost_yuan: float

def _as_float(value: object, field: str, row: int) -> float:
    try:
        number = float(value)
    except (TypeError, ValueError) as exc:
        raise ValueError(f"附件 1 第{row}行的{field}不是数值: {value!r}") from exc
    if not math.isfinite(number):
        raise ValueError(f"附件 1 第{row}行的{field}不是有限数值: {value!r}")
    return number

def read_input(path: Path) -> InputData:
    if not path.exists():
        raise FileNotFoundError(f"找不到附件 1 CSV: {path}")
    with path.open("r", encoding="utf-8-sig", newline="") as file:
        rows = list(csv.reader(file))
    if len(rows) != 145 or len(rows[0]) < 4:
        raise ValueError("附件 1 应包含 1 行表头、144 行数据和至少 4 列。")

    times, price, load, pv = [], [], [], []
    for row_number, row in enumerate(rows[1:], start=2):
        if len(row) < 4:
            raise ValueError(f"附件 1 第{row_number}行少于 4 列。")
        times.append(row[0].strip())
        price.append(_as_float(row[1], "电价", row_number))
        load.append(_as_float(row[2], "小区负载", row_number))
        pv.append(_as_float(row[3], "光伏预测功率", row_number))
    arrays = [np.asarray(values, dtype=float) for values in (price, load, pv)]
    if any(np.any(values < 0) for values in arrays):
        raise ValueError("电价、负载和光伏功率均不应为负数。")
    return InputData(times, arrays[0], arrays[1], arrays[2])

def _start_minute(interval_label: object) -> int:
    label = str(interval_label).strip()
    if "-" not in label:
        raise ValueError(f"无法解析时间段: {label!r}")
    start = label.split("-", 1)[0].replace("+1", "")
    hour_text, minute_text = start.split(":", 1)
    return (int(hour_text) % 24) * 60 + int(minute_text)

def template_chronological_order(template: Path) -> tuple[list[int], list[str]]:
    if not template.exists():
        raise FileNotFoundError(f"找不到 result1 模板: {template}")

```

```

workbook = load_workbook(template, read_only=True, data_only=True)
sheet = workbook.worksheets[0]
labels = [sheet.cell(row, 1).value for row in range(2, sheet.max_row + 1)]
workbook.close()
if len(labels) != 144 or any(label is None for label in labels):
    raise ValueError("result1 模板应包含 144 个完整时间段。")
minutes = [_start_minute(label) for label in labels]
if sorted(minutes) != list(range(0, 1440, 10)):
    raise ValueError("result1 模板未完整覆盖 144 个 10 分钟时段。")
order = sorted(range(144), key=minutes.__getitem__)
return order, [str(label) for label in labels]

def _metrics(actual: np.ndarray, fitted: np.ndarray) -> tuple[float, float, float]:
    residual = actual - fitted
    mae = float(np.mean(np.abs(residual)))
    rmse = float(np.sqrt(np.mean(residual**2)))
    total = float(np.sum((actual - np.mean(actual)) ** 2))
    r2 = 1.0 - float(np.sum(residual**2)) / total if total > 0 else 1.0
    return mae, rmse, r2

def fit_load_fourier(hours: np.ndarray, load_kw: np.ndarray) -> np.ndarray:
    omega = 2.0 * np.pi / 24.0
    design = np.column_stack(
        [
            np.ones_like(hours),
            np.cos(omega * hours),
            np.sin(omega * hours),
            np.cos(2.0 * omega * hours),
            np.sin(2.0 * omega * hours),
        ]
    )
    coefficients, *_ = np.linalg.lstsq(design, load_kw, rcond=None)
    return np.maximum(0.0, design @ coefficients)

def _haar_dwt(signal: np.ndarray, level: int) -> tuple[np.ndarray, list[np.ndarray]]:
    approximation = np.asarray(signal, dtype=float).copy()
    details: list[np.ndarray] = []
    scale = math.sqrt(2.0)
    for _ in range(level):
        if approximation.size % 2:
            raise ValueError("Haar 小波每层输入长度必须为偶数。")
        even = approximation[0::2]
        odd = approximation[1::2]
        details.append((even - odd) / scale)
        approximation = (even + odd) / scale
    return approximation, details

def _haar_idwt(approximation: np.ndarray, details: list[np.ndarray]) -> np.ndarray:
    reconstructed = np.asarray(approximation, dtype=float).copy()
    scale = math.sqrt(2.0)
    for detail in reversed(details):
        if reconstructed.size != detail.size:
            raise ValueError("Haar 小波重构时系数长度不匹配。")
        restored = np.empty(reconstructed.size * 2, dtype=float)
        restored[0::2] = (reconstructed + detail) / scale
        restored[1::2] = (reconstructed - detail) / scale
        reconstructed = restored
    return reconstructed

def wavelet_correct_fourier_residual(
    load_kw: np.ndarray, fourier_kw: np.ndarray, level: int = 3
) -> np.ndarray:
    residual = np.asarray(load_kw, dtype=float) - np.asarray(fourier_kw, dtype=float)
    if residual.size % (2**level):
        raise ValueError(f"样本数必须能被 2^{level} 整除。")
    approximation, details = _haar_dwt(residual, level)
    thresholded: list[np.ndarray] = []
    n = residual.size
    for detail in details:
        # MAD 是对高斯噪声标准差的稳健估计; 通用阈值避免把尖锐噪声当趋势。
        sigma = float(np.median(np.abs(detail - np.median(detail)))) / 0.6745
        threshold = sigma * math.sqrt(2.0 * math.log(n))
        thresholded.append(
            np.sign(detail) * np.maximum(np.abs(detail) - threshold, 0.0)
        )

```

```

corrected_residual = _haar_idwt(approximation, thresholded)
return np.maximum(0.0, fourier_kw + corrected_residual)

def fit_pv_truncated_sine(
    hours: np.ndarray, pv_kw: np.ndarray
) -> tuple[np.ndarray, float, float, float, float]:
    peak = float(np.max(pv_kw))
    if peak <= 0:
        return np.zeros_like(pv_kw), 0.0, 0.0, 0.0, 1.0
    positive = np.flatnonzero(pv_kw > max(1.0e-6, peak * 1.0e-5))
    first_hour = float(hours[positive[0]])
    last_hour = float(hours[positive[-1]])
    sunrise_grid = np.arange(max(0.0, first_hour - 1.5), first_hour + 0.51, 1 / 12)
    sunset_grid = np.arange(last_hour - 0.5, min(24.0, last_hour + 1.51) + 1e-9, 1 / 12)
    shape_grid = np.linspace(0.5, 3.0, 101)

    best_sse = math.inf
    best = (np.zeros_like(pv_kw), 0.0, first_hour, last_hour, 1.0)
    for sunrise in sunrise_grid:
        for sunset in sunset_grid:
            if sunset - sunrise < 6.0:
                continue
            phase = np.pi * (hours - sunrise) / (sunset - sunrise)
            sine = np.zeros_like(hours)
            daylight = (hours >= sunrise) & (hours <= sunset)
            sine[daylight] = np.maximum(0.0, np.sin(phase[daylight]))
            for shape in shape_grid:
                basis = sine**shape
                denominator = float(np.dot(basis, basis))
                if denominator <= 0:
                    continue
                amplitude = max(0.0, float(np.dot(basis, pv_kw) / denominator))
                fitted = amplitude * basis
                sse = float(np.sum((pv_kw - fitted) ** 2))
                if sse < best_sse:
                    best_sse = sse
                    best = (fitted, amplitude, float(sunrise), float(sunset), float(shape))
    return best

def fit_profiles(data: InputData, order: Iterable[int], cfg: Config) -> FitResult:
    order_array = np.asarray(list(order), dtype=int)
    hours = np.arange(144, dtype=float) * cfg.dt_hours
    load_actual = data.load_kw[order_array]
    pv_actual = data.pv_kw[order_array]
    load_fourier = fit_load_fourier(hours, load_actual)
    load_fitted = wavelet_correct_fourier_residual(
        load_actual, load_fourier, level=3
    )
    pv_fitted, amplitude, sunrise, sunset, shape = fit_pv_truncated_sine(
        hours, pv_actual
    )
    load_fourier_mae, load_fourier_rmse, load_fourier_r2 = _metrics(
        load_actual, load_fourier
    )
    load_mae, load_rmse, load_r2 = _metrics(load_actual, load_fitted)
    pv_mae, pv_rmse, pv_r2 = _metrics(pv_actual, pv_fitted)
    return FitResult(
        load_fourier,
        load_fitted,
        pv_fitted,
        load_fourier_mae,
        load_fourier_rmse,
        load_fourier_r2,
        load_mae,
        load_rmse,
        load_r2,
        pv_mae,
        pv_rmse,
        pv_r2,
        amplitude,
        sunrise,
        sunset,
        shape,
        "Haar",
        3,
    )

```

```

def solve_linear_program(
    price: np.ndarray, load_kw: np.ndarray, pv_kw: np.ndarray, cfg: Config
) -> Solution:
    try:
        from scipy.optimize import linprog
    except ModuleNotFoundError as exc:
        raise ModuleNotFoundError(
            "缺少 scipy, 请先执行: python -m pip install scipy"
        ) from exc

    n = len(price)
    if n != 144 or len(load_kw) != n or len(pv_kw) != n:
        raise ValueError("价格、负载和光伏数组都必须包含 144 个时段。")
    load_energy = load_kw * cfg.dt_hours
    pv_energy = pv_kw * cfg.dt_hours

    g0, c0, d0, w0, e0 = 0, n, 2 * n, 3 * n, 4 * n
    count = 5 * n + 1
    objective = np.zeros(count)
    objective[g0 : g0 + n] = price
    objective[c0 : c0 + n] = cfg.throughput_tie_breaker
    objective[d0 : d0 + n] = cfg.throughput_tie_breaker

    a_eq = np.zeros((2 * n, count), dtype=float)
    b_eq = np.zeros(2 * n, dtype=float)
    for t in range(n):
        # g + PV + d = load + c + w
        a_eq[t, g0 + t] = 1.0
        a_eq[t, c0 + t] = -1.0
        a_eq[t, d0 + t] = 1.0
        a_eq[t, w0 + t] = -1.0
        b_eq[t] = load_energy[t] - pv_energy[t]
        # e[t+1] = e[t] + eta_c*c - d/eta_d
        row = n + t
        a_eq[row, c0 + t] = -cfg.charge_efficiency
        a_eq[row, d0 + t] = 1.0 / cfg.discharge_efficiency
        a_eq[row, e0 + t] = -1.0
        a_eq[row, e0 + t + 1] = 1.0

    bounds: list[tuple[float | None, float | None]] = []
    bounds.extend([(0.0, None)] * n)
    bounds.extend([(0.0, cfg.max_charge_kwh)] * n)
    bounds.extend([(0.0, cfg.max_discharge_kwh)] * n)
    bounds.extend([(0.0, None)] * n)
    bounds.extend([(cfg.soc_min_kwh, cfg.soc_max_kwh)] * (n + 1))
    bounds[e0] = (cfg.initial_soc_kwh, cfg.initial_soc_kwh)
    bounds[e0 + n] = (cfg.initial_soc_kwh, cfg.initial_soc_kwh)

    result = linprog(
        objective,
        A_eq=a_eq,
        b_eq=b_eq,
        bounds=bounds,
        method="highs",
        options={"presolve": True},
    )
    if not result.success:
        raise RuntimeError(f"线性规划失败: status={result.status}, {result.message}")
    x = np.asarray(result.x, dtype=float)
    solution = Solution(
        x[g0 : g0 + n],
        x[c0 : c0 + n],
        x[d0 : d0 + n],
        x[w0 : w0 + n],
        x[e0 : e0 + n + 1],
        float(np.dot(price, x[g0 : g0 + n])),
    )
    validate_solution(solution, load_energy, pv_energy, price, cfg)
    return solution

def validate_solution(
    solution: Solution,
    load_energy: np.ndarray,
    pv_energy: np.ndarray,
    price: np.ndarray,
    cfg: Config,
    tolerance: float = 1.0e-5,
) -> None:
    balance = (

```

```

        solution.grid_kwh
        + pv_energy
        + solution.discharge_kwh
        - load_energy
        - solution.charge_kwh
        - solution.curtailment_kwh
    )
    next_soc = (
        solution.soc_kwh[:-1]
        + cfg.charge_efficiency * solution.charge_kwh
        - solution.discharge_kwh / cfg.discharge_efficiency
    )
    checks = {
        "电量平衡": float(np.max(np.abs(balance))),
        "SOC 递推": float(np.max(np.abs(solution.soc_kwh[1:] - next_soc))),
        "费用复算": abs(solution.total_cost_yuan - float(np.dot(price, solution.grid_kwh))),
        "初始 SOC": abs(solution.soc_kwh[0] - cfg.initial_soc_kwh),
        "终止 SOC": abs(solution.soc_kwh[-1] - cfg.initial_soc_kwh),
    }
    failed = {name: value for name, value in checks.items() if value > tolerance}
    if failed:
        raise RuntimeError(f"求解结果校验失败: {failed}")
    if np.min(solution.soc_kwh) < cfg.soc_min_kwh - tolerance:
        raise RuntimeError("SOC 低于下限。")
    if np.max(solution.soc_kwh) > cfg.soc_max_kwh + tolerance:
        raise RuntimeError("SOC 高于上限。")
    if np.max(np.minimum(solution.charge_kwh, solution.discharge_kwh)) > 1.0e-4:
        raise RuntimeError("出现同时充放电，应改用 MILP 加入互斥变量。")

def evaluate_fitted_dispatch(
    fitted: Solution,
    actual_price: np.ndarray,
    actual_load_kw: np.ndarray,
    actual_pv_kw: np.ndarray,
    cfg: Config,
) -> tuple[np.ndarray, np.ndarray, float]:
    net_need = (
        actual_load_kw * cfg.dt_hours
        + fitted.charge_kwh
        - actual_pv_kw * cfg.dt_hours
        - fitted.discharge_kwh
    )
    grid = np.maximum(0.0, net_need)
    curtailment = np.maximum(0.0, -net_need)
    return grid, curtailment, float(np.dot(actual_price, grid))

def _to_template_order(values: np.ndarray, order: list[int]) -> np.ndarray:
    mapped = np.empty_like(values)
    mapped[np.asarray(order, dtype=int)] = values
    return mapped

def write_result(template: Path, output: Path, order: list[int], solution: Solution) -> None:
    output.parent.mkdir(parents=True, exist_ok=True)
    if template.resolve() == output.resolve():
        raise ValueError("输出路径不能覆盖官方模板。")
    shutil.copy2(template, output)
    workbook = load_workbook(output)
    plan_sheet = workbook["计划购电量"]
    storage_sheet = workbook["充放电量"]
    for row, value in enumerate(_to_template_order(solution.grid_kwh, order), start=2):
        plan_sheet.cell(row, 2, round(float(value), 4))
    for block in range(6):
        start, stop = block * 24, (block + 1) * 24
        charge = round(float(np.sum(solution.charge_kwh[start:stop])), 4)
        discharge = round(float(np.sum(solution.discharge_kwh[start:stop])), 4)
        storage_sheet.cell(2 + block, 2, charge)
        storage_sheet.cell(2 + block, 3, discharge)
    storage_sheet["E2"] = round(float(solution.soc_kwh[0]), 4)
    storage_sheet["E3"] = round(float(solution.soc_kwh[-1]), 4)
    workbook.save(output)

def write_comparison(
    output: Path,
    price: np.ndarray,
    load_actual: np.ndarray,
    pv_actual: np.ndarray,

```

```

fit: FitResult,
actual_solution: Solution,
fourier_solution: Solution,
wavelet_solution: Solution,
fourier_grid_actual: np.ndarray,
wavelet_grid_actual: np.ndarray,
) -> None:
    output.parent.mkdir(parents=True, exist_ok=True)
    headers = [
        "时间",
        "电价(元/kWh)",
        "实际负载(kW)",
        "傅里叶拟合负载(kW)",
        "傅里叶-小波拟合负载(kW)",
        "实际光伏功率(kW)",
        "截断正弦拟合光伏功率(kW)",
        "原始数据最优购电量(kWh)",
        "傅里叶场景计划购电量(kWh)",
        "小波改进场景计划购电量(kWh)",
        "傅里叶策略在真实数据下购电量(kWh)",
        "小波改进策略在真实数据下购电量(kWh)",
        "原始策略充电量(kWh)",
        "原始策略放电量(kWh)",
        "原始策略 SOC 期末值(kWh)",
    ]
    with output.open("w", encoding="utf-8-sig", newline="") as file:
        writer = csv.writer(file, lineterminator="\n")
        writer.writerow(headers)
        for t in range(144):
            writer.writerow(
                [
                    f"{t // 6:02d}:{(t % 6) * 10:02d}",
                    f"{price[t]:.6f}",
                    f"{load_actual[t]:.6f}",
                    f"{fit.load_fourier_kw[t]:.6f}",
                    f"{fit.load_kw[t]:.6f}",
                    f"{pv_actual[t]:.6f}",
                    f"{fit.pv_kw[t]:.6f}",
                    f"{actual_solution.grid_kwh[t]:.6f}",
                    f"{fourier_solution.grid_kwh[t]:.6f}",
                    f"{wavelet_solution.grid_kwh[t]:.6f}",
                    f"{fourier_grid_actual[t]:.6f}",
                    f"{wavelet_grid_actual[t]:.6f}",
                    f"{actual_solution.charge_kwh[t]:.6f}",
                    f"{actual_solution.discharge_kwh[t]:.6f}",
                    f"{actual_solution.soc_kwh[t + 1]:.6f}",
                ]
            )

def print_summary(
    labels: list[str],
    order: list[int],
    fit: FitResult,
    actual: Solution,
    fourier: Solution,
    wavelet: Solution,
    fourier_actual_cost: float,
    wavelet_actual_cost: float,
) -> None:
    grid_by_template = _to_template_order(actual.grid_kwh, order)
    requested = {
        "10:00-10:10",
        "12:00-12:10",
        "14:00-14:10",
        "16:00-16:10",
        "18:00-18:10",
        "20:00-20:10",
    }
    print("\n 问题一原始数据线性规划")
    for label, value in zip(labels, grid_by_template):
        if label in requested:
            print(f"    {label}: {value:.4f} kWh")
    print(f" 全天购电量: {np.sum(actual.grid_kwh):.4f} kWh")
    print(f" 全天购电费: {actual.total_cost_yuan:.4f} 元")
    print(f" 弃光量: {np.sum(actual.curtailment_kwh):.4f} kWh")
    print(f" SOC 范围: {np.min(actual.soc_kwh):.4f} ~ {np.max(actual.soc_kwh):.4f} kWh")

    fourier_gap = fourier_actual_cost - actual.total_cost_yuan
    wavelet_gap = wavelet_actual_cost - actual.total_cost_yuan

```

```

fourier_gap_rate = (
    100.0 * fourier_gap / actual.total_cost_yuan if actual.total_cost_yuan else 0.0
)
wavelet_gap_rate = (
    100.0 * wavelet_gap / actual.total_cost_yuan if actual.total_cost_yuan else 0.0
)
print("\n 周期拟合、小波改进与策略对照")
print(
    "    傅里叶负载拟合: "
    f"MAE={fit.load_fourier_mae:.4f} kW, "
    f"RMSE={fit.load_fourier_rmse:.4f} kW, R^2={fit.load_fourier_r2:.6f}"
)
print(
    f"    傅里叶+{fit.wavelet_name} {fit.wavelet_level}层小波负载拟合: "
    f"MAE={fit.load_mae:.4f} kW, RMSE={fit.load_rmse:.4f} kW, "
    f"R^2={fit.load_r2:.6f}"
)
print(f"    光伏拟合: MAE={fit.pv_mae:.4f} kW, RMSE={fit.pv_rmse:.4f} kW, R^2={fit.pv_r2:.6f}")
print(
    "    光伏曲线参数: "
    f"A={fit.pv_amplitude_kw:.4f} kW, 日出={fit.sunrise_hour:.2f} h, "
    f"日落={fit.sunset_hour:.2f} h, γ={fit.pv_shape:.3f}"
)
print(f"    傅里叶场景内购电费: {fourier.total_cost_yuan:.4f} 元")
print(f"    傅里叶策略在真实数据下费用: {fourier_actual_cost:.4f} 元")
print(
    f"    傅里叶策略相对原始最优解成本损失: "
    f"{fourier_gap:.4f} 元 ({fourier_gap_rate:.6f}%) "
)
print(f"    小波改进场景内购电费: {wavelet.total_cost_yuan:.4f} 元")
print(f"    小波改进策略在真实数据下费用: {wavelet_actual_cost:.4f} 元")
print(
    f"    小波改进策略相对原始最优解成本损失: "
    f"{wavelet_gap:.4f} 元 ({wavelet_gap_rate:.6f}%) "
)

def parse_args() -> argparse.Namespace:
    parser = argparse.ArgumentParser(description="C 题问题一线性规划与周期拟合对照")
    parser.add_argument("--input", type=Path, default=DEFAULT_INPUT)
    parser.add_argument("--template", type=Path, default=DEFAULT_TEMPLATE)
    parser.add_argument("--output", type=Path, default=DEFAULT_OUTPUT)
    parser.add_argument("--comparison", type=Path, default=DEFAULT_COMPARISON)
    parser.add_argument(
        "--result-scenario",
        choices=("actual", "fourier", "wavelet"),
        default="wavelet",
        help=(
            "写入结果工作簿的数据场景: actual=原始数据, "
            "fourier=傅里叶负荷, wavelet=傅里叶-小波负荷(默认)"
        ),
    )
    return parser.parse_args()

def main() -> None:
    args = parse_args()
    cfg = Config()
    data = read_input(args.input)
    order, labels = template_chronological_order(args.template)
    index = np.asarray(order, dtype=int)
    price = data.price[index]
    load_actual = data.load_kw[index]
    pv_actual = data.pv_kw[index]

    fit = fit_profiles(data, order, cfg)
    actual_solution = solve_linear_program(price, load_actual, pv_actual, cfg)
    fourier_solution = solve_linear_program(
        price, fit.load_fourier_kw, fit.pv_kw, cfg
    )
    wavelet_solution = solve_linear_program(price, fit.load_kw, fit.pv_kw, cfg)
    fourier_grid_actual, _, fourier_actual_cost = evaluate_fitted_dispatch(
        fourier_solution, price, load_actual, pv_actual, cfg
    )
    wavelet_grid_actual, _, wavelet_actual_cost = evaluate_fitted_dispatch(
        wavelet_solution, price, load_actual, pv_actual, cfg
    )

    scenarios = {
        "actual": actual_solution,

```

```

        "fourier": fourier_solution,
        "wavelet": wavelet_solution,
    }
    result_solution = scenarios[args.result_scenario]
    write_result(args.template, args.output, order, result_solution)
    write_comparison(
        args.comparison,
        price,
        load_actual,
        pv_actual,
        fit,
        actual_solution,
        fourier_solution,
        wavelet_solution,
        fourier_grid_actual,
        wavelet_grid_actual,
    )
    print_summary(
        labels,
        order,
        fit,
        actual_solution,
        fourier_solution,
        wavelet_solution,
        fourier_actual_cost,
        wavelet_actual_cost,
    )
    scenario_name = {
        "actual": "原始数据",
        "fourier": "傅里叶负荷+截断正弦光伏",
        "wavelet": "傅里叶-小波负荷+截断正弦光伏",
    }[args.result_scenario]
    print(f"\n 正式结果 ({scenario_name}): {args.output.resolve()}")
    print(f"拟合对照: {args.comparison.resolve()}")

if __name__ == "__main__":
    main()

```

C.2 问题二程序 two.py

程序说明：构造严格前向的年度负荷与光伏日前预测，利用冻结的贝叶斯参数轨迹和 0.8 分位风险裕度生成净负荷需求，并采用 48 小时滚动线性规划消除单日终端效应。程序按实际数据回放紧急购电并写入 result2.xlsx。

```

# 每天 0:00 仅使用此前已观测的数据预测当天负荷和光伏功率：
# 1. 年度谐波—星期效应岭回归与历史轮廓融合预测负荷；
# 2. 二阶傅里叶与 Haar 小波重构日内负荷残差；
# 3. 截断幂正弦函数拟合光伏功率；
# 4. 历史净负荷预测误差的 80% 分位数作为安全裕度；
# 5. 采用 48 小时滚动线性规划，执行前 24 小时并连续传递 SOC；
# 6. 用真实数据回放，供电不足部分按当时电价 5 倍紧急购电。
# 7. 采用“训练训练测试”循环划分，并用严格前向贝叶斯优化和早停选择负荷参数。

# 默认输入：C 题/附件/csv/附件 1.csv 与附件 2 的两个 CSV
# 默认模板：C 题/附件/附件 5/result2.xlsx
# 默认输出：C 题/result2.xlsx
# 默认汇总：C 题/问题二逐日汇总.csv
# 默认检验：C 题/问题二_训练验证测试检验.csv

# 依赖：numpy、scipy、openpyxl
# 正式求解：python two.py
# 贝叶斯寻优：python two.py --bayes-optimize
# 自定义寻优：python two.py --bayes-optimize --bayes-trials 50 --bayes-patience 12 --bayes-seed 2026

from __future__ import annotations

import argparse
import csv
import json
import math
import os
import shutil
from copy import copy
from dataclasses import asdict, dataclass, replace
from datetime import date, datetime, timedelta

```

```

from pathlib import Path

import numpy as np
from openpyxl import load_workbook

ROOT = Path(__file__).resolve().parent
DEFAULT_BASELINE = ROOT / "C 题" / "附件" / "csv" / "附件 1.csv"
DEFAULT_LOAD = ROOT / "C 题" / "附件" / "csv" / "附件 2_小区负载.csv"
DEFAULT_PV = ROOT / "C 题" / "附件" / "csv" / "附件 2_光伏发电实际功率.csv"
DEFAULT_TEMPLATE = ROOT / "C 题" / "附件" / "附件 5" / "result2.xlsx"
DEFAULT_OUTPUT = ROOT / "C 题" / "result2.xlsx"
DEFAULT_SUMMARY = ROOT / "C 题" / "问题二逐日汇总.csv"
DEFAULT_SPLIT_REPORT = ROOT / "C 题" / "问题二_训练验证测试检验.csv"
DEFAULT_BAYES_REPORT = ROOT / "C 题" / "贝叶斯优化_负荷参数与评价_严格前向.json"

@dataclass(frozen=True)
class Config:
    dt_hours: float = 1.0 / 6.0
    soc_min_kwh: float = 1200.0
    soc_max_kwh: float = 10800.0
    initial_soc_kwh: float = 6000.0
    max_charge_power_kw: float = 5000.0
    max_discharge_power_kw: float = 5000.0
    charge_efficiency: float = 0.90
    discharge_efficiency: float = 0.90
    history_days: int = 28
    risk_days: int = 28
    minimum_risk_days: int = 7
    half_life_days: float = 7.0
    same_weekday_multiplier: float = 2.0
    wavelet_level: int = 3
    annual_harmonics: int = 3
    load_ridge_lambda: float = 3.0
    dynamic_load_blend: float = 0.65
    use_bayesian_load_schedule: bool = True
    risk_quantile: float = 0.80
    emergency_price_multiplier: float = 5.0
    throughput_tie_breaker: float = 1.0e-7
    emergency_tolerance_kwh: float = 1.0e-5

    @property
    def max_charge_kwh(self) -> float:
        return self.max_charge_power_kw * self.dt_hours

    @property
    def max_discharge_kwh(self) -> float:
        return self.max_discharge_power_kw * self.dt_hours

BAYESIAN_LOAD_PARAMETER_SCHEDULE: tuple[
    tuple[date, dict[str, float | int]], ...
] = (
    (
        date(2025, 2, 16),
        {
            "history_days": 38,
            "half_life_days": 25.29917197614039,
            "same_weekday_multiplier": 2.241902727000516,
            "wavelet_level": 2,
            "annual_harmonics": 3,
            "load_ridge_lambda": 0.003840001534395503,
            "dynamic_load_blend": 0.6206534746761987,
        },
    ),
    (
        date(2025, 3, 12),
        {
            "history_days": 27,
            "half_life_days": 27.866925493061643,
            "same_weekday_multiplier": 2.0302869528791616,
            "wavelet_level": 2,
            "annual_harmonics": 3,
            "load_ridge_lambda": 0.002040174751235714,
            "dynamic_load_blend": 0.6470383902370724,
        },
    ),
    (
        date(2025, 4, 23),
        {

```

```

        "history_days": 35,
        "half_life_days": 22.86236550087216,
        "same_weekday_multiplier": 1.7054409173409268,
        "wavelet_level": 2,
        "annual_harmonics": 3,
        "load_ridge_lambda": 0.0010398473604926101,
        "dynamic_load_blend": 0.8157826442179166,
    },
),
(
    date(2025, 5, 17),
    {
        "history_days": 23,
        "half_life_days": 22.16005337031691,
        "same_weekday_multiplier": 3.4204172241648596,
        "wavelet_level": 2,
        "annual_harmonics": 3,
        "load_ridge_lambda": 0.00010683581757667908,
        "dynamic_load_blend": 0.726727237291571,
    },
),
(
    date(2025, 8, 3),
    {
        "history_days": 35,
        "half_life_days": 22.86236550087216,
        "same_weekday_multiplier": 1.7054409173409268,
        "wavelet_level": 2,
        "annual_harmonics": 3,
        "load_ridge_lambda": 0.0010398473604926101,
        "dynamic_load_blend": 0.8157826442179166,
    },
),
)

def bayesian_load_config(cfg: Config, issue_date: date) -> Config:
    if not cfg.use_bayesian_load_schedule:
        return cfg
    overrides: dict[str, float | int] | None = None
    for effective_date, candidate in BAYESIAN_LOAD_PARAMETER_SCHEDULE:
        if issue_date < effective_date:
            break
        overrides = candidate
    return cfg if overrides is None else replace(cfg, **overrides)

@dataclass(frozen=True)
class DatasetSplit:
    train_end: date = date(2025, 7, 31)
    validation_end: date = date(2025, 9, 30)

    def training_cutoff(self, dates: list[date]) -> int:
        if self.validation_end <= self.train_end:
            raise ValueError("验证集结束日期必须晚于训练集结束日期。")
        cutoff = sum(day <= self.train_end for day in dates)
        if cutoff <= 0 or cutoff >= len(dates):
            raise ValueError("训练集必须非空，且不能覆盖全部数据。")
        return cutoff

    def label(self, day: date) -> str:
        if day <= self.train_end:
            return "训练集"
        if day <= self.validation_end:
            return "验证集"
        return "测试集"

    def cycle_labels(self, dates: list[date]) -> dict[str, list[int]]:
        groups = {"训练集": [], "验证集": [], "测试集": []}
        for index, _ in enumerate(dates):
            slot = index % 6
            label = "验证集" if slot == 3 else "测试集" if slot == 5 else "训练集"
            groups[label].append(index)
        return groups

@dataclass
class BaselineData:
    price: np.ndarray
    load_kw: np.ndarray

```

```

pv_kw: np.ndarray

@dataclass
class HistoricalData:
    dates: list[date]
    load_kw: np.ndarray
    pv_kw: np.ndarray

@dataclass
class Forecast:
    load_kw: np.ndarray
    pv_kw: np.ndarray
    load_fourier_kw: np.ndarray
    pv_amplitude_kw: float
    sunrise_hour: float
    sunset_hour: float
    pv_shape: float

@dataclass
class LPSolution:
    grid_kwh: np.ndarray
    charge_kwh: np.ndarray
    discharge_kwh: np.ndarray
    curtailment_kwh: np.ndarray
    soc_kwh: np.ndarray
    objective_yuan: float

@dataclass
class DailyResult:
    day: date
    forecast_load_kw: np.ndarray
    forecast_pv_kw: np.ndarray
    reserve_kwh: np.ndarray
    grid_kwh: np.ndarray
    charge_kwh: np.ndarray
    discharge_kwh: np.ndarray
    planned_curtailment_kwh: np.ndarray
    actual_curtailment_kwh: np.ndarray
    emergency_kwh: np.ndarray
    soc_start_kwh: float
    soc_end_kwh: float
    plan_cost_yuan: float
    emergency_cost_yuan: float

    @property
    def total_cost_yuan(self) -> float:
        return self.plan_cost_yuan + self.emergency_cost_yuan

def _float(value: object, context: str) -> float:
    try:
        number = float(value)
    except (TypeError, ValueError) as exc:
        raise ValueError(f"{context}不是数值: {value!r}") from exc
    if not math.isfinite(number):
        raise ValueError(f"{context}不是有限数值: {value!r}")
    return number

def _parse_source_minute(value: str) -> int:
    text = value.strip().replace("+1", "")
    parts = text.split(":")
    if len(parts) < 2:
        raise ValueError(f"无法解析时间: {value!r}")
    hour, minute = int(parts[0]) % 24, int(parts[1])
    return hour * 60 + minute

def read_baseline(path: Path) -> tuple[BaselineData, list[int]]:
    if not path.exists():
        raise FileNotFoundError(f"找不到附件1 CSV: {path}")
    with path.open("r", encoding="utf-8-sig", newline="") as file:
        rows = list(csv.reader(file))
    if len(rows) != 145 or len(rows[0]) < 4:
        raise ValueError("附件1 应包含1行表头和144行数据。")

```

```

times: list[str] = []
price: list[float] = []
load: list[float] = []
pv: list[float] = []
for row_number, row in enumerate(rows[1:], start=2):
    times.append(row[0].strip())
    price.append(_float(row[1], f"附件 1 第 {row_number} 行电价"))
    load.append(_float(row[2], f"附件 1 第 {row_number} 行负荷"))
    pv.append(_float(row[3], f"附件 1 第 {row_number} 行光伏"))

minutes = [_parse_source_minute(value) for value in times]
if sorted(minutes) != list(range(0, 1440, 10)):
    raise ValueError("附件 1 没有完整覆盖一天的 144 个 10 分钟时刻。")
chronological_order = sorted(range(144), key=minutes.__getitem__)
order = np.asarray(chronological_order, dtype=int)
return (
    BaselineData(
        np.asarray(price, dtype=float)[order],
        np.asarray(load, dtype=float)[order],
        np.asarray(pv, dtype=float)[order],
    ),
    chronological_order,
)

def read_daily_matrix(path: Path, chronological_order: list[int]) -> tuple[list[date], np.ndarray]:
    if not path.exists():
        raise FileNotFoundError(f"找不到附件 2 CSV: {path}")
    with path.open("r", encoding="utf-8-sig", newline="") as file:
        reader = csv.reader(file)
        header = next(reader, None)
        rows = list(reader)
    if header is None or len(header) != 145:
        raise ValueError(f"{path.name} 应包含日期列和 144 个时刻列。")
    if len(rows) != 365:
        raise ValueError(f"{path.name} 应包含 2025 年的 365 天数据。")

    dates: list[date] = []
    values: list[list[float]] = []
    for row_number, row in enumerate(rows, start=2):
        if len(row) != 145:
            raise ValueError(f"{path.name} 第 {row_number} 行列数不是 145。")
        try:
            current_date = datetime.fromisoformat(row[0].strip()).date()
        except ValueError as exc:
            raise ValueError(f"{path.name} 第 {row_number} 行日期无法解析: {row[0]!r}") from exc
        dates.append(current_date)
        values.append(
            [_float(row[i + 1], f"{path.name} 第 {row_number} 行第 {i + 2} 列") for i in chronological_order]
        )

    matrix = np.asarray(values, dtype=float)
    if np.any(matrix < 0):
        raise ValueError(f"{path.name} 中存在负功率。")
    return dates, matrix

def read_historical_data(
    load_path: Path, pv_path: Path, chronological_order: list[int]
) -> HistoricalData:
    load_dates, load = read_daily_matrix(load_path, chronological_order)
    pv_dates, pv = read_daily_matrix(pv_path, chronological_order)
    if load_dates != pv_dates:
        raise ValueError("附件 2 的负荷日期和光伏日期不一致。")
    expected = [date(2025, 1, 1) + timedelta(days=i) for i in range(365)]
    if load_dates != expected:
        raise ValueError("附件 2 日期应连续覆盖 2025-01-01 至 2025-12-31。")
    return HistoricalData(load_dates, load, pv)

def _fourier_design(hours: np.ndarray) -> np.ndarray:
    omega = 2.0 * np.pi / 24.0
    return np.column_stack(
        [
            np.ones_like(hours),
            np.cos(omega * hours),
            np.sin(omega * hours),
            np.cos(2.0 * omega * hours),
            np.sin(2.0 * omega * hours),
        ]
    )

```

```

)

def fit_load_fourier(hours: np.ndarray, load_kw: np.ndarray) -> np.ndarray:
    design = _fourier_design(hours)
    coefficients, *_ = np.linalg.lstsq(design, load_kw, rcond=None)
    return np.maximum(0.0, design @ coefficients)

def _haar_dwt(signal: np.ndarray, level: int) -> tuple[np.ndarray, list[np.ndarray]]:
    approximation = np.asarray(signal, dtype=float).copy()
    details: list[np.ndarray] = []
    scale = math.sqrt(2.0)
    for _ in range(level):
        if approximation.size % 2:
            raise ValueError("Haar 小波每层输入长度必须为偶数。")
        even = approximation[0::2]
        odd = approximation[1::2]
        details.append((even - odd) / scale)
        approximation = (even + odd) / scale
    return approximation, details

def _haar_idwt(approximation: np.ndarray, details: list[np.ndarray]) -> np.ndarray:
    reconstructed = np.asarray(approximation, dtype=float).copy()
    scale = math.sqrt(2.0)
    for detail in reversed(details):
        restored = np.empty(reconstructed.size * 2, dtype=float)
        restored[0::2] = (reconstructed + detail) / scale
        restored[1::2] = (reconstructed - detail) / scale
        reconstructed = restored
    return reconstructed

def wavelet_correct_residual(
    profile_kw: np.ndarray, fourier_kw: np.ndarray, level: int
) -> np.ndarray:
    residual = np.asarray(profile_kw) - np.asarray(fourier_kw)
    if residual.size % (2**level):
        raise ValueError(f"样本数必须能被 2^{level} 整除。")
    approximation, details = _haar_dwt(residual, level)
    thresholded: list[np.ndarray] = []
    for detail in details:
        sigma = float(np.median(np.abs(detail - np.median(detail)))) / 0.6745
        threshold = sigma * math.sqrt(2.0 * math.log(residual.size))
        thresholded.append(
            np.sign(detail) * np.maximum(np.abs(detail) - threshold, 0.0)
        )
    return _haar_idwt(approximation, thresholded)

def fit_pv_truncated_sine(
    hours: np.ndarray, pv_kw: np.ndarray
) -> tuple[np.ndarray, float, float, float, float]:
    peak = float(np.max(pv_kw))
    if peak <= 1.0e-8:
        return np.zeros_like(pv_kw), 0.0, 0.0, 0.0, 1.0

    positive = np.flatnonzero(pv_kw > max(1.0, peak * 0.002))
    if positive.size == 0:
        return np.zeros_like(pv_kw), 0.0, 0.0, 0.0, 1.0
    first_hour = float(hours[positive[0]])
    last_hour = float(hours[positive[-1]])
    sunrise_grid = np.arange(max(0.0, first_hour - 0.5), first_hour + 1.0e-9, 1.0 / 6.0)
    sunset_grid = np.arange(last_hour, min(24.0, last_hour + 0.5) + 1.0e-9, 1.0 / 6.0)
    shape_grid = np.linspace(0.6, 2.8, 45)

    best_sse = math.inf
    best_curve = np.zeros_like(pv_kw)
    best_parameters = (0.0, first_hour, last_hour, 1.0)
    for sunrise in sunrise_grid:
        for sunset in sunset_grid:
            if sunset - sunrise < 5.0:
                continue
            phase = np.pi * (hours - sunrise) / (sunset - sunrise)
            sine = np.zeros_like(hours)
            daylight = (hours >= sunrise) & (hours <= sunset)
            sine[daylight] = np.maximum(0.0, np.sin(phase[daylight]))
            basis = sine[None, :] ** shape_grid[:, None]
            denominator = np.sum(basis * basis, axis=1)

```

```

        numerator = basis @ pv_kw
        amplitudes = np.divide(
            numerator,
            denominator,
            out=np.zeros_like(numerator),
            where=denominator > 0,
        )
        amplitudes = np.maximum(0.0, amplitudes)
        curves = amplitudes[:, None] * basis
        errors = np.sum((curves - pv_kw[None, :]) ** 2, axis=1)
        index = int(np.argmin(errors))
        if float(errors[index]) < best_sse:
            best_sse = float(errors[index])
            best_curve = curves[index].copy()
            best_parameters = (
                float(amplitudes[index]),
                float(sunrise),
                float(sunset),
                float(shape_grid[index]),
            )
    )
    return best_curve, *best_parameters

def _weighted_history_profile(
    matrix: np.ndarray,
    historical_dates: list[date],
    cutoff: int,
    target_date: date,
    baseline: np.ndarray,
    cfg: Config,
    allowed_indices: set[int] | None = None,
) -> np.ndarray:
    if cutoff <= 0:
        return baseline.copy()
    first = max(0, cutoff - cfg.history_days)
    indices = np.arange(first, cutoff, dtype=int)
    if allowed_indices is not None:
        indices = np.asarray([i for i in indices if i in allowed_indices], dtype=int)
    if indices.size == 0:
        return baseline.copy()
    ages = cutoff - 1 - indices
    weights = np.exp(-math.log(2.0) * ages / cfg.half_life_days)
    same_weekday = np.asarray(
        [historical_dates[i].weekday() == target_date.weekday() for i in indices],
        dtype=float,
    )
    weights *= 1.0 + cfg.same_weekday_multiplier * same_weekday
    profile = np.average(matrix[indices], axis=0, weights=weights)

    # 前 7 天历史样本较少，逐步降低附件 1 先验曲线的权重。
    prior_share = max(0.0, (7.0 - cutoff) / 7.0)
    return (1.0 - prior_share) * profile + prior_share * baseline

def _calendar_features(day: date, harmonics: int) -> np.ndarray:
    day_of_year = day.timetuple().tm_yday - 1
    columns = [1.0]
    for harmonic in range(1, harmonics + 1):
        angle = 2.0 * math.pi * harmonic * day_of_year / 365.0
        columns.extend([math.cos(angle), math.sin(angle)])
    weekday = day.weekday()
    columns.extend([1.0 if weekday == value else 0.0 for value in range(1, 7)])
    return np.asarray(columns, dtype=float)

def fit_dynamic_load_profile(
    historical: HistoricalData,
    cutoff: int,
    target_date: date,
    baseline: np.ndarray,
    cfg: Config,
    allowed_indices: set[int] | None,
) -> np.ndarray:
    indices = [
        index for index in range(cutoff)
        if allowed_indices is None or index in allowed_indices
    ]
    feature_count = 1 + 2 * cfg.annual_harmonics + 6
    if len(indices) < max(14, feature_count + 1):
        return baseline.copy()

```

```

design = np.vstack(
    [_calendar_features(historical.dates[index], cfg.annual_harmonics) for index in indices]
)
target = historical.load_kw[indices]
penalty = np.eye(feature_count, dtype=float) * cfg.load_ridge_lambda
penalty[0, 0] = 0.0
coefficients = np.linalg.solve(
    design.T @ design + penalty,
    design.T @ target,
)
prediction = _calendar_features(target_date, cfg.annual_harmonics) @ coefficients
return np.maximum(0.0, prediction)

def make_forecast(
    historical: HistoricalData,
    baseline: BaselineData,
    cutoff: int,
    target_date: date,
    hours: np.ndarray,
    cfg: Config,
    allowed_indices: set[int] | None = None,
    pv_cfg: Config | None = None,
) -> Forecast:
    pv_settings = cfg if pv_cfg is None else pv_cfg
    load_profile = _weighted_history_profile(
        historical.load_kw,
        historical.dates,
        cutoff,
        target_date,
        baseline.load_kw,
        cfg,
        allowed_indices,
    )
    dynamic_load = fit_dynamic_load_profile(
        historical,
        cutoff,
        target_date,
        baseline.load_kw,
        cfg,
        allowed_indices,
    )
    blend = float(np.clip(cfg.dynamic_load_blend, 0.0, 1.0))
    load_profile = (1.0 - blend) * load_profile + blend * dynamic_load
    pv_profile = _weighted_history_profile(
        historical.pv_kw,
        historical.dates,
        cutoff,
        target_date,
        baseline.pv_kw,
        pv_settings,
        allowed_indices,
    )

    load_fourier = fit_load_fourier(hours, load_profile)
    wavelet_residual = wavelet_correct_residual(
        load_profile, load_fourier, cfg.wavelet_level
    )
    load_forecast = np.maximum(0.0, load_fourier + wavelet_residual)
    pv_forecast, amplitude, sunrise, sunset, shape = fit_pv_truncated_sine(
        hours, pv_profile
    )
    return Forecast(
        load_forecast,
        pv_forecast,
        load_fourier,
        amplitude,
        sunrise,
        sunset,
        shape,
    )

def risk_reserve(error_history: list[np.ndarray], cfg: Config) -> np.ndarray:
    if len(error_history) < cfg.minimum_risk_days:
        return np.zeros(144, dtype=float)
    errors = np.asarray(error_history[-cfg.risk_days:], dtype=float)
    reserve = np.quantile(errors, cfg.risk_quantile, axis=0)
    return np.maximum(0.0, reserve)

```

```

def solve_rolling_lp(
    price: np.ndarray,
    load_kw: np.ndarray,
    pv_kw: np.ndarray,
    initial_soc_kwh: float,
    cfg: Config,
) -> LPSolution:
    try:
        from scipy.optimize import linprog
        from scipy.sparse import coo_matrix
    except ModuleNotFoundError as exc:
        raise ModuleNotFoundError("缺少 scipy, 请先执行: python -m pip install scipy") from exc

    n = len(price)
    if n not in (144, 288) or len(load_kw) != n or len(pv_kw) != n:
        raise ValueError("滚动优化数组长度必须为 144 或 288 且彼此一致。")
    load_energy = load_kw * cfg.dt_hours
    pv_energy = pv_kw * cfg.dt_hours

    g0, c0, d0, w0, e0 = 0, n, 2 * n, 3 * n, 4 * n
    variable_count = 5 * n + 1
    objective = np.zeros(variable_count, dtype=float)
    objective[g0 : g0 + n] = price
    objective[c0 : c0 + n] = cfg.throughput_tie_breaker
    objective[d0 : d0 + n] = cfg.throughput_tie_breaker

    rows: list[int] = []
    columns: list[int] = []
    coefficients: list[float] = []
    b_eq = np.zeros(2 * n, dtype=float)
    for t in range(n):
        # g + PV + d = load + c + w
        rows.extend([t, t, t, t])
        columns.extend([g0 + t, c0 + t, d0 + t, w0 + t])
        coefficients.extend([1.0, -1.0, 1.0, -1.0])
        b_eq[t] = load_energy[t] - pv_energy[t]

        # e[t+1] = e[t] + eta_c*c - d/eta_d
        row = n + t
        rows.extend([row, row, row, row])
        columns.extend([c0 + t, d0 + t, e0 + t, e0 + t + 1])
        coefficients.extend(
            [-cfg.charge_efficiency, 1.0 / cfg.discharge_efficiency, -1.0, 1.0]
        )

    a_eq = coo_matrix(
        (coefficients, (rows, columns)), shape=(2 * n, variable_count)
    ).tocsr()

    bounds: list[tuple[float | None, float | None]] = []
    bounds.extend([(0.0, None)] * n)
    bounds.extend([(0.0, cfg.max_charge_kwh)] * n)
    bounds.extend([(0.0, cfg.max_discharge_kwh)] * n)
    bounds.extend([(0.0, None)] * n)
    bounds.extend([(cfg.soc_min_kwh, cfg.soc_max_kwh)] * (n + 1))
    bounds[e0] = (initial_soc_kwh, initial_soc_kwh)
    # 48 小时未恢复当前 SOC, 避免滚动窗口末端无成本放空储能。
    bounds[e0 + n] = (initial_soc_kwh, initial_soc_kwh)

    result = linprog(
        objective,
        A_eq=a_eq,
        b_eq=b_eq,
        bounds=bounds,
        method="highs",
        options={"presolve": True},
    )
    if not result.success:
        raise RuntimeError(f"线性规划失败: status={result.status}, {result.message}")

    x = np.asarray(result.x, dtype=float)
    solution = LPSolution(
        x[g0 : g0 + n],
        x[c0 : c0 + n],
        x[d0 : d0 + n],
        x[w0 : w0 + n],
        x[e0 : e0 + n + 1],
        float(np.dot(price, x[g0 : g0 + n])),
    )

```

```

    validate_lp(solution, load_energy, pv_energy, initial_soc_kwh, cfg)
    return solution

def validate_lp(
    solution: LPSolution,
    load_energy: np.ndarray,
    pv_energy: np.ndarray,
    initial_soc_kwh: float,
    cfg: Config,
    tolerance: float = 1.0e-5,
) -> None:
    balance = (
        solution.grid_kwh
        + pv_energy
        + solution.discharge_kwh
        - load_energy
        - solution.charge_kwh
        - solution.curtailment_kwh
    )
    next_soc = (
        solution.soc_kwh[:-1]
        + cfg.charge_efficiency * solution.charge_kwh
        - solution.discharge_kwh / cfg.discharge_efficiency
    )
    checks = {
        "电量平衡": float(np.max(np.abs(balance))),
        "SOC 递推": float(np.max(np.abs(solution.soc_kwh[1:] - next_soc))),
        "初始 SOC": abs(float(solution.soc_kwh[0]) - initial_soc_kwh),
        "滚动窗口终止 SOC": abs(float(solution.soc_kwh[-1]) - initial_soc_kwh),
    }
    failed = {name: value for name, value in checks.items() if value > tolerance}
    if failed:
        raise RuntimeError(f"线性规划结果校验失败: {failed}")
    if np.min(solution.soc_kwh) < cfg.soc_min_kwh - tolerance:
        raise RuntimeError("SOC 低于安全下限。")
    if np.max(solution.soc_kwh) > cfg.soc_max_kwh + tolerance:
        raise RuntimeError("SOC 高于安全上限。")
    if np.max(np.minimum(solution.charge_kwh, solution.discharge_kwh)) > 1.0e-4:
        raise RuntimeError("线性规划结果出现同一时段同时充放电。")

def replay_actual_day(
    actual_load_kw: np.ndarray,
    actual_pv_kw: np.ndarray,
    grid_kwh: np.ndarray,
    charge_kwh: np.ndarray,
    discharge_kwh: np.ndarray,
    cfg: Config,
) -> tuple[np.ndarray, np.ndarray]:
    deficit = (
        actual_load_kw * cfg.dt_hours
        + charge_kwh
        - actual_pv_kw * cfg.dt_hours
        - discharge_kwh
        - grid_kwh
    )
    emergency = np.maximum(0.0, deficit)
    curtailment = np.maximum(0.0, -deficit)
    balance = (
        grid_kwh
        + actual_pv_kw * cfg.dt_hours
        + discharge_kwh
        + emergency
        - actual_load_kw * cfg.dt_hours
        - charge_kwh
        - curtailment
    )
    if float(np.max(np.abs(balance))) > 1.0e-5:
        raise RuntimeError("真实数据回放电量不平衡。")
    return emergency, curtailment

def simulate_year(
    baseline: BaselineData,
    historical: HistoricalData,
    cfg: Config,
    training_cutoff: int | None = None,
    training_indices: set[int] | None = None,
) -> list[DailyResult]:

```

```

hours = np.arange(144, dtype=float) * cfg.dt_hours
price_48 = np.tile(baseline.price, 2)
error_history: list[np.ndarray] = []
results: list[DailyResult] = []
soc = cfg.initial_soc_kwh

for day_index, current_date in enumerate(historical.dates):
    # 训练期采用因果滚动拟合；进入验证/测试期后冻结训练样本边界。
    model_cutoff = (
        day_index
        if training_cutoff is None
        else min(day_index, training_cutoff)
    )
    forecast_cfg = bayesian_load_config(cfg, current_date)
    current_forecast = make_forecast(
        historical, baseline, model_cutoff, current_date, hours, forecast_cfg,
        allowed_indices=training_indices,
        pv_cfg=cfg,
    )
    next_forecast = make_forecast(
        historical,
        baseline,
        model_cutoff,
        current_date + timedelta(days=1),
        hours,
        forecast_cfg,
        allowed_indices=training_indices,
        pv_cfg=cfg,
    )
    reserve = risk_reserve(error_history, cfg)
    risk_load_current = current_forecast.load_kw + reserve / cfg.dt_hours
    risk_load_next = next_forecast.load_kw + reserve / cfg.dt_hours

    solution = solve_rolling_lp(
        price_48,
        np.concatenate([risk_load_current, risk_load_next]),
        np.concatenate([current_forecast.pv_kw, next_forecast.pv_kw]),
        soc,
        cfg,
    )
    grid = solution.grid_kwh[:144]
    charge = solution.charge_kwh[:144]
    discharge = solution.discharge_kwh[:144]
    planned_curtailment = solution.curtailment_kwh[:144]
    emergency, actual_curtailment = replay_actual_day(
        historical.load_kw[day_index],
        historical.pv_kw[day_index],
        grid,
        charge,
        discharge,
        cfg,
    )

    soc_start = soc
    soc = float(solution.soc_kwh[144])
    plan_cost = float(np.dot(baseline.price, grid))
    emergency_cost = float(
        np.dot(
            cfg.emergency_price_multiplier * baseline.price,
            emergency,
        )
    )

    results.append(
        DailyResult(
            current_date,
            current_forecast.load_kw,
            current_forecast.pv_kw,
            reserve,
            grid,
            charge,
            discharge,
            planned_curtailment,
            actual_curtailment,
            emergency,
            soc_start,
            soc,
            plan_cost,
            emergency_cost,
        )
    )

```

```

        actual_net = (
            historical.load_kw[day_index] - historical.pv_kw[day_index]
        ) * cfg.dt_hours
        forecast_net = (
            current_forecast.load_kw - current_forecast.pv_kw
        ) * cfg.dt_hours
        if training_indices is None or day_index in training_indices:
            error_history.append(actual_net - forecast_net)

        if (day_index + 1) % 30 == 0 or day_index == len(historical.dates) - 1:
            print(
                f"已完成 {current_date.isoformat()}: "
                f"SOC={soc:.2f} kWh, 紧急购电={np.sum(emergency):.2f} kWh"
            )
    return results

def _to_template_order(values: np.ndarray, chronological_order: list[int]) -> np.ndarray:
    mapped = np.empty_like(values)
    mapped[np.asarray(chronological_order, dtype=int)] = values
    return mapped

def _copy_cell_style(source, target) -> None:
    target._style = copy(source._style)
    target.number_format = source.number_format
    target.font = copy(source.font)
    target.fill = copy(source.fill)
    target.border = copy(source.border)
    target.alignment = copy(source.alignment)
    target.protection = copy(source.protection)

def _interval_label(start_index: int, end_index_exclusive: int) -> str:
    start_minutes = start_index * 10
    end_minutes = end_index_exclusive * 10

    def format_minutes(minutes: int) -> str:
        if minutes == 1440:
            return "24:00"
        return f"{minutes // 60}:{minutes % 60:02d}"

    return f"{format_minutes(start_minutes)}-{format_minutes(end_minutes)}"

def group_emergency(values: np.ndarray, tolerance: float) -> list[tuple[str, float]]:
    positive = values > tolerance
    groups: list[tuple[str, float]] = []
    index = 0
    while index < len(values):
        if not positive[index]:
            index += 1
            continue
        start = index
        while index + 1 < len(values) and positive[index + 1]:
            index += 1
        stop = index + 1
        groups.append((_interval_label(start, stop), float(np.sum(values[start:stop]))))
        index += 1
    return groups

def write_result_workbook(
    template: Path,
    output: Path,
    results: list[DailyResult],
    price: np.ndarray,
    chronological_order: list[int],
    cfg: Config,
) -> None:
    if not template.exists():
        raise FileNotFoundError(f"找不到 result2 模板: {template}")
    if template.resolve() == output.resolve():
        raise ValueError("输出文件不能覆盖官方模板。")
    output.parent.mkdir(parents=True, exist_ok=True)
    temporary = output.with_name(f"{output.stem}.building{output.suffix}")
    if temporary.exists():
        temporary.unlink()
    shutil.copy2(template, temporary)

```

```

try:
    workbook = load_workbook(temporary)
    plan_sheet = workbook["计划购电量"]
    storage_sheet = workbook["充放电量"]
    emergency_sheet = workbook["紧急购电量"]

    output_results = [item for item in results if item.day >= date(2025, 2, 1)]
    if len(output_results) != 334:
        raise RuntimeError("结果应包含 2025-02-01 至 2025-12-31 共 334 天。")

    template_dates = [
        plan_sheet.cell(row, 1).value.date()
        if isinstance(plan_sheet.cell(row, 1).value, datetime)
        else plan_sheet.cell(row, 1).value
        for row in range(2, plan_sheet.max_row + 1)
    ]
    expected_dates = [item.day for item in output_results]
    if template_dates != expected_dates:
        raise ValueError("result2 模板的计划购电日期与计算日期不一致。")

    for row, item in enumerate(output_results, start=2):
        grid_template = _to_template_order(item.grid_kwh, chronological_order)
        for column, value in enumerate(grid_template, start=2):
            plan_sheet.cell(row, column, round(float(value), 4))
        plan_sheet.cell(row, 146, round(float(np.sum(item.grid_kwh)), 4))
        plan_sheet.cell(row, 147, round(item.plan_cost_yuan, 4))

    storage_styles = [
        copy(storage_sheet.cell(row, column)) for column in range(1, 7)]
    for row in (2, 3)
    ]
    if storage_sheet.max_row > 1:
        storage_sheet.delete_rows(2, storage_sheet.max_row - 1)
    storage_row = 2
    periods = [
        "0:00-4:00",
        "4:00-8:00",
        "8:00-12:00",
        "12:00-16:00",
        "16:00-20:00",
        "20:00-24:00",
    ]
    for item in output_results:
        for block, period in enumerate(periods):
            style_row = storage_styles[0 if block == 0 else 1]
            for column in range(1, 7):
                _copy_cell_style(style_row[column - 1], storage_sheet.cell(storage_row, column))
            start, stop = block * 24, (block + 1) * 24
            storage_sheet.cell(storage_row, 1, item.day if block == 0 else None)
            storage_sheet.cell(storage_row, 2, period)
            storage_sheet.cell(
                storage_row, 3, round(float(np.sum(item.charge_kwh[start:stop])), 4)
            )
            storage_sheet.cell(
                storage_row,
                4,
                round(float(np.sum(item.discharge_kwh[start:stop])), 4),
            )
            if block == 0:
                storage_sheet.cell(storage_row, 5, "0:00")
                storage_sheet.cell(storage_row, 6, round(item.soc_start_kwh, 4))
            elif block == 1:
                storage_sheet.cell(storage_row, 5, "24:00")
                storage_sheet.cell(storage_row, 6, round(item.soc_end_kwh, 4))
            storage_row += 1

    emergency_styles = [
        copy(emergency_sheet.cell(row, column)) for column in range(1, 4)]
    for row in (2, 3)
    ]
    if emergency_sheet.max_row > 1:
        emergency_sheet.delete_rows(2, emergency_sheet.max_row - 1)
    emergency_row = 2
    for item in output_results:
        groups = group_emergency(item.emergency_kwh, cfg.emergency_tolerance_kwh)
        for group_index, (period, amount) in enumerate(groups):
            style_row = emergency_styles[0 if group_index == 0 else 1]
            for column in range(1, 4):
                _copy_cell_style(style_row[column - 1], emergency_sheet.cell(emergency_row, column))

```

```

        emergency_sheet.cell(
            emergency_row, 1, item.day if group_index == 0 else None
        )
        emergency_sheet.cell(emergency_row, 2, period)
        emergency_sheet.cell(emergency_row, 3, round(amount, 4))
        emergency_row += 1
    if emergency_row == 2:
        for column in range(1, 4):
            _copy_cell_style(emergency_styles[0][column - 1], emergency_sheet.cell(2, column))
        emergency_sheet.cell(2, 1, "无紧急购电")
        emergency_sheet.cell(2, 3, 0.0)

    workbook.save(temporary)
    try:
        os.replace(temporary, output)
    except PermissionError as exc:
        raise PermissionError(
            f"无法写入 {output}, 请关闭正在打开该文件的 Excel 窗口后重试。"
        ) from exc
finally:
    if temporary.exists():
        temporary.unlink()

def write_daily_summary(
    path: Path, historical: HistoricalData, results: list[DailyResult]
) -> None:
    path.parent.mkdir(parents=True, exist_ok=True)
    headers = [
        "日期",
        "计划购电量(kWh)",
        "计划购电费(元)",
        "紧急购电量(kWh)",
        "紧急购电费(元)",
        "总购电费(元)",
        "计划弃光量(kWh)",
        "真实弃光量(kWh)",
        "安全裕度合计(kWh)",
        "0:00 储电量(kWh)",
        "24:00 储电量(kWh)",
        "负荷预测 MAE(kW)",
        "光伏预测 MAE(kW)",
    ]
    with path.open("w", encoding="utf-8-sig", newline="") as file:
        writer = csv.writer(file, lineterminator="\n")
        writer.writerow(headers)
        for index, item in enumerate(results):
            writer.writerow(
                [
                    item.day.isoformat(),
                    f"{np.sum(item.grid_kwh):.6f}",
                    f"{item.plan_cost_yuan:.6f}",
                    f"{np.sum(item.emergency_kwh):.6f}",
                    f"{item.emergency_cost_yuan:.6f}",
                    f"{item.total_cost_yuan:.6f}",
                    f"{np.sum(item.planned_curtailment_kwh):.6f}",
                    f"{np.sum(item.actual_curtailment_kwh):.6f}",
                    f"{np.sum(item.reserve_kwh):.6f}",
                    f"{item.soc_start_kwh:.6f}",
                    f"{item.soc_end_kwh:.6f}",
                    f"{np.mean(np.abs(historical.load_kw[index] - item.forecast_load_kw)):.6f}",
                    f"{np.mean(np.abs(historical.pv_kw[index] - item.forecast_pv_kw)):.6f}",
                ]
            )

def _r_squared(actual: np.ndarray, forecast: np.ndarray) -> float:
    residual = float(np.sum((actual - forecast) ** 2))
    total = float(np.sum((actual - np.mean(actual)) ** 2))
    return float("nan") if total <= 0.0 else 1.0 - residual / total

def _split_indices(historical: HistoricalData, split: DatasetSplit) -> dict[str, list[int]]:
    groups = split.cycle_labels(historical.dates)
    if any(not indices for indices in groups.values()):
        raise ValueError(f"训练/验证/测试划分产生空数据集: {groups}")
    return groups

def split_period_metrics(

```

```

historical: HistoricalData,
results: list[DailyResult],
indices: list[int],
) -> dict[str, float | int | str]:
    selected = [results[index] for index in indices]
    actual_load = historical.load_kw[indices]
    actual_pv = historical.pv_kw[indices]
    forecast_load = np.asarray([item.forecast_load_kw for item in selected])
    forecast_pv = np.asarray([item.forecast_pv_kw for item in selected])
    actual_net = actual_load - actual_pv
    forecast_net = forecast_load - forecast_pv
    plan_energy = float(sum(np.sum(item.grid_kwh) for item in selected))
    emergency_energy = float(sum(np.sum(item.emergency_kwh) for item in selected))
    plan_cost = float(sum(item.plan_cost_yuan for item in selected))
    emergency_cost = float(sum(item.emergency_cost_yuan for item in selected))
    purchased = plan_energy + emergency_energy
    return {
        "开始日期": selected[0].day.isoformat(),
        "结束日期": selected[-1].day.isoformat(),
        "天数": len(selected),
        "负荷 MAE (kW)": float(np.mean(np.abs(actual_load - forecast_load))),
        "负荷 RMSE (kW)": float(np.sqrt(np.mean((actual_load - forecast_load) ** 2))),
        "负荷 R2": _r_squared(actual_load, forecast_load),
        "光伏 MAE (kW)": float(np.mean(np.abs(actual_pv - forecast_pv))),
        "光伏 RMSE (kW)": float(np.sqrt(np.mean((actual_pv - forecast_pv) ** 2))),
        "光伏 R2": _r_squared(actual_pv, forecast_pv),
        "净负荷 MAE (kW)": float(np.mean(np.abs(actual_net - forecast_net))),
        "净负荷 RMSE (kW)": float(np.sqrt(np.mean((actual_net - forecast_net) ** 2))),
        "计划购电量(kWh)": plan_energy,
        "紧急购电量(kWh)": emergency_energy,
        "紧急购电占比": 0.0 if purchased <= 0.0 else emergency_energy / purchased,
        "计划购电费(元)": plan_cost,
        "紧急购电费(元)": emergency_cost,
        "总购电费(元)": plan_cost + emergency_cost,
    }

```

```

def write_split_evaluation(
    path: Path,
    historical: HistoricalData,
    results: list[DailyResult],
    split: DatasetSplit,
) -> None:
    groups = _split_indices(historical, split)
    rows = []
    for label in ("训练集", "验证集", "测试集"):
        row = {"数据集": label}
        row.update(split_period_metrics(historical, results, groups[label]))
        rows.append(row)
    path.parent.mkdir(parents=True, exist_ok=True)
    headers = list(rows[0].keys())
    with path.open("w", encoding="utf-8-sig", newline="") as file:
        writer = csv.DictWriter(file, fieldnames=headers, lineterminator="\n")
        writer.writeheader()
        writer.writerows(rows)

    print("\n 问题二训练/验证/测试检验")
    for row in rows:
        print(
            f"    {row['数据集']} {row['开始日期']} 至 {row['结束日期']}: "
            f"    净负荷 RMSE={row['净负荷 RMSE (kW)']:.4f} kW, "
            f"    总费用={row['总购电费(元)']:.4f} 元, "
            f"    紧急购电占比={row['紧急购电占比']:.4%}"
        )
    print(f"数据集检验报告: {path.resolve()}")

```

```

def validate_year(
    historical: HistoricalData,
    results: list[DailyResult],
    cfg: Config,
) -> None:
    if len(results) != len(historical.dates):
        raise RuntimeError("全年结果数量与历史日期数量不一致。")
    tolerance = 1.0e-4
    if abs(results[0].soc_start_kwh - cfg.initial_soc_kwh) > tolerance:
        raise RuntimeError("2025-01-01 初始 SOC 不是 6000 kWh。")
    for index, item in enumerate(results):
        if item.day != historical.dates[index]:
            raise RuntimeError("逐日结果日期错位。")

```

```

        if index > 0 and abs(item.soc_start_kwh - results[index - 1].soc_end_kwh) > tolerance:
            raise RuntimeError(f"{item.day} 的 SOC 没有与前一天连续衔接。")
        if not (cfg.soc_min_kwh - tolerance <= item.soc_end_kwh <= cfg.soc_max_kwh + tolerance):
            raise RuntimeError(f"{item.day} 的日末 SOC 越界。")
        if np.any(item.grid_kwh < -tolerance) or np.any(item.emergency_kwh < -tolerance):
            raise RuntimeError(f"{item.day} 存在负购电量。")

def print_summary(
    historical: HistoricalData,
    results: list[DailyResult],
    output: Path,
    summary: Path,
) -> None:
    output_results = [item for item in results if item.day >= date(2025, 2, 1)]
    plan_energy = float(sum(np.sum(item.grid_kwh) for item in output_results))
    emergency_energy = float(sum(np.sum(item.emergency_kwh) for item in output_results))
    plan_cost = float(sum(item.plan_cost_yuan for item in output_results))
    emergency_cost = float(sum(item.emergency_cost_yuan for item in output_results))
    emergency_intervals = int(
        sum(np.count_nonzero(item.emergency_kwh > 1.0e-5) for item in output_results)
    )

    start_index = historical.dates.index(date(2025, 2, 1))
    actual_load = historical.load_kw[start_index:]
    actual_pv = historical.pv_kw[start_index:]
    forecast_load = np.asarray([item.forecast_load_kw for item in output_results])
    forecast_pv = np.asarray([item.forecast_pv_kw for item in output_results])
    load_mae = float(np.mean(np.abs(actual_load - forecast_load)))
    load_rmse = float(np.sqrt(np.mean((actual_load - forecast_load) ** 2)))
    pv_mae = float(np.mean(np.abs(actual_pv - forecast_pv)))
    pv_rmse = float(np.sqrt(np.mean((actual_pv - forecast_pv) ** 2)))

    print("\n 问题二计算结果 (2025-02-01 至 2025-12-31) ")
    print(f" 计划购电量: {plan_energy:.4f} kWh")
    print(f" 计划购电费: {plan_cost:.4f} 元")
    print(f" 紧急购电量: {emergency_energy:.4f} kWh")
    print(f" 紧急购电费: {emergency_cost:.4f} 元")
    print(f" 总购电费: {plan_cost + emergency_cost:.4f} 元")
    print(f" 发生紧急购电的 10 分钟时段数: {emergency_intervals}")
    print(f" 负荷滚动预测: MAE={load_mae:.4f} kW, RMSE={load_rmse:.4f} kW")
    print(f" 光伏滚动预测: MAE={pv_mae:.4f} kW, RMSE={pv_rmse:.4f} kW")
    print(
        f" SOC 范围: {min(item.soc_start_kwh for item in results):.4f} ~"
        f" {max(item.soc_end_kwh for item in results):.4f} kWh"
    )

specified = {date(2025, 3, 20), date(2025, 6, 21), date(2025, 9, 23), date(2025, 12, 21)}
print("\n 表 3 指定日期紧急购电")
for item in output_results:
    if item.day not in specified:
        continue
    groups = group_emergency(item.emergency_kwh, 1.0e-5)
    if not groups:
        print(f" {item.day.isoformat()}: 无")
    else:
        print(f" {item.day.isoformat()}: ")
        for period, amount in groups:
            print(f" {period}: {amount:.4f} kWh")

print(f"\n 正式结果: {output.resolve()}")
print(f" 逐日汇总: {summary.resolve()}")

@dataclass(frozen=True)
class LoadSearchPoint:
    history_days: int
    half_life_days: float
    same_weekday_multiplier: float
    wavelet_level: int
    annual_harmonics: int
    load_ridge_lambda: float
    dynamic_load_blend: float

    def config(self) -> Config:
        return replace(Config(), **asdict(self))

def _decode_load_point(unit: np.ndarray) -> LoadSearchPoint:
    x = np.clip(np.asarray(unit, dtype=float), 0.0, 1.0)

```

```

return LoadSearchPoint(
    history_days=int(round(14 + x[0] * 46)),
    half_life_days=float(3.0 + x[1] * 27.0),
    same_weekday_multiplier=float(x[2] * 5.0),
    wavelet_level=int(round(1 + x[3] * 3)),
    annual_harmonics=int(round(1 + x[4] * 7)),
    load_ridge_lambda=float(10.0 ** (-4.0 + 7.0 * x[5])),
    dynamic_load_blend=float(0.20 + x[6] * 0.80),
)

def _encode_load_point(point: LoadSearchPoint) -> np.ndarray:
    return np.asarray(
        [
            (point.history_days - 14) / 46,
            (point.half_life_days - 3.0) / 27.0,
            point.same_weekday_multiplier / 5.0,
            (point.wavelet_level - 1) / 3.0,
            (point.annual_harmonics - 1) / 7.0,
            (math.log10(point.load_ridge_lambda) + 4.0) / 7.0,
            (point.dynamic_load_blend - 0.20) / 0.80,
        ],
        dtype=float,
    )

def _load_point_key(point: LoadSearchPoint) -> tuple[object, ...]:
    return (
        point.history_days,
        round(point.half_life_days, 6),
        round(point.same_weekday_multiplier, 6),
        point.wavelet_level,
        point.annual_harmonics,
        round(math.log10(point.load_ridge_lambda), 6),
        round(point.dynamic_load_blend, 6),
    )

def _matern52(
    a: np.ndarray, b: np.ndarray, length_scale: float
) -> np.ndarray:
    distance = np.sqrt(
        np.maximum(0.0, np.sum((a[:, None, :] - b[None, :, :]) ** 2, axis=2))
    )
    scaled = math.sqrt(5.0) * distance / length_scale
    return (1.0 + scaled + scaled**2 / 3.0) * np.exp(-scaled)

def _expected_improvement(
    observed_x: np.ndarray,
    observed_y: np.ndarray,
    candidates: np.ndarray,
    length_scale: float,
) -> np.ndarray:
    mean_y = float(np.mean(observed_y))
    scale_y = max(float(np.std(observed_y)), 1.0e-12)
    normalized_y = (observed_y - mean_y) / scale_y
    kernel = _matern52(observed_x, observed_x, length_scale)
    kernel.flat[:: kernel.shape[0] + 1] += 1.0e-7
    try:
        chol = np.linalg.cholesky(kernel)
    except np.linalg.LinAlgError:
        kernel.flat[:: kernel.shape[0] + 1] += 1.0e-5
        chol = np.linalg.cholesky(kernel)
    alpha = np.linalg.solve(chol.T, np.linalg.solve(chol, normalized_y))
    cross = _matern52(observed_x, candidates, length_scale)
    mean = cross.T @ alpha
    solved = np.linalg.solve(chol, cross)
    sigma = np.sqrt(np.maximum(1.0e-12, 1.0 - np.sum(solved**2, axis=0)))
    best = float(np.min(normalized_y))
    improvement = best - mean - 0.001
    z = improvement / sigma
    cdf = 0.5 * (
        1.0 + np.asarray([math.erf(value / math.sqrt(2.0)) for value in z])
    )
    pdf = np.exp(-0.5 * z**2) / math.sqrt(2.0 * math.pi)
    return improvement * cdf + sigma * pdf

def _predict_load_indices(

```

```

baseline: BaselineData,
historical: HistoricalData,
cfg: Config,
indices: list[int],
training_indices: set[int],
incumbent_rmse: float | None = None,
early_stop_margin: float = 0.01,
) -> tuple[np.ndarray | None, float, bool]:
    hours = np.arange(144, dtype=float) / 6.0
    forecasts: list[np.ndarray] = []
    squared_error = 0.0
    total_values = len(indices) * 144
    for position, index in enumerate(indices, start=1):
        forecast = make_forecast(
            historical,
            baseline,
            index,
            historical.dates[index],
            hours,
            cfg,
            allowed_indices=training_indices,
        ).load_kw
        forecasts.append(forecast)
        squared_error += float(np.sum((historical.load_kw[index] - forecast) ** 2))
        if incumbent_rmse is not None and position >= 8:
            lower_bound = math.sqrt(squared_error / total_values)
            if lower_bound > incumbent_rmse * (1.0 + early_stop_margin):
                return None, lower_bound, True
    return (
        np.asarray(forecasts, dtype=float),
        math.sqrt(squared_error / total_values),
        False,
    )

def _load_metrics(actual: np.ndarray, forecast: np.ndarray) -> dict[str, float]:
    error = actual - forecast
    return {
        "mae_kw": float(np.mean(np.abs(error))),
        "rmse_kw": float(np.sqrt(np.mean(error**2))),
        "r2": _r_squared(actual, forecast),
    }

def run_load_bayesian_optimization(args: argparse.Namespace) -> None:
    if args.bayes_trials < 2 or not 1 <= args.bayes_initial < args.bayes_trials:
        raise ValueError("应满足 bayes-trials >= 2 且 1 <= bayes-initial < bayes-trials。")
    baseline, order = read_baseline(args.baseline)
    historical = read_historical_data(args.load, args.pv, order)
    groups = DatasetSplit().cycle_labels(historical.dates)
    training_indices = set(groups["训练集"])
    labels = {
        index: label for label, indices in groups.items() for index in indices
    }
    rng = np.random.default_rng(args.bayes_seed)
    base_cfg = Config()
    default_point = LoadSearchPoint(
        base_cfg.history_days,
        base_cfg.half_life_days,
        base_cfg.same_weekday_multiplier,
        base_cfg.wavelet_level,
        base_cfg.annual_harmonics,
        base_cfg.load_ridge_lambda,
        base_cfg.dynamic_load_blend,
    )
    current = default_point
    candidates = [default_point]
    candidate_sse = [0.0]
    seen = {_load_point_key(default_point)}
    validation_seen: list[int] = []
    all_forecasts: list[np.ndarray] = []
    proposals: list[dict[str, object]] = []
    selection_history: list[dict[str, object]] = []
    proposal_count = 1
    no_improvement = 0
    search_stopped = False
    hours = np.arange(144, dtype=float) / 6.0

    print("严格前向负荷贝叶斯优化: 测试集不参与参数搜索。")
    for index, day in enumerate(historical.dates):

```

```

current_forecast = make_forecast(
    historical,
    baseline,
    index,
    day,
    hours,
    current.config(),
    allowed_indices=training_indices,
).load_kw
all_forecasts.append(current_forecast)
if labels[index] != "验证集":
    continue
validation_seen.append(index)
actual = historical.load_kw[index]
for candidate_index, point in enumerate(candidates):
    forecast = (
        current_forecast
        if _load_point_key(point) == _load_point_key(current)
        else make_forecast(
            historical,
            baseline,
            index,
            day,
            hours,
            point.config(),
            allowed_indices=training_indices,
        ).load_kw
    )
    candidate_sse[candidate_index] += float(np.sum((actual - forecast) ** 2))

denominator = len(validation_seen) * 144
scores = np.sqrt(np.asarray(candidate_sse) / denominator)
incumbent = float(np.min(scores))
if proposal_count < args.bayes_trials and not search_stopped:
    if len(candidates) < args.bayes_initial:
        unit = rng.random(7)
        method = "随机初始化"
    else:
        pool = rng.random((args.bayes_candidate_pool, 7))
        acquisition = _expected_improvement(
            np.asarray([_encode_load_point(point) for point in candidates]),
            scores,
            pool,
            0.35,
        )
        unit = pool[int(np.argmax(acquisition))]
        method = "贝叶斯期望改进"
    point = _decode_load_point(unit)
    attempts = 0
    while _load_point_key(point) in seen and attempts < 100:
        point = _decode_load_point(rng.random(7))
        attempts += 1
    seen.add(_load_point_key(point))
    proposal_count += 1
    _, proposed_score, candidate_stopped = _predict_load_indices(
        baseline,
        historical,
        point.config(),
        validation_seen,
        training_indices,
        incumbent,
    )
    improved = (
        not candidate_stopped
        and proposed_score < incumbent - args.bayes_min_delta
    )
    if not candidate_stopped:
        candidates.append(point)
        candidate_sse.append(proposed_score**2 * denominator)
        no_improvement = 0 if improved else no_improvement + 1
        if len(candidates) >= args.bayes_initial:
            no_improvement += 1
    elif len(candidates) >= args.bayes_initial:
        no_improvement += 1
    proposals.append(
        {
            "proposal": proposal_count,
            "validation_date": day.isoformat(),
            "validation_days_available": len(validation_seen),
            "method": method,

```

```

        "validation_rmse_kw": proposed_score,
        "candidate_early_stopped": candidate_stopped,
        "improved": improved,
        "parameters": asdict(point),
    }
)
print(f"[{day}] 累计验证 RMSE={proposed_score:.4f} kW")
if len(candidates) >= args.bayes_initial and no_improvement >= args.bayes_patience:
    search_stopped = True
    print(f"连续 {args.bayes_patience} 个候选无显著改善, 停止寻优。")

scores = np.sqrt(np.asarray(candidate_sse) / denominator)
selected_index = int(np.argmin(scores))
if len(validation_seen) >= args.bayes_initial:
    current = candidates[selected_index]
    selection_history.append(
        {
            "date": day.isoformat(),
            "validation_days_available": len(validation_seen),
            "selected_validation_rmse_kw": float(scores[selected_index]),
            "selected_parameters": asdict(current),
        }
    )

forward = np.asarray(all_forecasts, dtype=float)
final_metrics = {
    label: _load_metrics(historical.load_kw[indices], forward[indices])
    for label, indices in groups.items()
}
baseline_metrics: dict[str, dict[str, float]] = {}
for label, indices in groups.items():
    forecast, _, stopped = _predict_load_indices(
        baseline,
        historical,
        default_point.config(),
        indices,
        training_indices,
    )
    if stopped or forecast is None:
        raise RuntimeError("最终评价不应提前停止。")
    baseline_metrics[label] = _load_metrics(historical.load_kw[indices], forecast)
report = {
    "data_split": {
        "rule": "训练、训练、训练、验证、训练、测试",
        "counts": {label: len(indices) for label, indices in groups.items()},
        "strict_training_freeze": True,
        "test_data_used_in_optimization": False,
    },
    "optimization": {
        "method": "strict forward Gaussian-process Bayesian optimization",
        "objective": "截至当前日期的累计验证集 RMSE",
        "early_stopping_patience": args.bayes_patience,
        "minimum_improvement_kw": args.bayes_min_delta,
        "maximum_proposals": args.bayes_trials,
        "completed_proposals": proposal_count,
        "search_early_stopped": search_stopped,
        "seed": args.bayes_seed,
    },
    "final_selected_parameters": asdict(current),
    "baseline_metrics": baseline_metrics,
    "metrics": final_metrics,
    "proposals": proposals,
    "selection_history": selection_history,
}
args.bayes_report.parent.mkdir(parents=True, exist_ok=True)
args.bayes_report.write_text(
    json.dumps(report, ensure_ascii=False, indent=2), encoding="utf-8"
)
for label in ("训练集", "验证集", "测试集"):
    values = final_metrics[label]
    print(
        f"{label}: MAE={values['mae_kw']:.4f} kW, "
        f"RMSE={values['rmse_kw']:.4f} kW, R2={values['r2']:.6f}"
    )
print(f"实验报告: {args.bayes_report.resolve()}")

```

```

def parse_args() -> argparse.Namespace:
    parser = argparse.ArgumentParser(description="C 题问题二滚动预测与购电线性规划")
    parser.add_argument("--baseline", type=Path, default=DEFAULT_BASELINE)

```

```

parser.add_argument("--load", type=Path, default=DEFAULT_LOAD)
parser.add_argument("--pv", type=Path, default=DEFAULT_PV)
parser.add_argument("--template", type=Path, default=DEFAULT_TEMPLATE)
parser.add_argument("--output", type=Path, default=DEFAULT_OUTPUT)
parser.add_argument("--summary", type=Path, default=DEFAULT_SUMMARY)
parser.add_argument("--split-report", type=Path, default=DEFAULT_SPLIT_REPORT)
parser.add_argument("--train-end", type=date.fromisoformat, default=date(2025, 7, 31))
parser.add_argument(
    "--validation-end", type=date.fromisoformat, default=date(2025, 9, 30)
)
)
parser.add_argument(
    "--bayes-optimize",
    action="store_true",
    help="执行严格前向贝叶斯参数寻优，不生成正式购电结果表。",
)
parser.add_argument("--bayes-trials", type=int, default=40)
parser.add_argument("--bayes-initial", type=int, default=10)
parser.add_argument("--bayes-patience", type=int, default=12)
parser.add_argument("--bayes-min-delta", type=float, default=0.05)
parser.add_argument("--bayes-candidate-pool", type=int, default=2000)
parser.add_argument("--bayes-seed", type=int, default=2026)
parser.add_argument("--bayes-report", type=Path, default=DEFAULT_BAYES_REPORT)
return parser.parse_args()

def main() -> None:
    args = parse_args()
    if args.bayes_optimize:
        run_load_bayesian_optimization(args)
        return
    cfg = Config()
    baseline, chronological_order = read_baseline(args.baseline)
    historical = read_historical_data(args.load, args.pv, chronological_order)
    split = DatasetSplit(args.train_end, args.validation_end)
    groups = split.cycle_labels(historical.dates)
    training_indices = set(groups["训练集"])
    results = simulate_year(
        baseline, historical, cfg, training_indices=training_indices
    )
    validate_year(historical, results, cfg)
    write_result_workbook(
        args.template,
        args.output,
        results,
        baseline.price,
        chronological_order,
        cfg,
    )
    write_daily_summary(args.summary, historical, results)
    write_split_evaluation(args.split_report, historical, results, split)
    print_summary(historical, results, args.output, args.summary)

if __name__ == "__main__":
    main()

```

C.3 问题三程序 three.py

程序说明：在问题二基准计划上引入 6:00、12:00 和 18:00 预报信息，完成光伏预报插值、负荷偏差在线修正和分时段 MPC-LP 重优化，同时校验调整购电、减购边界、储能状态与紧急购电，输出 result3.xlsx。

```

# 在问题二模型基础上：
# - 负荷继承严格前向贝叶斯优化后的年度谐波—岭回归—傅里叶—Haar 小波模型，并用当日已观测负荷在线纠偏；
# - 光伏使用附件 3 在 0:00、6:00、12:00、18:00 发布的未来 24 小时预报；
# - 每次求解未来 24 小时，只执行到下一次预报发布时刻；
# - 增购部分按 1.5 倍电价，减购部分退回原价并支付 50%违约金；
# - 真实供电缺口按 5 倍电价紧急购电。

# 程序比较四种预报组合，并将总费用最低的策略写入正式结果。默认生成：
# C 题/result3.xlsx
# C 题/问题三逐日汇总.csv
# C 题/问题三预报时刻对比.csv
# 其中策略只按验证集费用选择，测试集仅用于最终泛化检验。

# 依赖：two.py、numpy、scipy、openpyxl
# 运行：python three.py

```

```

from __future__ import annotations

import argparse
import csv
import math
import os
import shutil
from copy import copy
from dataclasses import dataclass
from datetime import date, datetime, timedelta
from pathlib import Path

import numpy as np
from openpyxl import load_workbook

import two

ROOT = Path(__file__).resolve().parent
DEFAULT_FORECAST = ROOT / "C 题" / "附件" / "csv" / "附件 3.csv"
DEFAULT_TEMPLATE = ROOT / "C 题" / "附件" / "附件 5" / "result3.xlsx"
DEFAULT_OUTPUT = ROOT / "C 题" / "result3.xlsx"
DEFAULT_SUMMARY = ROOT / "C 题" / "问题三逐日汇总.csv"
DEFAULT_COMPARISON = ROOT / "C 题" / "问题三预报时刻对比.csv"
DEFAULT_SPLIT_REPORT = ROOT / "C 题" / "问题三_训练验证测试检验.csv"

@dataclass(frozen=True)
class Config(two.Config):
    adjustment_up_multiplier: float = 1.50
    adjustment_down_penalty: float = 0.50
    load_bias_half_life_hours: float = 6.0

@dataclass
class ForecastData:
    values: dict[tuple[date, int], np.ndarray]

@dataclass
class DecisionInput:
    load_kw: np.ndarray
    pv_kw: np.ndarray
    reserve_kwh: np.ndarray

@dataclass
class AdjustmentSolution:
    grid_kwh: np.ndarray
    charge_kwh: np.ndarray
    discharge_kwh: np.ndarray
    curtailment_kwh: np.ndarray
    increase_kwh: np.ndarray
    decrease_kwh: np.ndarray
    soc_kwh: np.ndarray

@dataclass
class DailyResult:
    day: date
    plan_grid_kwh: np.ndarray
    final_grid_kwh: np.ndarray
    charge_kwh: np.ndarray
    discharge_kwh: np.ndarray
    emergency_kwh: np.ndarray
    actual_curtailment_kwh: np.ndarray
    soc_start_kwh: float
    soc_end_kwh: float
    plan_cost_yuan: float
    settled_purchase_cost_yuan: float
    emergency_cost_yuan: float

    @property
    def adjustment_delta_cost_yuan(self) -> float:
        return self.settled_purchase_cost_yuan - self.plan_cost_yuan

    @property
    def total_cost_yuan(self) -> float:
        return self.settled_purchase_cost_yuan + self.emergency_cost_yuan

```

```

@dataclass
class StrategySummary:
    name: str
    release_hours: tuple[int, ...]
    results: list[DailyResult]
    plan_energy_kwh: float
    final_energy_kwh: float
    increase_energy_kwh: float
    decrease_energy_kwh: float
    emergency_energy_kwh: float
    plan_cost_yuan: float
    settled_purchase_cost_yuan: float
    emergency_cost_yuan: float
    total_cost_yuan: float

def read_forecasts(path: Path) -> ForecastData:
    if not path.exists():
        raise FileNotFoundError(f"找不到附件 3 CSV: {path}")
    with path.open("r", encoding="utf-8-sig", newline="") as file:
        reader = csv.reader(file)
        header = next(reader, None)
        rows = list(reader)
    if header is None or len(header) != 26:
        raise ValueError("附件 3 应包含日期、预报时刻和未来 24 小时预报共 26 列。")
    if len(rows) != 365 * 4:
        raise ValueError("附件 3 应包含 365 天、每天 4 个发布时间的数据。")

    values: dict[tuple[date, int], np.ndarray] = {}
    current_date: date | None = None
    for row_number, row in enumerate(rows, start=2):
        if len(row) != 26:
            raise ValueError(f"附件 3 第 {row_number} 行列数不是 26。")
        if row[0].strip():
            try:
                current_date = datetime.strptime(row[0].strip(), "%Y-%m-%d").date()
            except ValueError as exc:
                raise ValueError(f"附件 3 第 {row_number} 行日期无法解析: {row[0]!r}") from exc
        if current_date is None:
            raise ValueError(f"附件 3 第 {row_number} 行缺少所属日期。")
        try:
            release_hour = int(row[1].strip().split(":", 1)[0])
        except (ValueError, IndexError) as exc:
            raise ValueError(f"附件 3 第 {row_number} 行预报时刻无法解析。") from exc
        if release_hour not in (0, 6, 12, 18):
            raise ValueError(f"附件 3 第 {row_number} 行预报时刻不是 0、6、12 或 18 时。")
        forecast = np.asarray(
            [two._float(value, f"附件 3 第 {row_number} 行预报值") for value in row[2:]],
            dtype=float,
        )
        if np.any(forecast < 0):
            raise ValueError(f"附件 3 第 {row_number} 行包含负光伏功率。")
        key = (current_date, release_hour)
        if key in values:
            raise ValueError(f"附件 3 存在重复预报: {key}")
        values[key] = forecast

    expected_dates = [date(2025, 1, 1) + timedelta(days=i) for i in range(365)]
    expected_keys = {(day, hour) for day in expected_dates for hour in (0, 6, 12, 18)}
    if set(values) != expected_keys:
        raise ValueError("附件 3 没有完整覆盖 2025 年每天的四个预报时刻。")
    return ForecastData(values)

def forecast_load_profile(
    historical: two.HistoricalData,
    baseline: two.BaselineData,
    cutoff: int,
    target_date: date,
    hours: np.ndarray,
    cfg: Config,
    allowed_indices: set[int] | None = None,
) -> np.ndarray:
    profile = two._weighted_history_profile(
        historical.load_kw,
        historical.dates,
        cutoff,
        target_date,

```

```

        baseline.load_kw,
        cfg,
        allowed_indices,
    )
    dynamic_load = two.fit_dynamic_load_profile(
        historical,
        cutoff,
        target_date,
        baseline.load_kw,
        cfg,
        allowed_indices,
    )
    blend = float(np.clip(cfg.dynamic_load_blend, 0.0, 1.0))
    profile = (1.0 - blend) * profile + blend * dynamic_load
    fourier = two.fit_load_fourier(hours, profile)
    residual = two.wavelet_correct_residual(profile, fourier, cfg.wavelet_level)
    return np.maximum(0.0, fourier + residual)

def _horizon_from_daily(
    current: np.ndarray, following: np.ndarray, start_index: int
) -> np.ndarray:
    if start_index == 0:
        return current.copy()
    return np.concatenate([current[start_index:], following[:start_index]])

def interpolate_pv_forecast(
    hourly_forecast: np.ndarray, anchor_power_kw: float
) -> np.ndarray:
    xp = np.arange(25, dtype=float)
    fp = np.concatenate([[max(0.0, anchor_power_kw)], hourly_forecast])
    target = np.arange(1, 145, dtype=float) / 6.0
    return np.maximum(0.0, np.interp(target, xp, fp))

def actual_horizon(matrix: np.ndarray, day_index: int, start_index: int) -> np.ndarray:
    if start_index == 0:
        return matrix[day_index].copy()
    if day_index + 1 >= len(matrix):
        raise IndexError("年末最后一天之后没有真实数据，不能构造误差回放窗口。")
    return np.concatenate(
        [matrix[day_index, start_index:], matrix[day_index + 1, :start_index]]
    )

def prepare_decision_inputs(
    baseline: two.BaselineData,
    historical: two.HistoricalData,
    forecasts: ForecastData,
    cfg: Config,
    training_cutoff: int | None = None,
    training_indices: set[int] | None = None,
) -> dict[tuple[int, int], DecisionInput]:
    hours = np.arange(144, dtype=float) * cfg.dt_hours
    raw: dict[tuple[int, int], tuple[np.ndarray, np.ndarray]] = {}

    for day_index, current_date in enumerate(historical.dates):
        # 训练期逐日因果拟合；验证和测试阶段固定使用训练集样本。
        model_cutoff = (
            day_index
            if training_cutoff is None
            else min(day_index, training_cutoff)
        )
        allowed_indices = training_indices
        forecast_cfg = two.bayesian_load_config(cfg, current_date)
        current_load = forecast_load_profile(
            historical, baseline, model_cutoff, current_date, hours, forecast_cfg,
            allowed_indices=allowed_indices,
        )
        next_load = forecast_load_profile(
            historical,
            baseline,
            model_cutoff,
            current_date + timedelta(days=1),
            hours,
            forecast_cfg,
            allowed_indices=training_indices,
        )
        for release_hour in (0, 6, 12, 18):

```

```

start = release_hour * 6
load_horizon = _horizon_from_daily(current_load, next_load, start)

# 利用发布时刻之前最近 1 小时的真实负荷偏差进行在线修正。
if start > 0:
    observed = historical.load_kw[day_index, max(0, start - 6) : start]
    predicted = current_load[max(0, start - 6) : start]
    bias = float(np.mean(observed - predicted))
    half_life_steps = cfg.load_bias_half_life_hours / cfg.dt_hours
    decay = np.exp(-math.log(2.0) * np.arange(144) / half_life_steps)
    load_horizon = np.maximum(0.0, load_horizon + bias * decay)

    anchor = float(historical.pv_kw[day_index, start])
    pv_horizon = interpolate_pv_forecast(
        forecasts.values[(current_date, release_hour)], anchor
    )
    raw[(day_index, release_hour)] = (load_horizon, pv_horizon)

prepared: dict[tuple[int, int], DecisionInput] = {}
for day_index, _ in enumerate(historical.dates):
    for release_hour in (0, 6, 12, 18):
        load_horizon, pv_horizon = raw[(day_index, release_hour)]
        residuals: list[np.ndarray] = []
        first = max(0, day_index - cfg.risk_days)
        for historical_index in range(first, day_index):
            if training_indices is not None and historical_index not in training_indices:
                continue
            # 历史同一发布时间的 24 小时误差在当前发布时间已经全部可观测。
            hist_load, hist_pv = raw[(historical_index, release_hour)]
            actual_load = actual_horizon(
                historical.load_kw,
                historical_index,
                release_hour * 6,
            )
            actual_pv = actual_horizon(
                historical.pv_kw,
                historical_index,
                release_hour * 6,
            )
            residuals.append(
                (actual_load - actual_pv - hist_load + hist_pv) * cfg.dt_hours
            )
        if len(residuals) >= cfg.minimum_risk_days:
            reserve = np.maximum(
                0.0,
                np.quantile(np.asarray(residuals), cfg.risk_quantile, axis=0),
            )
        else:
            reserve = np.zeros(144, dtype=float)
        prepared[(day_index, release_hour)] = DecisionInput(
            load_horizon, pv_horizon, reserve
        )
    return prepared

def solve_adjustment_lp(
    price: np.ndarray,
    load_kw: np.ndarray,
    pv_kw: np.ndarray,
    initial_soc_kwh: float,
    original_plan_current_day: np.ndarray,
    current_day_slots: int,
    cfg: Config,
) -> AdjustmentSolution:
    try:
        from scipy.optimize import linprog
        from scipy.sparse import coo_matrix
    except ModuleNotFoundError as exc:
        raise ModuleNotFoundError("缺少 scipy, 请先执行: python -m pip install scipy") from exc

    n = 144
    if any(len(array) != n for array in (price, load_kw, pv_kw)):
        raise ValueError("MPC 调整模型的预测窗口必须包含 144 个 10 分钟时段。")
    if len(original_plan_current_day) != current_day_slots:
        raise ValueError("原计划参考量长度与当天剩余时段数不一致。")
    load_energy = load_kw * cfg.dt_hours
    pv_energy = pv_kw * cfg.dt_hours

    g0, c0, d0, w0, u0, v0, e0 = (
        0,

```

```

n,
2 * n,
3 * n,
4 * n,
5 * n,
6 * n,
)
variable_count = 7 * n + 1
objective = np.zeros(variable_count, dtype=float)
objective[c0 : c0 + n] = cfg.throughput_tie_breaker
objective[d0 : d0 + n] = cfg.throughput_tie_breaker
objective[u0 : u0 + current_day_slots] = (
    cfg.adjustment_up_multiplier * price[:current_day_slots]
)
objective[v0 : v0 + current_day_slots] = (
    -cfg.adjustment_down_penalty * price[:current_day_slots]
)
# 跨过午夜的远期时段尚未形成下一日原计划，按普通购电价格计入终端规划。
objective[g0 + current_day_slots : g0 + n] = price[current_day_slots:]

rows: list[int] = []
columns: list[int] = []
coefficients: list[float] = []
equality_count = 2 * n + current_day_slots
b_eq = np.zeros(equality_count, dtype=float)
for t in range(n):
    rows.extend([t, t, t, t])
    columns.extend([g0 + t, c0 + t, d0 + t, w0 + t])
    coefficients.extend([1.0, -1.0, 1.0, -1.0])
    b_eq[t] = load_energy[t] - pv_energy[t]

    row = n + t
    rows.extend([row, row, row, row])
    columns.extend([c0 + t, d0 + t, e0 + t, e0 + t + 1])
    coefficients.extend(
        [-cfg.charge_efficiency, 1.0 / cfg.discharge_efficiency, -1.0, 1.0]
    )

for t in range(current_day_slots):
    row = 2 * n + t
    # g = G_plan + u - v
    rows.extend([row, row, row])
    columns.extend([g0 + t, u0 + t, v0 + t])
    coefficients.extend([1.0, -1.0, 1.0])
    b_eq[row] = original_plan_current_day[t]

a_eq = coo_matrix(
    (coefficients, (rows, columns)), shape=(equality_count, variable_count)
).tocsr()

bounds: list[tuple[float | None, float | None]] = []
bounds.extend([(0.0, None)] * n)
bounds.extend([(0.0, cfg.max_charge_kwh)] * n)
bounds.extend([(0.0, cfg.max_discharge_kwh)] * n)
bounds.extend([(0.0, None)] * n)
bounds.extend([(0.0, None)] * current_day_slots)
bounds.extend([(0.0, 0.0)] * (n - current_day_slots))
bounds.extend([(0.0, None)] * current_day_slots)
bounds.extend([(0.0, 0.0)] * (n - current_day_slots))
bounds.extend([(cfg.soc_min_kwh, cfg.soc_max_kwh)] * (n + 1))
bounds[e0] = (initial_soc_kwh, initial_soc_kwh)
bounds[e0 + n] = (initial_soc_kwh, initial_soc_kwh)

result = linprog(
    objective,
    A_eq=a_eq,
    b_eq=b_eq,
    bounds=bounds,
    method="highs",
    options={"presolve": True},
)
if not result.success:
    raise RuntimeError(f"MPC 调整线性规划失败: {result.status}, {result.message}")

x = np.asarray(result.x, dtype=float)
solution = AdjustmentSolution(
    x[g0 : g0 + n],
    x[c0 : c0 + n],
    x[d0 : d0 + n],
    x[w0 : w0 + n],

```

```

        x[u0 : u0 + n],
        x[v0 : v0 + n],
        x[e0 : e0 + n + 1],
    )
    validate_adjustment_solution(
        solution,
        load_energy,
        pv_energy,
        initial_soc_kwh,
        original_plan_current_day,
        current_day_slots,
        cfg,
    )
    return solution

def validate_adjustment_solution(
    solution: AdjustmentSolution,
    load_energy: np.ndarray,
    pv_energy: np.ndarray,
    initial_soc_kwh: float,
    original_plan: np.ndarray,
    current_day_slots: int,
    cfg: Config,
) -> None:
    tolerance = 1.0e-5
    balance = (
        solution.grid_kwh
        + pv_energy
        + solution.discharge_kwh
        - load_energy
        - solution.charge_kwh
        - solution.curtailment_kwh
    )
    next_soc = (
        solution.soc_kwh[:-1]
        + cfg.charge_efficiency * solution.charge_kwh
        - solution.discharge_kwh / cfg.discharge_efficiency
    )
    adjustment = (
        solution.grid_kwh[current_day_slots]
        - solution.increase_kwh[current_day_slots]
        + solution.decrease_kwh[current_day_slots]
        - original_plan
    )
    checks = {
        "电量平衡": float(np.max(np.abs(balance))),
        "SOC 递推": float(np.max(np.abs(solution.soc_kwh[1:] - next_soc))),
        "调整量关系": float(np.max(np.abs(adjustment))),
        "初始 SOC": abs(float(solution.soc_kwh[0]) - initial_soc_kwh),
        "终端 SOC": abs(float(solution.soc_kwh[-1]) - initial_soc_kwh),
    }
    failed = {name: value for name, value in checks.items() if value > tolerance}
    if failed:
        raise RuntimeError(f"MPC 调整结果校验失败: {failed}")
    if np.min(solution.soc_kwh) < cfg.soc_min_kwh - tolerance:
        raise RuntimeError("MPC 调整结果 SOC 低于下限。")
    if np.max(solution.soc_kwh) > cfg.soc_max_kwh + tolerance:
        raise RuntimeError("MPC 调整结果 SOC 高于上限。")
    if np.max(np.minimum(solution.charge_kwh, solution.discharge_kwh)) > 1.0e-4:
        raise RuntimeError("MPC 调整结果出现同一时段同时充放电。")

def settled_purchase_cost(
    price: np.ndarray,
    original_plan: np.ndarray,
    final_grid: np.ndarray,
    cfg: Config,
) -> float:
    increase = np.maximum(0.0, final_grid - original_plan)
    decrease = np.maximum(0.0, original_plan - final_grid)
    # 计划费用 + 1.5 倍增购费用 - 已取消电量原价退款 + 50%违约金
    return float(
        np.dot(price, original_plan)
        + np.dot(cfg.adjustment_up_multiplier * price, increase)
        - np.dot((1.0 - cfg.adjustment_down_penalty) * price, decrease)
    )

def execute_soc(

```

```

    initial_soc: float,
    charge: np.ndarray,
    discharge: np.ndarray,
    cfg: Config,
) -> float:
    soc = float(initial_soc)
    for c_value, d_value in zip(charge, discharge):
        soc += cfg.charge_efficiency * float(c_value)
        soc -= float(d_value) / cfg.discharge_efficiency
    if not (cfg.soc_min_kwh - 1.0e-4 <= soc <= cfg.soc_max_kwh + 1.0e-4):
        raise RuntimeError(f"执行阶段 SOC 越界: {soc}")
    return soc

def simulate_strategy(
    name: str,
    release_hours: tuple[int, ...],
    baseline: two.BaselineData,
    historical: two.HistoricalData,
    decision_inputs: dict[tuple[int, int], DecisionInput],
    cfg: Config,
) -> StrategySummary:
    if 0 not in release_hours:
        raise ValueError("所有策略都必须包含 0:00 计划。")
    results: list[DailyResult] = []
    soc = cfg.initial_soc_kwh

    for day_index, current_date in enumerate(historical.dates):
        soc_start = soc
        input_zero = decision_inputs[(day_index, 0)]
        risk_load = input_zero.load_kw + input_zero.reserve_kwh / cfg.dt_hours
        plan_solution = two.solve_rolling_lp(
            baseline.price,
            risk_load,
            input_zero.pv_kw,
            soc,
            cfg,
        )
        plan_grid = plan_solution.grid_kwh.copy()
        final_grid = plan_solution.grid_kwh.copy()
        final_charge = plan_solution.charge_kwh.copy()
        final_discharge = plan_solution.discharge_kwh.copy()

        for release_hour in (0, 6, 12, 18):
            start = release_hour * 6
            if release_hour > 0 and release_hour in release_hours:
                decision = decision_inputs[(day_index, release_hour)]
                risk_load = decision.load_kw + decision.reserve_kwh / cfg.dt_hours
                price_horizon = np.concatenate(
                    [baseline.price[start:], baseline.price[:start]]
                )
                current_slots = 144 - start
                adjusted = solve_adjustment_lp(
                    price_horizon,
                    risk_load,
                    decision.pv_kw,
                    soc,
                    plan_grid[start:],
                    current_slots,
                    cfg,
                )
                final_grid[start:] = adjusted.grid_kwh[:current_slots]
                final_charge[start:] = adjusted.charge_kwh[:current_slots]
                final_discharge[start:] = adjusted.discharge_kwh[:current_slots]

            stop = min(start + 36, 144)
            soc = execute_soc(
                soc,
                final_charge[start:stop],
                final_discharge[start:stop],
                cfg,
            )

    emergency, actual_curtailment = two.replay_actual_day(
        historical.load_kw[day_index],
        historical.pv_kw[day_index],
        final_grid,
        final_charge,
        final_discharge,
        cfg,
    )

```

```

    )
    plan_cost = float(np.dot(baseline.price, plan_grid))
    purchase_cost = settled_purchase_cost(
        baseline.price, plan_grid, final_grid, cfg
    )
    emergency_cost = float(
        np.dot(cfg.emergency_price_multiplier * baseline.price, emergency)
    )
    results.append(
        DailyResult(
            current_date,
            plan_grid,
            final_grid,
            final_charge,
            final_discharge,
            emergency,
            actual_curtailment,
            soc_start,
            soc,
            plan_cost,
            purchase_cost,
            emergency_cost,
        )
    )

    if (day_index + 1) % 90 == 0 or day_index == len(historical.dates) - 1:
        print(
            f"{name} 已完成 {current_date.isoformat()}: "
            f"SOC={soc:.2f} kWh, 紧急购电={np.sum(emergency):.2f} kWh"
        )

    output_results = [item for item in results if item.day >= date(2025, 2, 1)]
    plan_energy = float(sum(np.sum(item.plan_grid_kwh) for item in output_results))
    final_energy = float(sum(np.sum(item.final_grid_kwh) for item in output_results))
    increase_energy = float(
        sum(
            np.sum(np.maximum(0.0, item.final_grid_kwh - item.plan_grid_kwh))
            for item in output_results
        )
    )
    decrease_energy = float(
        sum(
            np.sum(np.maximum(0.0, item.plan_grid_kwh - item.final_grid_kwh))
            for item in output_results
        )
    )
    emergency_energy = float(
        sum(np.sum(item.emergency_kwh) for item in output_results)
    )
    plan_cost = float(sum(item.plan_cost_yuan for item in output_results))
    purchase_cost = float(
        sum(item.settled_purchase_cost_yuan for item in output_results)
    )
    emergency_cost = float(sum(item.emergency_cost_yuan for item in output_results))
    return StrategySummary(
        name,
        release_hours,
        results,
        plan_energy,
        final_energy,
        increase_energy,
        decrease_energy,
        emergency_energy,
        plan_cost,
        purchase_cost,
        emergency_cost,
        purchase_cost + emergency_cost,
    )

def validate_strategy(summary: StrategySummary, cfg: Config) -> None:
    if len(summary.results) != 365:
        raise RuntimeError(f"{summary.name} 没有生成 365 天结果。")
    if abs(summary.results[0].soc_start_kwh - cfg.initial_soc_kwh) > 1.0e-4:
        raise RuntimeError(f"{summary.name} 的 1 月 1 日初始 SOC 错误。")
    for index, item in enumerate(summary.results):
        if index > 0:
            previous = summary.results[index - 1]
            if abs(item.soc_start_kwh - previous.soc_end_kwh) > 1.0e-4:
                raise RuntimeError(f"{summary.name} 在 {item.day} 的跨日 SOC 不连续。")

```

```

        if np.any(item.plan_grid_kwh < -1.0e-5):
            raise RuntimeError(f"{summary.name} 存在负计划购电量。")
        if np.any(item.final_grid_kwh < -1.0e-5):
            raise RuntimeError(f"{summary.name} 存在负调整购电量。")

def _copy_cell_style(source, target) -> None:
    target._style = copy(source._style)
    target.number_format = source.number_format
    target.font = copy(source.font)
    target.fill = copy(source.fill)
    target.border = copy(source.border)
    target.alignment = copy(source.alignment)
    target.protection = copy(source.protection)

def write_result_workbook(
    template: Path,
    output: Path,
    summary: StrategySummary,
    baseline: two.BaselineData,
    chronological_order: list[int],
    cfg: Config,
) -> None:
    if not template.exists():
        raise FileNotFoundError(f"找不到 result3 模板: {template}")
    if template.resolve() == output.resolve():
        raise ValueError("输出路径不能覆盖官方模板。")
    output.parent.mkdir(parents=True, exist_ok=True)
    temporary = output.with_name(f"{output.stem}.building{output.suffix}")
    if temporary.exists():
        temporary.unlink()
    shutil.copy2(template, temporary)

    try:
        workbook = load_workbook(temporary)
        plan_sheet = workbook["计划购电量"]
        adjust_sheet = workbook["调整购电量"]
        storage_sheet = workbook["充放电量"]
        emergency_sheet = workbook["紧急购电量"]
        output_results = [
            item for item in summary.results if item.day >= date(2025, 2, 1)
        ]
        if len(output_results) != 334:
            raise RuntimeError("result3 应包含 2025-02-01 至 2025-12-31 共 334 天。")

        for sheet in (plan_sheet, adjust_sheet):
            template_dates = [
                sheet.cell(row, 1).value.date()
                if isinstance(sheet.cell(row, 1).value, datetime)
                else sheet.cell(row, 1).value
                for row in range(2, sheet.max_row + 1)
            ]
            if template_dates != [item.day for item in output_results]:
                raise ValueError(f"{sheet.title} 日期与计算日期不一致。")

        for row, item in enumerate(output_results, start=2):
            plan_values = two._to_template_order(
                item.plan_grid_kwh, chronological_order
            )
            adjusted_values = two._to_template_order(
                item.final_grid_kwh, chronological_order
            )
            for column, value in enumerate(plan_values, start=2):
                plan_sheet.cell(row, column, round(float(value), 4))
            for column, value in enumerate(adjusted_values, start=2):
                adjust_sheet.cell(row, column, round(float(value), 4))
            plan_sheet.cell(row, 146, round(float(np.sum(item.plan_grid_kwh)), 4))
            plan_sheet.cell(row, 147, round(item.plan_cost_yuan, 4))
            adjust_sheet.cell(row, 146, round(float(np.sum(item.final_grid_kwh)), 4))
            adjust_sheet.cell(
                row, 147, round(item.settled_purchase_cost_yuan, 4)
            )

        storage_styles = [
            [copy(storage_sheet.cell(row, column)) for column in range(1, 7)]
            for row in (2, 3)
        ]
        if storage_sheet.max_row > 1:
            storage_sheet.delete_rows(2, storage_sheet.max_row - 1)

```

```

periods = [
    "0:00-4:00",
    "4:00-8:00",
    "8:00-12:00",
    "12:00-16:00",
    "16:00-20:00",
    "20:00-24:00",
]
storage_row = 2
for item in output_results:
    for block, period in enumerate(periods):
        style_row = storage_styles[0 if block == 0 else 1]
        for column in range(1, 7):
            _copy_cell_style(
                style_row[column - 1], storage_sheet.cell(storage_row, column)
            )
        start, stop = block * 24, (block + 1) * 24
        storage_sheet.cell(storage_row, 1, item.day if block == 0 else None)
        storage_sheet.cell(storage_row, 2, period)
        storage_sheet.cell(
            storage_row,
            3,
            round(float(np.sum(item.charge_kwh[start:stop])), 4),
        )
        storage_sheet.cell(
            storage_row,
            4,
            round(float(np.sum(item.discharge_kwh[start:stop])), 4),
        )
        if block == 0:
            storage_sheet.cell(storage_row, 5, "0:00")
            storage_sheet.cell(storage_row, 6, round(item.soc_start_kwh, 4))
        elif block == 1:
            storage_sheet.cell(storage_row, 5, "24:00")
            storage_sheet.cell(storage_row, 6, round(item.soc_end_kwh, 4))
        storage_row += 1

emergency_styles = [
    [copy(emergency_sheet.cell(row, column)) for column in range(1, 4)]
    for row in (2, 3)
]
if emergency_sheet.max_row > 1:
    emergency_sheet.delete_rows(2, emergency_sheet.max_row - 1)
emergency_row = 2
for item in output_results:
    groups = two.group_emergency(
        item.emergency_kwh, cfg.emergency_tolerance_kwh
    )
    for group_index, (period, amount) in enumerate(groups):
        style_row = emergency_styles[0 if group_index == 0 else 1]
        for column in range(1, 4):
            _copy_cell_style(
                style_row[column - 1],
                emergency_sheet.cell(emergency_row, column),
            )
        emergency_sheet.cell(
            emergency_row, 1, item.day if group_index == 0 else None
        )
        emergency_sheet.cell(emergency_row, 2, period)
        emergency_sheet.cell(emergency_row, 3, round(amount, 4))
        emergency_row += 1

workbook.save(temporary)
try:
    os.replace(temporary, output)
except PermissionError as exc:
    raise PermissionError(
        f"无法写入 {output}, 请关闭正在打开该文件的 Excel 窗口后重试。"
    ) from exc
finally:
    if temporary.exists():
        temporary.unlink()

def write_daily_summary(path: Path, summary: StrategySummary) -> None:
    path.parent.mkdir(parents=True, exist_ok=True)
    with path.open("w", encoding="utf-8-sig", newline="") as file:
        writer = csv.writer(file, lineterminator="\n")
        writer.writerow(
            [
                "日期",

```

```

        "0 点计划购电量(kWh)",
        "最终调整购电量(kWh)",
        "增加购电量(kWh)",
        "减少购电量(kWh)",
        "0 点计划购电费(元)",
        "调整后购电结算费(元)",
        "调整相关费用变化(元)",
        "紧急购电量(kWh)",
        "紧急购电费(元)",
        "总购电费(元)",
        "0:00 储电量(kWh)",
        "24:00 储电量(kWh)",
    ]
)
for item in summary.results:
    increase = float(
        np.sum(np.maximum(0.0, item.final_grid_kwh - item.plan_grid_kwh))
    )
    decrease = float(
        np.sum(np.maximum(0.0, item.plan_grid_kwh - item.final_grid_kwh))
    )
    writer.writerow(
        [
            item.day.isoformat(),
            f"{np.sum(item.plan_grid_kwh):.6f}",
            f"{np.sum(item.final_grid_kwh):.6f}",
            f"{increase:.6f}",
            f"{decrease:.6f}",
            f"{item.plan_cost_yuan:.6f}",
            f"{item.settled_purchase_cost_yuan:.6f}",
            f"{item.adjustment_delta_cost_yuan:.6f}",
            f"{np.sum(item.emergency_kwh):.6f}",
            f"{item.emergency_cost_yuan:.6f}",
            f"{item.total_cost_yuan:.6f}",
            f"{item.soc_start_kwh:.6f}",
            f"{item.soc_end_kwh:.6f}",
        ]
    )
)

def write_strategy_comparison(path: Path, summaries: list[StrategySummary]) -> None:
    path.parent.mkdir(parents=True, exist_ok=True)
    baseline_total = summaries[0].total_cost_yuan
    with path.open("w", encoding="utf-8-sig", newline="") as file:
        writer = csv.writer(file, lineterminator="\n")
        writer.writerow(
            [
                "策略",
                "使用预报时刻",
                "计划购电量(kWh)",
                "最终购电量(kWh)",
                "增加购电量(kWh)",
                "减少购电量(kWh)",
                "紧急购电量(kWh)",
                "计划购电费(元)",
                "调整后购电结算费(元)",
                "紧急购电费(元)",
                "总购电费(元)",
                "相对仅 0 点策略节省(元)",
                "相对仅 0 点策略节省率",
            ]
        )
    for item in summaries:
        saving = baseline_total - item.total_cost_yuan
        writer.writerow(
            [
                item.name,
                "/".join(f"{hour}:00" for hour in item.release_hours),
                f"{item.plan_energy_kwh:.6f}",
                f"{item.final_energy_kwh:.6f}",
                f"{item.increase_energy_kwh:.6f}",
                f"{item.decrease_energy_kwh:.6f}",
                f"{item.emergency_energy_kwh:.6f}",
                f"{item.plan_cost_yuan:.6f}",
                f"{item.settled_purchase_cost_yuan:.6f}",
                f"{item.emergency_cost_yuan:.6f}",
                f"{item.total_cost_yuan:.6f}",
                f"{saving:.6f}",
                f"{saving / baseline_total:.8%}",
            ]
        )
)

```

```

def strategy_period_metrics(
    summary: StrategySummary,
    split: two.DatasetSplit,
    label: str,
) -> dict[str, float | int | str]:
    groups = split.cycle_labels([item.day for item in summary.results])
    selected_indices = set(groups[label])
    selected = [item for index, item in enumerate(summary.results) if index in selected_indices]
    if not selected:
        raise ValueError(f"{summary.name} 的 {label} 为空。")
    plan_energy = float(sum(np.sum(item.plan_grid_kwh) for item in selected))
    final_energy = float(sum(np.sum(item.final_grid_kwh) for item in selected))
    increase_energy = float(
        sum(
            np.sum(np.maximum(0.0, item.final_grid_kwh - item.plan_grid_kwh))
            for item in selected
        )
    )
    decrease_energy = float(
        sum(
            np.sum(np.maximum(0.0, item.plan_grid_kwh - item.final_grid_kwh))
            for item in selected
        )
    )
    emergency_energy = float(sum(np.sum(item.emergency_kwh) for item in selected))
    plan_cost = float(sum(item.plan_cost_yuan for item in selected))
    purchase_cost = float(sum(item.settled_purchase_cost_yuan for item in selected))
    emergency_cost = float(sum(item.emergency_cost_yuan for item in selected))
    purchased = final_energy + emergency_energy
    return {
        "开始日期": selected[0].day.isoformat(),
        "结束日期": selected[-1].day.isoformat(),
        "天数": len(selected),
        "初始计划购电量(kWh)": plan_energy,
        "最终购电量(kWh)": final_energy,
        "增购量(kWh)": increase_energy,
        "退购量(kWh)": decrease_energy,
        "紧急购电量(kWh)": emergency_energy,
        "紧急购电占比": 0.0 if purchased <= 0.0 else emergency_energy / purchased,
        "初始计划购电费(元)": plan_cost,
        "调整后购电费(元)": purchase_cost,
        "紧急购电费(元)": emergency_cost,
        "总购电费(元)": purchase_cost + emergency_cost,
    }

def select_strategy_on_validation(
    summaries: list[StrategySummary], split: two.DatasetSplit
) -> StrategySummary:
    return min(
        summaries,
        key=lambda item: float(
            strategy_period_metrics(item, split, "验证集")["总购电费(元)"]
        ),
    )

def write_split_evaluation(
    path: Path,
    summaries: list[StrategySummary],
    selected: StrategySummary,
    split: two.DatasetSplit,
) -> None:
    rows: list[dict[str, object]] = []
    for label in ("训练集", "验证集", "测试集"):
        for summary in summaries:
            row: dict[str, object] = {
                "数据集": label,
                "策略": summary.name,
                "验证集选定策略": "是" if summary.name == selected.name else "否",
            }
            row.update(strategy_period_metrics(summary, split, label))
            rows.append(row)
    path.parent.mkdir(parents=True, exist_ok=True)
    with path.open("w", encoding="utf-8-sig", newline="") as file:
        writer = csv.DictWriter(file, fieldnames=list(rows[0].keys()), lineterminator="\n")
        writer.writeheader()
        writer.writerows(rows)

```

```

validation_cost = strategy_period_metrics(selected, split, "验证集")["总购电费(元)"]
test_cost = strategy_period_metrics(selected, split, "测试集")["总购电费(元)"]
print(f"\n 问题三训练/验证/测试检验")
print(f" 验证集选定策略: {selected.name}, 验证集费用={validation_cost:.4f} 元")
print(f" 冻结策略后的测试集费用: {test_cost:.4f} 元")
print(f"数据集检验报告: {path.resolve()}")

def print_summary(
    summaries: list[StrategySummary], selected: StrategySummary, output: Path
) -> None:
    print(f"\n 问题三不同预报时刻组合对比 (2025-02-01 至 2025-12-31) ")
    previous: StrategySummary | None = None
    for item in summaries:
        marginal = 0.0 if previous is None else previous.total_cost_yuan - item.total_cost_yuan
        print(
            f" {item.name}: 总费用={item.total_cost_yuan:.4f} 元, "
            f"紧急购电={item.emergency_energy_kwh:.4f} kWh, "
            f"相对上一策略边际节省={marginal:.4f} 元"
        )
        previous = item
    print(f" 最低费用策略: {selected.name}")

specified = {date(2025, 3, 20), date(2025, 6, 21), date(2025, 9, 23), date(2025, 12, 21)}
print(f"\n 正式采用的{selected.name}策略指定日期紧急购电")
for item in selected.results:
    if item.day not in specified:
        continue
    groups = two.group_emergency(item.emergency_kwh, 1.0e-5)
    if not groups:
        print(f" {item.day.isoformat()}: 无")
    else:
        print(f" {item.day.isoformat()}: ")
        for period, amount in groups:
            print(f" {period}: {amount:.4f} kWh")
print(f"\n 正式结果: {output.resolve()}")

def parse_args() -> argparse.Namespace:
    parser = argparse.ArgumentParser(description="C 问题三 MPC-LP 滚动购电策略")
    parser.add_argument("--baseline", type=Path, default=two.DEFAULT_BASELINE)
    parser.add_argument("--load", type=Path, default=two.DEFAULT_LOAD)
    parser.add_argument("--pv", type=Path, default=two.DEFAULT_PV)
    parser.add_argument("--forecast", type=Path, default=DEFAULT_FORECAST)
    parser.add_argument("--template", type=Path, default=DEFAULT_TEMPLATE)
    parser.add_argument("--output", type=Path, default=DEFAULT_OUTPUT)
    parser.add_argument("--summary", type=Path, default=DEFAULT_SUMMARY)
    parser.add_argument("--comparison", type=Path, default=DEFAULT_COMPARISON)
    parser.add_argument("--split-report", type=Path, default=DEFAULT_SPLIT_REPORT)
    parser.add_argument("--train-end", type=date.fromisoformat, default=date(2025, 7, 31))
    parser.add_argument(
        "--validation-end", type=date.fromisoformat, default=date(2025, 9, 30)
    )
    return parser.parse_args()

def main() -> None:
    args = parse_args()
    cfg = Config()
    baseline, chronological_order = two.read_baseline(args.baseline)
    historical = two.read_historical_data(args.load, args.pv, chronological_order)
    forecasts = read_forecasts(args.forecast)
    split = two.DatasetSplit(args.train_end, args.validation_end)
    groups = split.cycle_labels(historical.dates)
    training_indices = set(groups["训练集"])
    print("正在构造四个发布时间的因果预测与历史误差安全裕度.....")
    decision_inputs = prepare_decision_inputs(
        baseline,
        historical,
        forecasts,
        cfg,
        training_indices=training_indices,
    )

    strategy_specs = [
        ("S0_仅0点", (0,)),
        ("S1_增加6点", (0, 6)),
        ("S2_增加12点", (0, 6, 12)),
        ("S3_完整MPC", (0, 6, 12, 18)),
    ]
    summaries: list[StrategySummary] = []
    for name, release_hours in strategy_specs:

```

```

print(f"\n 开始计算 {name}.....")
summary = simulate_strategy(
    name,
    release_hours,
    baseline,
    historical,
    decision_inputs,
    cfg,
)
validate_strategy(summary, cfg)
summaries.append(summary)

selected_strategy = select_strategy_on_validation(summaries, split)
write_result_workbook(
    args.template,
    args.output,
    selected_strategy,
    baseline,
    chronological_order,
    cfg,
)
write_daily_summary(args.summary, selected_strategy)
write_strategy_comparison(args.comparison, summaries)
write_split_evaluation(args.split_report, summaries, selected_strategy, split)
print_summary(summaries, selected_strategy, args.output)

if __name__ == "__main__":
    main()

```

C.4 问题四程序 four.py

程序说明：对动态电价进行严格前向预测，将电价预测分别嵌入问题二滚动线性规划与问题三 MPC-LP，采用实际电价统一结算，并输出 result4-2.xlsx 与 result4-3.xlsx。

```

# 电价预测链：
# 1. 严格前向贝叶斯优化历史窗口、权重及分解阶数，并采用早停；
# 2. 对加权电价轮廓作傅里叶拟合，提取日周期；
# 3. 对拟合残差作 Haar 小波软阈值重构，恢复局部尖峰与突变；
# 4. 使用前一日同一时刻的预测偏差滚动修正，并在线估计修正系数；
# 5. 对最终预测作非负截断。

# 问题 4-2 沿用问题二的 48 小时滚动线性规划；问题 4-3 沿用问题三的 MPC-LP，
# 分别生成官方模板要求的 result4-2.xlsx 和 result4-3.xlsx。
# 问题 4-3 策略只按验证集费用选择，测试集不参与调参或策略选择。

# 依赖：two.py、three.py、numpy、scipy、openpyxl。
# 正式求解：python four.py
# 贝叶斯寻优：python four.py --bayes-optimize
# 无 Hampel 消融：python four.py --bayes-optimize --bayes-disable-hampel

```

```

from __future__ import annotations

import argparse
import csv
import json
import math
import os
import shutil
from copy import copy
from dataclasses import asdict, dataclass, replace
from datetime import date, datetime, timedelta
from pathlib import Path

import numpy as np
from openpyxl import load_workbook

import three
import two

ROOT = Path(__file__).resolve().parent
DEFAULT_PRICE = ROOT / "C 题" / "附件" / "附件 4.xlsx"
DEFAULT_FORECAST = ROOT / "C 题" / "附件" / "csv" / "附件 3.csv"
DEFAULT_TEMPLATE_42 = ROOT / "C 题" / "附件" / "附件 5" / "result4-2.xlsx"
DEFAULT_TEMPLATE_43 = ROOT / "C 题" / "附件" / "附件 5" / "result4-3.xlsx"
DEFAULT_OUTPUT_42 = ROOT / "C 题" / "result4-2.xlsx"

```

```

DEFAULT_OUTPUT_43 = ROOT / "C 题" / "result4-3.xlsx"
DEFAULT_SUMMARY_42 = ROOT / "C 题" / "问题四_对应问题二逐日汇总.csv"
DEFAULT_SUMMARY_43 = ROOT / "C 题" / "问题四_对应问题三逐日汇总.csv"
DEFAULT_COMPARISON = ROOT / "C 题" / "问题四_MPC 策略对比.csv"
DEFAULT_PRICE_EVALUATION = ROOT / "C 题" / "问题四_电价预测评估.csv"
DEFAULT_SPLIT_REPORT = ROOT / "C 题" / "问题四_训练验证测试检验.csv"
DEFAULT_MODEL = ROOT / "C 题" / "问题四_动态电价数学模型.md"
DEFAULT_PRICE_BAYES_REPORT = ROOT / "C 题" / "贝叶斯优化_Hampel 电价参数与评价_严格前向.json"
DEFAULT_PRICE_BAYES_CSV = ROOT / "C 题" / "贝叶斯优化_Hampel 电价逐日评价_严格前向.csv"
DEFAULT_PRICE_BAYES_NO_HAMPEL_REPORT = ROOT / "C 题" / "贝叶斯优化_无 Hampel 电价参数与评价_严格前向.json"
DEFAULT_PRICE_BAYES_NO_HAMPEL_CSV = ROOT / "C 题" / "贝叶斯优化_无 Hampel 电价逐日评价_严格前向.csv"

```

```

@dataclass(frozen=True)
class Config(three.Config):
    price_history_days: int = 28
    price_half_life_days: float = 7.0
    price_same_weekday_multiplier: float = 2.0
    price_fourier_harmonics: int = 6
    price_wavelet_level: int = 3
    price_bias_days: int = 28
    minimum_bias_pairs: int = 7
    default_bias_weight: float = 0.60
    use_bayesian_price_schedule: bool = True

_PRICE_PARAMETER_SCHEDULE = (
    (
        date(2025, 2, 28),
        dict(
            price_history_days=40,
            price_half_life_days=28.260560329514153,
            price_same_weekday_multiplier=3.335679870659064,
            price_fourier_harmonics=5,
            price_wavelet_level=2,
            price_bias_days=32,
            default_bias_weight=0.6296739771158961,
        ),
    ),
    (
        date(2025, 3, 6),
        dict(
            price_history_days=31,
            price_half_life_days=24.43488696871456,
            price_same_weekday_multiplier=3.46299237674132,
            price_fourier_harmonics=5,
            price_wavelet_level=2,
            price_bias_days=47,
            default_bias_weight=0.6456718349291368,
        ),
    ),
    (
        date(2025, 3, 12),
        dict(
            price_history_days=33,
            price_half_life_days=25.145760809373442,
            price_same_weekday_multiplier=4.009445223594914,
            price_fourier_harmonics=3,
            price_wavelet_level=2,
            price_bias_days=38,
            default_bias_weight=0.44690879842079867,
        ),
    ),
    (
        date(2025, 4, 5),
        dict(
            price_history_days=22,
            price_half_life_days=27.116107451115006,
            price_same_weekday_multiplier=4.444737582066487,
            price_fourier_harmonics=2,
            price_wavelet_level=2,
            price_bias_days=51,
            default_bias_weight=0.6237624262745828,
        ),
    ),
)

```

```

def bayesian_price_config(cfg: Config, issue_date: date) -> Config:
    if not cfg.use_bayesian_price_schedule:
        return cfg

```

```

        selected = cfg
        for effective_date, parameters in _PRICE_PARAMETER_SCHEDULE:
            if issue_date < effective_date:
                break
            selected = replace(cfg, **parameters)
        return selected

@dataclass
class PriceData:
    dates: list[date]
    price: np.ndarray

@dataclass
class PriceForecast:
    value: np.ndarray
    periodic: np.ndarray
    wavelet_residual: np.ndarray
    previous_day_bias: np.ndarray
    bias_weight: float

@dataclass
class Daily42:
    day: date
    forecast_price: np.ndarray
    grid_kwh: np.ndarray
    charge_kwh: np.ndarray
    discharge_kwh: np.ndarray
    emergency_kwh: np.ndarray
    soc_start_kwh: float
    soc_end_kwh: float
    plan_cost_yuan: float
    emergency_cost_yuan: float

    @property
    def total_cost_yuan(self) -> float:
        return self.plan_cost_yuan + self.emergency_cost_yuan

def read_dynamic_prices(path: Path, chronological_order: list[int]) -> PriceData:
    if not path.exists():
        raise FileNotFoundError(f"找不到附件 4: {path}")
    workbook = load_workbook(path, read_only=True, data_only=True)
    worksheet = workbook.active
    if worksheet.max_row != 366 or worksheet.max_column != 145:
        raise ValueError("附件 4 应包含 1 行表头、365 天和每天 144 个实时电价。")

    dates: list[date] = []
    rows: list[list[float]] = []
    order = np.asarray(chronological_order, dtype=int)
    for row_number, row in enumerate(
        worksheet.iter_rows(min_row=2, max_row=366, values_only=True), start=2
    ):
        raw_date = row[0]
        if isinstance(raw_date, datetime):
            current_date = raw_date.date()
        elif isinstance(raw_date, date):
            current_date = raw_date
        else:
            current_date = datetime.fromisoformat(str(raw_date)).date()
        values = np.asarray(
            [two._float(value, f"附件 4 第 {row_number} 行电价") for value in row[1:]],
            dtype=float,
        )[order]
        dates.append(current_date)
        rows.append(values.tolist())
    workbook.close()

    expected = [date(2025, 1, 1) + timedelta(days=i) for i in range(365)]
    if dates != expected:
        raise ValueError("附件 4 日期应连续覆盖 2025-01-01 至 2025-12-31。")
    matrix = np.asarray(rows, dtype=float)
    if not np.all(np.isfinite(matrix)) or np.any(matrix < 0):
        raise ValueError("附件 4 包含负值或非有限电价。")
    return PriceData(dates, matrix)

def _price_fourier_design(hours: np.ndarray, harmonics: int) -> np.ndarray:
    omega = 2.0 * np.pi / 24.0

```

```

columns = [np.ones_like(hours)]
for order in range(1, harmonics + 1):
    columns.extend(
        [np.cos(order * omega * hours), np.sin(order * omega * hours)]
    )
return np.column_stack(columns)

def fit_price_fourier(
    hours: np.ndarray, profile: np.ndarray, harmonics: int
) -> np.ndarray:
    design = _price_fourier_design(hours, harmonics)
    coefficients, *_ = np.linalg.lstsq(design, profile, rcond=None)
    return design @ coefficients

def _price_history_profile(
    prices: PriceData,
    cutoff: int,
    target_date: date,
    baseline_price: np.ndarray,
    cfg: Config,
    allowed_indices: set[int] | None = None,
) -> np.ndarray:
    if cutoff <= 0:
        return baseline_price.copy()
    first = max(0, cutoff - cfg.price_history_days)
    indices = np.arange(first, cutoff, dtype=int)
    if allowed_indices is not None:
        indices = np.asarray([i for i in indices if i in allowed_indices], dtype=int)
    if indices.size == 0:
        return baseline_price.copy()
    ages = cutoff - 1 - indices
    weights = np.exp(-math.log(2.0) * ages / cfg.price_half_life_days)
    same_weekday = np.asarray(
        [prices.dates[index].weekday() == target_date.weekday() for index in indices],
        dtype=float,
    )
    weights *= 1.0 + cfg.price_same_weekday_multiplier * same_weekday
    return np.average(prices.price[indices], axis=0, weights=weights)

def _causal_price_bases(
    prices: PriceData,
    baseline_price: np.ndarray,
    cfg: Config,
    training_cutoff: int | None = None,
    training_indices: set[int] | None = None,
) -> tuple[list[np.ndarray], list[np.ndarray], list[np.ndarray]]:
    hours = np.arange(144, dtype=float) * cfg.dt_hours
    bases: list[np.ndarray] = []
    periodic_parts: list[np.ndarray] = []
    wavelet_parts: list[np.ndarray] = []
    for cutoff, current_date in enumerate(prices.dates):
        model_cutoff = (
            cutoff
            if training_cutoff is None
            else min(cutoff, training_cutoff)
        )
        allowed_indices = training_indices
        profile = _price_history_profile(
            prices, model_cutoff, current_date, baseline_price, cfg, allowed_indices
        )
        periodic = fit_price_fourier(
            hours, profile, cfg.price_fourier_harmonics
        )
        wavelet = two.wavelet_correct_residual(
            profile, periodic, cfg.price_wavelet_level
        )
        bases.append(periodic + wavelet)
        periodic_parts.append(periodic)
        wavelet_parts.append(wavelet)
    return bases, periodic_parts, wavelet_parts

def _estimate_bias_weight(
    prices: PriceData,
    causal_bases: list[np.ndarray],
    cutoff: int,
    cfg: Config,

```

```

    allowed_indices: set[int] | None = None,
) -> float:
    pair_end = cutoff - 1
    pair_start = max(1, pair_end - cfg.price_bias_days + 1)
    if pair_end - pair_start + 1 < cfg.minimum_bias_pairs:
        return cfg.default_bias_weight
    previous: list[np.ndarray] = []
    current: list[np.ndarray] = []
    for index in range(pair_start, pair_end + 1):
        if allowed_indices is not None and (index not in allowed_indices or index - 1 not in allowed_indices):
            continue
        previous.append(prices.price[index - 1] - causal_bases[index - 1])
        current.append(prices.price[index] - causal_bases[index])
    if len(previous) < cfg.minimum_bias_pairs:
        return cfg.default_bias_weight
    x = np.concatenate(previous)
    y = np.concatenate(current)
    denominator = float(np.dot(x, x))
    if denominator <= 1.0e-12:
        return cfg.default_bias_weight
    return float(np.clip(np.dot(x, y) / denominator, 0.0, 1.0))

def make_price_forecast(
    prices: PriceData,
    baseline_price: np.ndarray,
    causal_bases: list[np.ndarray],
    cutoff: int,
    target_date: date,
    cfg: Config,
    training_cutoff: int | None = None,
    training_indices: set[int] | None = None,
) -> PriceForecast:
    hours = np.arange(144, dtype=float) * cfg.dt_hours
    model_cutoff = (
        cutoff
        if training_cutoff is None
        else min(cutoff, training_cutoff)
    )
    profile = _price_history_profile(
        prices, model_cutoff, target_date, baseline_price, cfg, training_indices
    )
    periodic = fit_price_fourier(hours, profile, cfg.price_fourier_harmonics)
    wavelet = two.wavelet_correct_residual(
        profile, periodic, cfg.price_wavelet_level
    )
    latest_observed = cutoff - 1
    if latest_observed >= 0 and (training_indices is None or latest_observed in training_indices):
        bias = prices.price[latest_observed] - causal_bases[latest_observed]
        # 偏差修正系数只在训练集估计; 前一日偏差仍是预测时可得信息。
        weight = _estimate_bias_weight(
            prices, causal_bases, model_cutoff, cfg, training_indices
        )
    else:
        bias = np.zeros(144, dtype=float)
        weight = 0.0
    value = np.maximum(0.0, periodic + wavelet + weight * bias)
    return PriceForecast(value, periodic, wavelet, bias, weight)

def prepare_daily_price_forecasts(
    prices: PriceData,
    baseline_price: np.ndarray,
    cfg: Config,
    training_cutoff: int | None = None,
    training_indices: set[int] | None = None,
) -> tuple[list[PriceForecast], list[np.ndarray]]:
    daily_configs = [bayesian_price_config(cfg, day) for day in prices.dates]
    bases_cache: dict[Config, list[np.ndarray]] = {}
    for forecast_cfg in dict.fromkeys(daily_configs):
        bases_cache[forecast_cfg], _, _ = _causal_price_bases(
            prices,
            baseline_price,
            forecast_cfg,
            training_cutoff=training_cutoff,
            training_indices=training_indices,
        )
    current: list[PriceForecast] = []
    following: list[np.ndarray] = []
    for cutoff, (current_date, forecast_cfg) in enumerate(

```

```

        zip(prices.dates, daily_configs)
    ):
        causal_bases = bases_cache[forecast_cfg]
        current.append(
            make_price_forecast(
                prices,
                baseline_price,
                causal_bases,
                cutoff,
                current_date,
                forecast_cfg,
                training_cutoff=training_cutoff,
                training_indices=training_indices,
            )
        )
        following.append(
            make_price_forecast(
                prices,
                baseline_price,
                causal_bases,
                cutoff,
                current_date + timedelta(days=1),
                forecast_cfg,
                training_cutoff=training_cutoff,
                training_indices=training_indices,
            ).value
        )
    return current, following

def simulate_42(
    baseline: two.BaselineData,
    historical: two.HistoricalData,
    prices: PriceData,
    current_prices: list[PriceForecast],
    following_prices: list[np.ndarray],
    cfg: Config,
    training_indices: set[int],
) -> list[Daily42]:
    hours = np.arange(144, dtype=float) * cfg.dt_hours
    error_history: list[np.ndarray] = []
    results: list[Daily42] = []
    soc = cfg.initial_soc_kwh

    for day_index, current_date in enumerate(historical.dates):
        allowed_indices = training_indices
        forecast_cfg = two.bayesian_load_config(cfg, current_date)
        current_forecast = two.make_forecast(
            historical, baseline, day_index, current_date, hours, forecast_cfg,
            allowed_indices=allowed_indices,
            pv_cfg=cfg,
        )
        next_forecast = two.make_forecast(
            historical,
            baseline,
            day_index,
            current_date + timedelta(days=1),
            hours,
            forecast_cfg,
            allowed_indices=allowed_indices,
            pv_cfg=cfg,
        )
        reserve = two.risk_reserve(error_history, cfg)
        price_horizon = np.concatenate(
            [current_prices[day_index].value, following_prices[day_index]]
        )
        solution = two.solve_rolling_lp(
            price_horizon,
            np.concatenate(
                [
                    current_forecast.load_kw + reserve / cfg.dt_hours,
                    next_forecast.load_kw + reserve / cfg.dt_hours,
                ]
            ),
            np.concatenate([current_forecast.pv_kw, next_forecast.pv_kw]),
            soc,
            cfg,
        )
        grid = solution.grid_kwh[:144]
        charge = solution.charge_kwh[:144]
        discharge = solution.discharge_kwh[:144]

```

```

emergency, _ = two.replay_actual_day(
    historical.load_kw[day_index],
    historical.pv_kw[day_index],
    grid,
    charge,
    discharge,
    cfg,
)
soc_start = soc
soc = float(solution.soc_kwh[144])
actual_price = prices.price[day_index]
plan_cost = float(np.dot(actual_price, grid))
emergency_cost = float(
    np.dot(cfg.emergency_price_multiplier * actual_price, emergency)
)
results.append(
    Daily42(
        current_date,
        current_prices[day_index].value,
        grid,
        charge,
        discharge,
        emergency,
        soc_start,
        soc,
        plan_cost,
        emergency_cost,
    )
)
actual_net = (
    historical.load_kw[day_index] - historical.pv_kw[day_index]
) * cfg.dt_hours
forecast_net = (
    current_forecast.load_kw - current_forecast.pv_kw
) * cfg.dt_hours
if day_index in training_indices:
    error_history.append(actual_net - forecast_net)
if (day_index + 1) % 60 == 0 or day_index == 364:
    print(f"问题 4-2 已完成 {current_date}, SOC={soc:.2f} kWh")
return results

def price_horizon(
    day_index: int,
    release_hour: int,
    current_prices: list[PriceForecast],
    following_prices: list[np.ndarray],
) -> np.ndarray:
    start = release_hour * 6
    if start == 0:
        return current_prices[day_index].value.copy()
    return np.concatenate(
        [current_prices[day_index].value[start:], following_prices[day_index][:start]]
    )

def simulate_43_strategy(
    name: str,
    release_hours: tuple[int, ...],
    historical: two.HistoricalData,
    prices: PriceData,
    decision_inputs: dict[tuple[int, int], three.DecisionInput],
    current_prices: list[PriceForecast],
    following_prices: list[np.ndarray],
    cfg: Config,
) -> three.StrategySummary:
    results: list[three.DailyResult] = []
    soc = cfg.initial_soc_kwh
    for day_index, current_date in enumerate(historical.dates):
        soc_start = soc
        decision_zero = decision_inputs[(day_index, 0)]
        plan_solution = two.solve_rolling_lp(
            price_horizon(
                day_index, 0, current_prices, following_prices
            ),
            decision_zero.load_kw + decision_zero.reserve_kwh / cfg.dt_hours,
            decision_zero.pv_kw,
            soc,
            cfg,
        )
        plan_grid = plan_solution.grid_kwh.copy()

```

```

final_grid = plan_solution.grid_kwh.copy()
final_charge = plan_solution.charge_kwh.copy()
final_discharge = plan_solution.discharge_kwh.copy()

for release_hour in (0, 6, 12, 18):
    start = release_hour * 6
    if release_hour > 0 and release_hour in release_hours:
        decision = decision_inputs[(day_index, release_hour)]
        adjusted = three.solve_adjustment_lp(
            price_horizon(
                day_index,
                release_hour,
                current_prices,
                following_prices,
            ),
            decision.load_kw + decision.reserve_kwh / cfg.dt_hours,
            decision.pv_kw,
            soc,
            plan_grid[start:],
            144 - start,
            cfg,
        )
        final_grid[start:] = adjusted.grid_kwh[: 144 - start]
        final_charge[start:] = adjusted.charge_kwh[: 144 - start]
        final_discharge[start:] = adjusted.discharge_kwh[: 144 - start]
    stop = min(start + 36, 144)
    soc = three.execute_soc(
        soc,
        final_charge[start:stop],
        final_discharge[start:stop],
        cfg,
    )

emergency, curtailment = two.replay_actual_day(
    historical.load_kw[day_index],
    historical.pv_kw[day_index],
    final_grid,
    final_charge,
    final_discharge,
    cfg,
)

actual_price = prices.price[day_index]
plan_cost = float(np.dot(actual_price, plan_grid))
settled_cost = three.settled_purchase_cost(
    actual_price, plan_grid, final_grid, cfg
)

emergency_cost = float(
    np.dot(cfg.emergency_price_multiplier * actual_price, emergency)
)

results.append(
    three.DailyResult(
        current_date,
        plan_grid,
        final_grid,
        final_charge,
        final_discharge,
        emergency,
        curtailment,
        soc_start,
        soc,
        plan_cost,
        settled_cost,
        emergency_cost,
    )
)

if (day_index + 1) % 120 == 0 or day_index == 364:
    print(f"{name}已完成{current_date}, SOC={soc:.2f} kWh")

selected = [item for item in results if item.day >= date(2025, 2, 1)]
plan_energy = float(sum(np.sum(item.plan_grid_kwh) for item in selected))
final_energy = float(sum(np.sum(item.final_grid_kwh) for item in selected))
increase_energy = float(
    sum(
        np.sum(np.maximum(0.0, item.final_grid_kwh - item.plan_grid_kwh))
        for item in selected
    )
)

decrease_energy = float(
    sum(
        np.sum(np.maximum(0.0, item.plan_grid_kwh - item.final_grid_kwh))

```

```

        for item in selected
    )
)
emergency_energy = float(sum(np.sum(item.emergency_kwh for item in selected))
plan_cost = float(sum(item.plan_cost_yuan for item in selected))
settled_cost = float(sum(item.settled_purchase_cost_yuan for item in selected))
emergency_cost = float(sum(item.emergency_cost_yuan for item in selected))
return three.StrategySummary(
    name,
    release_hours,
    results,
    plan_energy,
    final_energy,
    increase_energy,
    decrease_energy,
    emergency_energy,
    plan_cost,
    settled_cost,
    emergency_cost,
    settled_cost + emergency_cost,
)

def validate_42(results: list[Daily42], cfg: Config) -> None:
    if len(results) != 365:
        raise RuntimeError("问题 4-2 没有生成 365 天结果。")
    for index, item in enumerate(results):
        if index and abs(item.soc_start_kwh - results[index - 1].soc_end_kwh) > 1e-4:
            raise RuntimeError(f"问题 4-2 在 {item.day} 的跨日 SOC 不连续。")
        if np.any(item.grid_kwh < -1e-6) or np.any(item.emergency_kwh < -1e-6):
            raise RuntimeError(f"问题 4-2 在 {item.day} 出现负购电量。")
        if not cfg.soc_min_kwh - 1e-4 <= item.soc_end_kwh <= cfg.soc_max_kwh + 1e-4:
            raise RuntimeError(f"问题 4-2 在 {item.day} 的 SOC 越界。")

def _storage_rows(results: list, q43: bool = False) -> list[list[object]]:
    rows: list[list[object]] = []
    periods = [
        "0:00-4:00",
        "4:00-8:00",
        "8:00-12:00",
        "12:00-16:00",
        "16:00-20:00",
        "20:00-24:00",
    ]
    for item in results:
        charge = item.charge_kwh
        discharge = item.discharge_kwh
        for block, period in enumerate(periods):
            start, stop = block * 24, (block + 1) * 24
            rows.append(
                [
                    item.day.isoformat() if block == 0 else None,
                    period,
                    round(float(np.sum(charge[start:stop])), 4),
                    round(float(np.sum(discharge[start:stop])), 4),
                    "0:00" if block == 0 else ("24:00" if block == 1 else None),
                    round(
                        float(item.soc_start_kwh if block == 0 else item.soc_end_kwh),
                        4,
                    )
                    if block in (0, 1)
                    else None,
                ]
            )
    return rows

def _emergency_rows(results: list, tolerance: float) -> list[list[object]]:
    rows: list[list[object]] = []
    for item in results:
        groups = two.group_emergency(item.emergency_kwh, tolerance)
        for group_index, (period, amount) in enumerate(groups):
            rows.append(
                [
                    item.day.isoformat() if group_index == 0 else None,
                    period,
                    round(amount, 4),
                ]
            )
    return rows

```

```

return rows or [["无紧急购电", None, 0.0]]

def make_workbook_payload(
    results42: list[Daily42],
    selected43: three.StrategySummary,
    chronological_order: list[int],
    cfg: Config,
) -> dict[str, object]:
    selected42 = [item for item in results42 if item.day >= date(2025, 2, 1)]
    selected43_results = [
        item for item in selected43.results if item.day >= date(2025, 2, 1)
    ]
    plan42 = [
        [
            round(float(value), 4)
            for value in two._to_template_order(
                item.grid_kwh, chronological_order
            )
        ]
        + [round(float(np.sum(item.grid_kwh)), 4), round(item.plan_cost_yuan, 4)]
        for item in selected42
    ]
    plan43 = [
        [
            round(float(value), 4)
            for value in two._to_template_order(
                item.plan_grid_kwh, chronological_order
            )
        ]
        + [
            round(float(np.sum(item.plan_grid_kwh)), 4),
            round(item.plan_cost_yuan, 4),
        ]
        for item in selected43_results
    ]
    adjust43 = [
        [
            round(float(value), 4)
            for value in two._to_template_order(
                item.final_grid_kwh, chronological_order
            )
        ]
        + [
            round(float(np.sum(item.final_grid_kwh)), 4),
            round(item.settled_purchase_cost_yuan, 4),
        ]
        for item in selected43_results
    ]
    return {
        "q42": {
            "plan": plan42,
            "storage": _storage_rows(selected42),
            "emergency": _emergency_rows(
                selected42, cfg.emergency_tolerance_kwh
            ),
        },
        "q43": {
            "plan": plan43,
            "adjust": adjust43,
            "storage": _storage_rows(selected43_results, q43=True),
            "emergency": _emergency_rows(
                selected43_results, cfg.emergency_tolerance_kwh
            ),
        },
    }

def write_csv_outputs(
    results42: list[Daily42],
    summaries43: list[three.StrategySummary],
    selected43: three.StrategySummary,
    prices: PriceData,
    price_forecasts: list[PriceForecast],
    args: argparse.Namespace,
) -> None:
    with args.summary42.open("w", encoding="utf-8-sig", newline="") as file:
        writer = csv.writer(file, lineterminator="\n")
        writer.writerow(
            [
                "日期",

```

```

        "计划购电量(kWh)",
        "计划购电费(元)",
        "紧急购电量(kWh)",
        "紧急购电费(元)",
        "总购电费(元)",
        "0:00 储电量(kWh)",
        "24:00 储电量(kWh)",
        "电价预测 MAE(元/kWh)",
    ]
)
for index, item in enumerate(results42):
    writer.writerow(
        [
            item.day.isoformat(),
            f"{np.sum(item.grid_kwh):.6f}",
            f"{item.plan_cost_yuan:.6f}",
            f"{np.sum(item.emergency_kwh):.6f}",
            f"{item.emergency_cost_yuan:.6f}",
            f"{item.total_cost_yuan:.6f}",
            f"{item.soc_start_kwh:.6f}",
            f"{item.soc_end_kwh:.6f}",
            f"{np.mean(np.abs(prices.price[index] - item.forecast_price)):.8f}",
        ]
    )
three.write_daily_summary(args.summary43, selected43)
three.write_strategy_comparison(args.comparison, summaries43)
with args.price_evaluation.open("w", encoding="utf-8-sig", newline="") as file:
    writer = csv.writer(file, lineterminator="\n")
    writer.writerow(
        [
            "日期",
            "MAE(元/kWh)",
            "RMSE(元/kWh)",
            "MAPE",
            "前日偏差权重",
            "实际均价(元/kWh)",
            "预测均价(元/kWh)",
        ]
    )
)
for index, forecast in enumerate(price_forecasts):
    actual = prices.price[index]
    error = actual - forecast.value
    writer.writerow(
        [
            prices.dates[index].isoformat(),
            f"{np.mean(np.abs(error)):.8f}",
            f"{np.sqrt(np.mean(error**2)):.8f}",
            f"{np.mean(np.abs(error) / np.maximum(actual, 0.05)):.8%}",
            f"{forecast.bias_weight:.8f}",
            f"{np.mean(actual):.8f}",
            f"{np.mean(forecast.value):.8f}",
        ]
    )
)

def _price_period_metrics(
    prices: PriceData,
    price_forecasts: list[PriceForecast],
    indices: list[int],
) -> dict[str, float]:
    actual = prices.price[indices]
    forecast = np.asarray([price_forecasts[index].value for index in indices])
    error = actual - forecast
    return {
        "电价 MAE(元/kWh)": float(np.mean(np.abs(error))),
        "电价 RMSE(元/kWh)": float(np.sqrt(np.mean(error**2))),
        "电价修正 MAPE": float(
            np.mean(np.abs(error) / np.maximum(actual, 0.05))
        ),
    }

def _q42_period_metrics(results: list[Daily42]) -> dict[str, float]:
    plan_energy = float(sum(np.sum(item.grid_kwh) for item in results))
    emergency_energy = float(sum(np.sum(item.emergency_kwh) for item in results))
    plan_cost = float(sum(item.plan_cost_yuan for item in results))
    emergency_cost = float(sum(item.emergency_cost_yuan for item in results))
    purchased = plan_energy + emergency_energy
    return {
        "计划或最终购电量(kWh)": plan_energy,
    }

```

```

        "紧急购电量(kWh)": emergency_energy,
        "紧急购电占比": 0.0 if purchased <= 0.0 else emergency_energy / purchased,
        "常规或调整购电费(元)": plan_cost,
        "紧急购电费(元)": emergency_cost,
        "总购电费(元)": plan_cost + emergency_cost,
    }

def write_split_evaluation(
    path: Path,
    prices: PriceData,
    price_forecasts: list[PriceForecast],
    results42: list[Daily42],
    summaries43: list[three.StrategySummary],
    selected43: three.StrategySummary,
    split: two.DatasetSplit,
) -> None:
    rows: list[dict[str, object]] = []
    for label in ("训练集", "验证集", "测试集"):
        indices = split.cycle_labels(prices.dates)[label]
        if not indices:
            raise ValueError(f"问题四的{label}为空。")
        common: dict[str, object] = {
            "数据集": label,
            "开始日期": prices.dates[indices[0]].isoformat(),
            "结束日期": prices.dates[indices[-1]].isoformat(),
            "天数": len(indices),
        }
        price_metrics = _price_period_metrics(prices, price_forecasts, indices)
        price_row = {
            **common,
            "模型或策略": "动态电价预测",
            "验证集选定策略": "",
            **price_metrics,
        }
        rows.append(price_row)

        q42_selected = [results42[index] for index in indices]
        rows.append(
            {
                **common,
                "模型或策略": "问题 4-2_动态电价 LP",
                "验证集选定策略": "",
                **_q42_period_metrics(q42_selected),
            }
        )
    for summary in summaries43:
        metrics43 = three.strategy_period_metrics(summary, split, label)
        rows.append(
            {
                **common,
                "模型或策略": f"问题 4-3_{summary.name}",
                "验证集选定策略": "是" if summary.name == selected43.name else "否",
                "计划或最终购电量(kWh)": metrics43["最终购电量(kWh)"],
                "紧急购电量(kWh)": metrics43["紧急购电量(kWh)"],
                "紧急购电占比": metrics43["紧急购电占比"],
                "常规或调整购电费(元)": metrics43["调整后购电费(元)"],
                "紧急购电费(元)": metrics43["紧急购电费(元)"],
                "总购电费(元)": metrics43["总购电费(元)"],
            }
        )
    )

headers = [
    "数据集",
    "开始日期",
    "结束日期",
    "天数",
    "模型或策略",
    "验证集选定策略",
    "电价 MAE(元/kWh)",
    "电价 RMSE(元/kWh)",
    "电价修正 MAPE",
    "计划或最终购电量(kWh)",
    "紧急购电量(kWh)",
    "紧急购电占比",
    "常规或调整购电费(元)",
    "紧急购电费(元)",
    "总购电费(元)",
]

path.parent.mkdir(parents=True, exist_ok=True)
with path.open("w", encoding="utf-8-sig", newline="") as file:

```

```

writer = csv.DictWriter(file, fieldnames=headers, lineterminator="\n")
writer.writeheader()
writer.writerows(rows)

validation = three.strategy_period_metrics(selected43, split, "验证集")
test = three.strategy_period_metrics(selected43, split, "测试集")
print("\n问题四训练/验证/测试检验")
print(
    f" 验证集选定策略: {selected43.name}, "
    f"验证集费用={validation['总购电费(元)']:.4f} 元"
)
print(f" 冻结策略后的测试集费用: {test['总购电费(元)']:.4f} 元")
print(f"数据集检验报告: {path.resolve()}")

def write_workbooks(
    payload: dict[str, object], args: argparse.Namespace
) -> None:
    def copy_style(source, target) -> None:
        target._style = copy(source._style)
        target.number_format = source.number_format
        target.font = copy(source.font)
        target.fill = copy(source.fill)
        target.border = copy(source.border)
        target.alignment = copy(source.alignment)
        target.protection = copy(source.protection)

    def write_rows(sheet, rows, column_count: int, style_period: int) -> None:
        style_count = min(style_period, max(sheet.max_row - 1, 1))
        styles = [
            [copy(sheet.cell(row, column)) for column in range(1, column_count + 1)]
            for row in range(2, 2 + style_count)
        ]
        if sheet.max_row > 1:
            sheet.delete_rows(2, sheet.max_row - 1)
        for row_index, values in enumerate(rows, start=2):
            style_row = styles[(row_index - 2) % style_count]
            for column in range(1, column_count + 1):
                target = sheet.cell(row_index, column)
                copy_style(style_row[column - 1], target)
                value = values[column - 1]
                if column == 1 and isinstance(value, str):
                    try:
                        value = date.fromisoformat(value)
                    except ValueError:
                        pass
                target.value = value

    def build(
        template: Path,
        output: Path,
        section: dict[str, object],
        q43: bool,
    ) -> None:
        if not template.exists():
            raise FileNotFoundError(f"找不到结果模板: {template}")
        if template.resolve() == output.resolve():
            raise ValueError("输出路径不能覆盖官方模板。")
        output.parent.mkdir(parents=True, exist_ok=True)
        temporary = output.with_name(f"{output.stem}.building{output.suffix}")
        temporary.unlink(missing_ok=True)
        shutil.copy2(template, temporary)
        try:
            workbook = load_workbook(temporary)
            plan = workbook["计划购电量"]
            for row, values in enumerate(section["plan"], start=2):
                for column, value in enumerate(values, start=2):
                    plan.cell(row, column, value)
            if q43:
                adjust = workbook["调整购电量"]
                for row, values in enumerate(section["adjust"], start=2):
                    for column, value in enumerate(values, start=2):
                        adjust.cell(row, column, value)
            write_rows(
                workbook["充放电量"],
                section["storage"],
                6,
                24 if q43 else 18,
            )
            write_rows(workbook["紧急购电量"], section["emergency"], 3, 2)
            workbook.calculation.fullCalcOnLoad = True

```

```

        workbook.calculation.forceFullCalc = True
        workbook.save(temporary)
        try:
            os.replace(temporary, output)
        except PermissionError as exc:
            raise PermissionError(
                f"无法写入 {output}, 请关闭正在打开该文件的 Excel 窗口后重试。"
            ) from exc
    finally:
        temporary.unlink(missing_ok=True)

    build(args.template42, args.output42, payload["q42"], q43=False)
    build(args.template43, args.output43, payload["q43"], q43=True)

def print_summary(
    results42: list[Daily42],
    summaries43: list[three.StrategySummary],
    selected43: three.StrategySummary,
    prices: PriceData,
    price_forecasts: list[PriceForecast],
    args: argparse.Namespace,
) -> None:
    output42 = [item for item in results42 if item.day >= date(2025, 2, 1)]
    forecast_matrix = np.asarray(
        [item.value for item in price_forecasts[31:]], dtype=float
    )
    actual_matrix = prices.price[31:]
    mae = float(np.mean(np.abs(actual_matrix - forecast_matrix)))
    rmse = float(np.sqrt(np.mean((actual_matrix - forecast_matrix) ** 2)))
    total42 = float(sum(item.total_cost_yuan for item in output42))
    emergency42 = float(sum(np.sum(item.emergency_kwh for item in output42))
    print("\n 问题四动态电价预测 (2025-02-01 至 2025-12-31) ")
    print(f"   MAE={mae:.6f} 元/kWh, RMSE={rmse:.6f} 元/kWh")
    print(f" 问题 4-2 总费用={total42:.4f} 元, 紧急购电={emergency42:.4f} kWh")
    print("\n 问题 4-3 策略比较")
    for item in summaries43:
        print(
            f"   {item.name}: 总费用={item.total_cost_yuan:.4f} 元, "
            f"   紧急购电={item.emergency_energy_kwh:.4f} kWh"
        )
    print(f" 正式采用: {selected43.name}")
    print(f"\n 结果文件: {args.output42.resolve()}")
    print(f"结果文件: {args.output43.resolve()}")

@dataclass(frozen=True)
class PriceSearchPoint:
    history_days: int
    half_life_days: float
    same_weekday_multiplier: float
    fourier_harmonics: int
    wavelet_level: int
    bias_days: int
    default_bias_weight: float
    hamper_radius: int
    hamper_threshold: float
    hamper_strength: float

def _default_price_search_point() -> PriceSearchPoint:
    cfg = Config()
    return PriceSearchPoint(
        cfg.price_history_days,
        cfg.price_half_life_days,
        cfg.price_same_weekday_multiplier,
        cfg.price_fourier_harmonics,
        cfg.price_wavelet_level,
        cfg.price_bias_days,
        cfg.default_bias_weight,
        3,
        3.0,
        0.0,
    )

def _decode_price_point(
    unit: np.ndarray, disable_hampel: bool
) -> PriceSearchPoint:
    x = np.clip(np.asarray(unit, dtype=float), 0.0, 1.0)
    return PriceSearchPoint(
        history_days=int(round(14 + 46 * x[0])),

```

```

        half_life_days=float(3.0 + 27.0 * x[1]),
        same_weekday_multiplier=float(5.0 * x[2]),
        fourier_harmonics=int(round(2 + 8 * x[3])),
        wavelet_level=int(round(1 + 3 * x[4])),
        bias_days=int(round(7 + 53 * x[5])),
        default_bias_weight=float(x[6]),
        hampel_radius=3 if disable_hampel else int(round(2 + 8 * x[7])),
        hampel_threshold=3.0 if disable_hampel else float(2.0 + 3.0 * x[8]),
        hampel_strength=0.0 if disable_hampel else float(0.25 + 0.75 * x[9]),
    )

def _encode_price_point(
    point: PriceSearchPoint, disable_hampel: bool
) -> np.ndarray:
    encoded = np.asarray(
        [
            (point.history_days - 14) / 46,
            (point.half_life_days - 3.0) / 27.0,
            point.same_weekday_multiplier / 5.0,
            (point.fourier_harmonics - 2) / 8,
            (point.wavelet_level - 1) / 3,
            (point.bias_days - 7) / 53,
            point.default_bias_weight,
            (point.hampel_radius - 2) / 8,
            (point.hampel_threshold - 2.0) / 3.0,
            max(0.0, (point.hampel_strength - 0.25) / 0.75),
        ],
        dtype=float,
    )
    return encoded[:7] if disable_hampel else encoded

def _price_point_key(point: PriceSearchPoint) -> tuple[object, ...]:
    return (
        point.history_days,
        round(point.half_life_days, 6),
        round(point.same_weekday_multiplier, 6),
        point.fourier_harmonics,
        point.wavelet_level,
        point.bias_days,
        round(point.default_bias_weight, 6),
        point.hampel_radius,
        round(point.hampel_threshold, 6),
        round(point.hampel_strength, 6),
    )

def _hampel_filter_price_history(
    matrix: np.ndarray,
    radius: int,
    threshold: float,
    strength: float,
) -> tuple[np.ndarray, int]:
    if strength <= 0.0 or matrix.shape[0] < 3:
        return matrix.copy(), 0
    filtered = matrix.copy()
    replacements = 0
    for row in range(matrix.shape[0]):
        left = max(0, row - radius)
        right = min(matrix.shape[0], row + radius + 1)
        window = matrix[left:right]
        median = np.median(window, axis=0)
        mad = np.median(np.abs(window - median), axis=0)
        robust_sigma = 1.4826 * mad
        mask = (robust_sigma > 1.0e-12) & (
            np.abs(matrix[row] - median) > threshold * robust_sigma
        )
        if np.any(mask):
            filtered[row, mask] = (
                (1.0 - strength) * matrix[row, mask] + strength * median[mask]
            )
            replacements += int(np.count_nonzero(mask))
    return filtered, replacements

def _search_price_history_profile(
    prices: PriceData,
    cutoff: int,
    target_date: date,
    baseline_price: np.ndarray,

```

```

        point: PriceSearchPoint,
        training_indices: set[int],
    ) -> tuple[np.ndarray, int]:
        first = max(0, cutoff - point.history_days)
        indices = np.asarray(
            [index for index in range(first, cutoff) if index in training_indices],
            dtype=int,
        )
        if indices.size == 0:
            return baseline_price.copy(), 0
        matrix, replacements = _hampel_filter_price_history(
            prices.price[indices],
            point.hampel_radius,
            point.hampel_threshold,
            point.hampel_strength,
        )
        ages = cutoff - 1 - indices
        weights = np.exp(-math.log(2.0) * ages / point.half_life_days)
        same_weekday = np.asarray(
            [prices.dates[index].weekday() == target_date.weekday() for index in indices],
            dtype=float,
        )
        weights *= 1.0 + point.same_weekday_multiplier * same_weekday
        return np.average(matrix, axis=0, weights=weights), replacements

def _search_price_bases(
    prices: PriceData,
    baseline_price: np.ndarray,
    point: PriceSearchPoint,
    training_indices: set[int],
) -> tuple[np.ndarray, int]:
    hours = np.arange(144, dtype=float) / 6.0
    bases: list[np.ndarray] = []
    replacements = 0
    for cutoff, day in enumerate(prices.dates):
        profile, count = _search_price_history_profile(
            prices, cutoff, day, baseline_price, point, training_indices
        )
        periodic = fit_price_fourier(hours, profile, point.fourier_harmonics)
        wavelet = two.wavelet_correct_residual(profile, periodic, point.wavelet_level)
        bases.append(periodic + wavelet)
        replacements += count
    return np.asarray(bases), replacements

def _search_bias_weight(
    prices: PriceData,
    bases: np.ndarray,
    cutoff: int,
    point: PriceSearchPoint,
    training_indices: set[int],
) -> float:
    pair_end = cutoff - 1
    pair_start = max(1, pair_end - point.bias_days + 1)
    previous: list[np.ndarray] = []
    current: list[np.ndarray] = []
    for index in range(pair_start, pair_end + 1):
        if index in training_indices and index - 1 in training_indices:
            previous.append(prices.price[index - 1] - bases[index - 1])
            current.append(prices.price[index] - bases[index])
    if len(previous) < Config().minimum_bias_pairs:
        return point.default_bias_weight
    x = np.concatenate(previous)
    y = np.concatenate(current)
    denominator = float(np.dot(x, x))
    if denominator <= 1.0e-12:
        return point.default_bias_weight
    return float(np.clip(np.dot(x, y) / denominator, 0.0, 1.0))

def _search_price_forecasts(
    prices: PriceData,
    baseline_price: np.ndarray,
    point: PriceSearchPoint,
    training_indices: set[int],
) -> tuple[np.ndarray, int]:
    hours = np.arange(144, dtype=float) / 6.0
    bases, replacements = _search_price_bases(
        prices, baseline_price, point, training_indices
    )
    forecasts: list[np.ndarray] = []

```

```

for cutoff, day in enumerate(prices.dates):
    profile, _ = _search_price_history_profile(
        prices, cutoff, day, baseline_price, point, training_indices
    )
    periodic = fit_price_fourier(hours, profile, point.fourier_harmonics)
    wavelet = two.wavelet_correct_residual(profile, periodic, point.wavelet_level)
    latest = cutoff - 1
    if latest >= 0 and latest in training_indices:
        bias = prices.price[latest] - bases[latest]
        weight = _search_bias_weight(
            prices, bases, cutoff, point, training_indices
        )
    else:
        bias = np.zeros(144, dtype=float)
        weight = 0.0
    forecasts.append(np.maximum(0.0, periodic + wavelet + weight * bias))
return np.asarray(forecasts), replacements

def _bayes_price_metrics(
    actual: np.ndarray, forecast: np.ndarray
) -> dict[str, float]:
    error = actual - forecast
    denominator = np.maximum(np.abs(actual), 0.05)
    residual = float(np.sum(error**2))
    total = float(np.sum((actual - np.mean(actual)) ** 2))
    return {
        "mae": float(np.mean(np.abs(error))),
        "rmse": float(np.sqrt(np.mean(error**2))),
        "mape": float(np.mean(np.abs(error) / denominator)),
        "r2": float("nan") if total <= 0.0 else 1.0 - residual / total,
    }

def run_price_bayesian_optimization(args: argparse.Namespace) -> None:
    if args.bayes_trials < 2 or not 1 <= args.bayes_initial < args.bayes_trials:
        raise ValueError("应满足 bayes-trials >= 2 且 1 <= bayes-initial < bayes-trials。")
    report_path = args.bayes_report
    csv_path = args.bayes_csv
    if args.bayes_disable_hampel:
        if report_path == DEFAULT_PRICE_BAYES_REPORT:
            report_path = DEFAULT_PRICE_BAYES_NO_HAMPEL_REPORT
        if csv_path == DEFAULT_PRICE_BAYES_CSV:
            csv_path = DEFAULT_PRICE_BAYES_NO_HAMPEL_CSV

    baseline, order = two.read_baseline(args.baseline)
    prices = read_dynamic_prices(args.price, order)
    groups = two.DatasetSplit().cycle_labels(prices.dates)
    training_indices = set(groups["训练集"])
    labels = {
        index: label for label, indices in groups.items() for index in indices
    }
    rng = np.random.default_rng(args.bayes_seed)
    base_point = _default_price_search_point()
    candidates = [base_point]
    base_prediction, base_replacements = _search_price_forecasts(
        prices, baseline.price, base_point, training_indices
    )
    predictions = [base_prediction]
    replacement_counts = [base_replacements]
    seen = {_price_point_key(base_point)}
    validation_seen: list[int] = []
    current_index = 0
    forward_prediction: list[np.ndarray] = []
    selection_history: list[dict[str, object]] = []
    proposals: list[dict[str, object]] = []
    proposal_count = 1
    no_improvement = 0
    search_stopped = False
    print("严格前向电价贝叶斯优化: 测试集不参与参数搜索。")
    for index, day in enumerate(prices.dates):
        forward_prediction.append(predictions[current_index][index])
        if labels[index] != "验证集":
            continue
        validation_seen.append(index)
        actual_seen = prices.price[validation_seen]
        scores = np.asarray(
            [
                _bayes_price_metrics(actual_seen, prediction[validation_seen])["mape"]
                for prediction in predictions
            ]
        )

```

```

    ]
)
incumbent = float(np.min(scores))
if proposal_count < args.bayes_trials and not search_stopped:
    dimensions = 7 if args.bayes_disable_hampel else 10
    if len(candidates) < args.bayes_initial:
        unit = rng.random(dimensions)
        method = "随机初始化"
    else:
        pool = rng.random((args.bayes_candidate_pool, dimensions))
        acquisition = two._expected_improvement(
            np.asarray(
                [
                    _encode_price_point(point, args.bayes_disable_hampel)
                    for point in candidates
                ]
            ),
            scores,
            pool,
            0.42,
        )
        unit = pool[int(np.argmax(acquisition))]
        method = "贝叶斯期望改进"
    point = _decode_price_point(unit, args.bayes_disable_hampel)
    attempts = 0
    while _price_point_key(point) in seen and attempts < 100:
        point = _decode_price_point(
            rng.random(dimensions), args.bayes_disable_hampel
        )
        attempts += 1
    seen.add(_price_point_key(point))
    proposal_count += 1
    prediction, replacement_count = _search_price_forecasts(
        prices, baseline.price, point, training_indices
    )
    candidate_score = _bayes_price_metrics(
        actual_seen, prediction[validation_seen]
    )["mape"]
    improved = candidate_score < incumbent - args.bayes_min_delta
    candidates.append(point)
    predictions.append(prediction)
    replacement_counts.append(replacement_count)
    no_improvement = 0 if improved else no_improvement + 1
    proposal_count >= args.bayes_initial
)
proposals.append(
    {
        "proposal": proposal_count,
        "validation_date": day.isoformat(),
        "validation_days_available": len(validation_seen),
        "method": method,
        "validation_mape": candidate_score,
        "improved": improved,
        "hampel_replacements_in_all_causal_windows": replacement_count,
        "parameters": asdict(point),
    }
)
print(f"[{day}] 累计验证 MAPE={candidate_score:.4%}")
if len(candidates) >= args.bayes_initial and no_improvement >= args.bayes_patience:
    search_stopped = True
    print(f"连续 {args.bayes_patience} 个候选无显著改善，停止寻优。")

scores = np.asarray(
    [
        _bayes_price_metrics(actual_seen, prediction[validation_seen])["mape"]
        for prediction in predictions
    ]
)
selected = int(np.argmin(scores))
if len(validation_seen) >= args.bayes_initial:
    current_index = selected
    selection_history.append(
        {
            "date": day.isoformat(),
            "validation_days_available": len(validation_seen),
            "selected_candidate": current_index,
            "selected_validation_mape": float(scores[current_index]),
            "selected_parameters": asdict(candidates[current_index]),
        }
    )
)

```

```

forward = np.asarray(forward_prediction)
baseline_metrics = {
    label: _bayes_price_metrics(prices.price[indices], base_prediction[indices])
    for label, indices in groups.items()
}
forward_metrics = {
    label: _bayes_price_metrics(prices.price[indices], forward[indices])
    for label, indices in groups.items()
}
report = {
    "data_split": {
        "rule": "训练、训练、训练、验证、训练、测试",
        "counts": {label: len(indices) for label, indices in groups.items()},
        "strict_training_freeze": True,
        "test_data_used_in_optimization": False,
    },
    "optimization": {
        "method": "严格前向高斯过程贝叶斯优化 + 早停",
        "hampel_enabled": not args.bayes_disable_hampel,
        "objective": "截至当前日期已开放验证集的累计 MAPE",
        "maximum_proposals": args.bayes_trials,
        "completed_proposals": proposal_count,
        "patience": args.bayes_patience,
        "minimum_absolute_mape_improvement": args.bayes_min_delta,
        "search_early_stopped": search_stopped,
        "seed": args.bayes_seed,
    },
    "baseline_formal_parameters_metrics": baseline_metrics,
    "strict_forward_experiment_metrics": forward_metrics,
    "year_end_selected_parameters": asdict(candidates[current_index]),
    "year_end_hampel_replacements_in_all_causal_windows": replacement_counts[current_index],
    "proposals": proposals,
    "selection_history": selection_history,
}
report_path.parent.mkdir(parents=True, exist_ok=True)
report_path.write_text(
    json.dumps(report, ensure_ascii=False, indent=2), encoding="utf-8"
)
csv_path.parent.mkdir(parents=True, exist_ok=True)
with csv_path.open("w", encoding="utf-8-sig", newline="") as file:
    writer = csv.writer(file, lineterminator="\n")
    writer.writerow(
        [
            "日期",
            "数据集",
            "实际均价",
            "正式模型预测均价",
            "实验模型预测均价",
            "正式模型 MAPE",
            "实验模型 MAPE",
        ]
    )
    for index, day in enumerate(prices.dates):
        actual = prices.price[index]
        base = base_prediction[index]
        experimental = forward[index]
        denominator = np.maximum(np.abs(actual), 0.05)
        writer.writerow(
            [
                day.isoformat(),
                labels[index],
                f"{np.mean(actual):.8f}",
                f"{np.mean(base):.8f}",
                f"{np.mean(experimental):.8f}",
                f"{np.mean(np.abs(actual - base) / denominator):.8f}",
                f"{np.mean(np.abs(actual - experimental) / denominator):.8f}",
            ]
        )
    for label in ("训练集", "验证集", "测试集"):
        values = forward_metrics[label]
        print(
            f"{label}: MAE={values['mae']:.6f}, "
            f"RMSE={values['rmse']:.6f}, MAPE={values['mape']:.4%}"
        )
print(f"实验报告: {report_path.resolve()}")
print(f"逐日评价: {csv_path.resolve()}")

def parse_args() -> argparse.Namespace:
    parser = argparse.ArgumentParser(

```

```

        description="C 问题四傅里叶一小波动态电价预测与 LP/MPC 购电策略"
    )
    parser.add_argument("--baseline", type=Path, default=two.DEFAULT_BASELINE)
    parser.add_argument("--load", type=Path, default=two.DEFAULT_LOAD)
    parser.add_argument("--pv", type=Path, default=two.DEFAULT_PV)
    parser.add_argument("--forecast", type=Path, default=DEFAULT_FORECAST)
    parser.add_argument("--price", type=Path, default=DEFAULT_PRICE)
    parser.add_argument("--template42", type=Path, default=DEFAULT_TEMPLATE_42)
    parser.add_argument("--template43", type=Path, default=DEFAULT_TEMPLATE_43)
    parser.add_argument("--output42", type=Path, default=DEFAULT_OUTPUT_42)
    parser.add_argument("--output43", type=Path, default=DEFAULT_OUTPUT_43)
    parser.add_argument("--summary42", type=Path, default=DEFAULT_SUMMARY_42)
    parser.add_argument("--summary43", type=Path, default=DEFAULT_SUMMARY_43)
    parser.add_argument("--comparison", type=Path, default=DEFAULT_COMPARISON)
    parser.add_argument(
        "--price-evaluation", type=Path, default=DEFAULT_PRICE_EVALUATION
    )
    parser.add_argument("--split-report", type=Path, default=DEFAULT_SPLIT_REPORT)
    parser.add_argument("--train-end", type=date.fromisoformat, default=date(2025, 7, 31))
    parser.add_argument(
        "--validation-end", type=date.fromisoformat, default=date(2025, 9, 30)
    )
    parser.add_argument(
        "--skip-workbooks",
        action="store_true",
        help="只生成 CSV 与训练/验证/测试检验报告, 不重写 Excel 结果表。",
    )
    parser.add_argument(
        "--bayes-optimize",
        action="store_true",
        help="执行严格前向电价贝叶斯寻优, 不运行 4-2 和 4-3 正式调度。",
    )
    parser.add_argument("--bayes-trials", type=int, default=36)
    parser.add_argument("--bayes-initial", type=int, default=10)
    parser.add_argument("--bayes-patience", type=int, default=10)
    parser.add_argument("--bayes-min-delta", type=float, default=0.0005)
    parser.add_argument("--bayes-candidate-pool", type=int, default=2500)
    parser.add_argument("--bayes-seed", type=int, default=2026)
    parser.add_argument("--bayes-report", type=Path, default=DEFAULT_PRICE_BAYES_REPORT)
    parser.add_argument("--bayes-csv", type=Path, default=DEFAULT_PRICE_BAYES_CSV)
    parser.add_argument(
        "--bayes-disable-hampel",
        action="store_true",
        help="执行关闭 Hampel 滤波的贝叶斯消融实验。",
    )
    return parser.parse_args()

def main() -> None:
    args = parse_args()
    if args.bayes_optimize:
        run_price_bayesian_optimization(args)
        return
    cfg = Config()
    baseline, chronological_order = two.read_baseline(args.baseline)
    historical = two.read_historical_data(args.load, args.pv, chronological_order)
    prices = read_dynamic_prices(args.price, chronological_order)
    if prices.dates != historical.dates:
        raise ValueError("附件 2 与附件 4 的日期不一致。")
    forecasts = three.read_forecasts(args.forecast)
    split = two.DatasetSplit(args.train_end, args.validation_end)
    groups = split.cycle_labels(historical.dates)
    training_indices = set(groups["训练集"])

    print("正在构造傅里叶一小波一前日偏差动态电价预测……")
    current_prices, following_prices = prepare_daily_price_forecasts(
        prices,
        baseline.price,
        cfg,
        training_indices=training_indices,
    )
    print("正在计算问题 4-2……")
    results42 = simulate_42(
        baseline,
        historical,
        prices,
        current_prices,
        following_prices,
        cfg,
        training_indices=training_indices,
    )

```

```

validate_42(results42, cfg)

print("正在构造问题 4-3 负荷、光伏决策输入.....")
decision_inputs = three.prepare_decision_inputs(
    baseline,
    historical,
    forecasts,
    cfg,
    training_indices=training_indices,
)
strategy_specs = [
    ("S0_仅 0 点", (0,)),
    ("S1_增加 6 点", (0, 6)),
    ("S2_增加 12 点", (0, 6, 12)),
    ("S3_完整 MPC", (0, 6, 12, 18)),
]
summaries43: list[three.StrategySummary] = []
for name, release_hours in strategy_specs:
    print(f"正在计算问题 4-3 策略 {name}.....")
    summary = simulate_43_strategy(
        name,
        release_hours,
        historical,
        prices,
        decision_inputs,
        current_prices,
        following_prices,
        cfg,
    )
    three.validate_strategy(summary, cfg)
    summaries43.append(summary)
selected43 = three.select_strategy_on_validation(summaries43, split)

for path in (
    args.summary42,
    args.summary43,
    args.comparison,
    args.price_evaluation,
    args.split_report,
):
    path.parent.mkdir(parents=True, exist_ok=True)
write_csv_outputs(
    results42,
    summaries43,
    selected43,
    prices,
    current_prices,
    args,
)
write_split_evaluation(
    args.split_report,
    prices,
    current_prices,
    results42,
    summaries43,
    selected43,
    split,
)
if not args.skip_workbooks:
    payload = make_workbook_payload(
        results42, selected43, chronological_order, cfg
    )
    write_workbooks(payload, args)
print_summary(
    results42,
    summaries43,
    selected43,
    prices,
    current_prices,
    args,
)

if __name__ == "__main__":
    main()

```

C.5 依赖文件 requirements.txt

程序说明：记录完整程序复现所需的第三方 Python 库版本。

```
numpy==2.5.3  
scipy==1.18.1  
openpyxl==3.1.5
```